

**SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
VARAŽDIN**

Zlatko Pračić

**IZRADA AGENTA S GENERATIVNOM
UMJETNOM INTELIGENCIJOM TEKSTA
SPECIFIČNE NAMJENE**

DIPLOMSKI RAD

Varaždin, 2026.

SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
V A R A Ž D I N

Zlatko Pračić

Matični broj: 0016145097

Studij: Baze podataka i baze znanja

IZRADA AGENTA S GENERATIVNOM UMJETNOM INTELIGENCIJOM
TEKSTA SPECIFIČNE NAMJENE

DIPLOMSKI RAD

Mentor:

Prof. dr. sc. Markus Schatten

Varaždin, rujan 2026.

Izjava o izvornosti

Izjavljujem kako je ovaj diplomski rad izvorni rezultat mog rada te da se u izradi istoga nisam koristio drugim izvorima osim onima koji su u njemu navedeni. Za izradu rada su korištene etički prikladne i prihvatljive metode i tehnike rada.

Autor potvrdio prihvatanjem odredbi u sustavu FOI Radovi

Sažetak

Opsega od 100 do 300 riječi. Sažetak upućuje na temu rada, ukratko se iznosi čime se rad bavi, teorijsko-metodološka polazišta, glavne teze i smjer rada te zaključci.

Ključne riječi: riječ; riječ; riječ; Obuhvaća 7 ± 2 ključna pojma koji su glavni predmet rasprave u radu.

Sadržaj

1. Uvod	1
2. Povijest Chatbotova	2
2.1. Teorijske i matematičke pretpostavke	2
2.1.1. Markovljev lanac	2
2.1.2. Turingov test	3
2.2. Prva generacija - tekstualni chatbotovi bazirani na pravilima	3
2.2.1. ELIZA	3
2.2.2. PARRY	4
2.2.3. Racter	5
2.3. Druga generacija - skriptni jezici	5
2.3.1. Mr. Spock u TINYMUD okruženju	6
2.3.2. ALICE i AIML	6
2.4. Treća generacija - komercijalni i praktični chatboti	6
2.4.1. SmarterChild	7
2.4.2. Web i društvene mreže	7
2.5. Četvrta generacija - glasovni asistenti i konverzacijska sučelja	7
2.5.1. Siri, Google Assistant, Cortana, Alexa	8
2.6. Peta generacija - duboko učenje, transformeri i LLM chatbotovi	8
2.6.1. Sekvencijski modeli	8
2.6.2. Transformer arhitektura	9
2.6.3. Veliki jezični modeli i generativni chatboti (ChatGPT, Bard ...)	9
3. AI inženjerstvo - koncepti i ključne tehnike	10
3.1. Pojam i uloga AI inženjerstva	10
3.2. Temeljni modeli i veliki jezični modeli (LLM)	10
3.2.1. Temeljni modeli	10
3.2.2. Veliki jezični modeli (LLM)	11
3.3. Tehničke osnove: transformer i <i>attention</i>	12
3.3.1. Transformer arhitektura	12
3.3.2. <i>Attention</i> mehanizam	13
3.4. Rad s LLM-ovima u aplikacijama: tokeni, kontekst i kontrola generiranja	13
3.4.1. Tokeni i tokenizacija	13
3.4.2. Kontekst modela	14
3.4.3. Parametri generiranja: <i>temperature</i> , top-k i top-p	15
3.5. Prilagodba ponašanja bez treniranja: prompt engineering i <i>in-context</i> učenje	16

3.5.1.	Prompt engineering	17
3.5.2.	Sistemi i korisnički prompt	18
3.5.3.	<i>In-context</i> learning	18
3.6.	Prilagodba modela treniranjem i optimizacija primjene	19
3.6.1.	Fino podešavanje i PEFT	19
3.6.2.	Preference fine-tuning	20
3.6.3.	Kvantizacija i destilacija	21
3.7.	RAG: povezivanje modela s vanjskim znanjem	21
3.7.1.	Osnovna ideja RAG-a	22
3.7.2.	Vektorska reprezentacija i vektorske baze podataka	22
3.7.3.	Aproksimativno pretraživanje najbližih susjeda	23
3.7.4.	Segmentacija dokumenata	24
3.7.5.	Optimizacija upita i ponovno rangiranje rezultata	25
3.8.	GraphRAG	26
4.	Opis izrade	27
4.1.	Opis domene i zahtjevi	27
4.2.	Korištene programske biblioteke	28
4.2.1.	Osnovni model bez dohvata konteksta (<i>Vanilla</i>)	28
4.2.2.	Retrieval-Augmented Generation (RAG)	29
4.2.3.	GraphRAG	29
4.3.	Zajednička implementacijska osnova triju verzija chatbota	30
4.3.1.	Temeljni jezični model i konfiguracija kvantizacije	30
4.3.2.	Okolina izvođenja	31
4.3.3.	Sistemi upit (<i>system prompt</i>)	33
4.3.4.	Postupak generiranja odgovora	33
4.3.5.	Skup podataka za evaluaciju i postupak paketne obrade	34
4.3.6.	Evaluacija rezultata	35
4.4.	Vanilla verzija chatbota	35
4.4.1.	Funkcija <code>vanilla_chat</code>	35
4.4.2.	Izlazni podaci	36
4.5.	RAG verzija chatbota	36
4.5.1.	Model za generiranje vektorskih reprezentacija	37
4.5.2.	Priprema korpusa dokumenata	38
4.5.3.	Vektorski indeks	38
4.5.4.	Postupak dohvata relevantnih odlomaka	39
4.5.5.	Integracija dohvata u generiranje odgovora	39
4.5.6.	Prilagodba parametara prema kategoriji pitanja	40
4.5.7.	Funkcija <code>rag_chat</code> i izlazni podaci	40
4.6.	GraphRAG verzija chatbota	41
4.6.1.	Graf baza podataka i shema	41

4.6.2.	Izgradnja grafa iz dokumenata	42
4.6.3.	Automatsko izvođenje tematskih veza	43
4.6.4.	Postupak dohvata konteksta	44
4.6.5.	Integracija u generiranje odgovora	45
4.6.6.	Alati za dijagnostiku grafa	45
4.6.7.	Izlazni podaci	46
5.	Evaluacija chatbotova	48
5.1.	Operativni aspekti: inferenca i latencija	48
5.1.1.	Online i batch inferenca	48
5.1.2.	Latencija	48
5.2.	Automatske metrike	49
5.2.1.	Leksičke metrike	49
5.2.1.1.	ROUGE-L	49
5.2.2.	Semantičke metrike	50
5.2.2.1.	BERTScore	50
5.2.2.2.	Kosinusna sličnost (Cosine Similarity)	50
5.2.3.	Implementacija izračuna metrika	50
5.2.4.	Ograničenja automatskih metrika u pravnoj domeni	51
5.3.	Ljudska evaluacija	52
5.3.1.	Evaluatori i dimenzije ocjenjivanja	52
5.3.2.	Google Obrasci	53
5.3.3.	Međuocjenjivačka pouzdanost	53
5.4.	LLM-as-a-Judge	53
5.4.1.	Pristup i motivacija	54
5.4.2.	Implementacija i dizajn prompta	54
5.4.3.	Poznata ograničenja	54
5.5.	Obrazloženje odabira kombinacije metoda	55
6.	Statistička analiza rezultata evaluacije	56
6.1.	Pregled prikupljenih podataka	56
6.2.	Agregacija i vizualizacija automatskih metrika	57
6.3.	Usporedba konfiguracija	60
6.4.	Korelacija automatskih metrika s ljudskom evaluacijom	61
6.5.	Međuocjenjivačka pouzdanost	62
6.6.	LLM-as-a-Judge: Claude i Gemini	63
6.6.1.	Korelacija s ljudskom evaluacijom	63
6.6.2.	Međusobna usporedba: Claude vs. Gemini	64
6.7.	Latencija	65
6.8.	Sinteza rezultata	66
7.	Iterativni razvoj i dijagnostika sustava	68

7.1. Evolucija RAG sustava kroz iterativno poboljšavanje	68
7.2. Specifični tehnički izazovi i njihova rješenja	68
7.3. Eksperimentiranje s izborom osnovnog modela: EuroLLM naspram Qwen2.5	68
7.4. Optimizacija generacijskih parametara i njihov utjecaj	70
7.5. Zašto GraphRAG nije briljirao?	71
8. Odraditi?	72
9. Zaključak	73
Popis literature	76
Popis slika	77
Popis tablica	78
1. Prilog 1 - Popis pitanja	80
2. Prilog 2 - Colab - Vanilla verzija ChatBota	81
3. Prilog 3 - Colab - RAG verzija ChatBota	82
4. Prilog 4 - Colab - GraphRAG verzija ChatBota	83

1. Uvod

"We are drowning in information but starved for knowledge." [1]

– John Naisbitt, Megatrends

Uvod u temu rada, motivacija, problematika, cilj i svrha rada.

2. Povijest Chatbotova

Popularnost chatbotova naglo je porasla predstavljanjem ChatGPT-a 2022. godine. Iz te se činjenice može zaključiti kako su chatbotovi postojali i prije ove nagle popularizacije. Stoga će se u ovom poglavlju prikazati povijesni razvoj chatbota, kao i teorijska osnova koja je utjecala na razvoj ideje o stroju koji razgovara.

2.1. Teorijske i matematičke pretpostavke

Iako je Andrey Markov uvođenjem Markovljevih lanaca u svom radu postavio važan temelj, ideja njegovog rada nije bila ni približna konceptu “stroja koji razgovara”. Vrijednost modela Markovljevog lanca za kasnija otkrića i razvoj znanstvene misli je neupitna.

Alan Turing promišlja o budućnosti strojeva čije su se sposobnosti ubrzano razvijale i postajale sve složenije. Zanimalo ga je može li stroj tijekom komunikacije uvjeriti ispitivača da nije stroj, što se u nastavku teksta formalizira u obliku Turingova testa. Iako se u tom kontekstu izravno ne spominju chatbotovi, upravo se na njih ovaj test u velikoj mjeri može primijeniti.

2.1.1. Markovljev lanac

Al-Amin i ostali u svom radu predstavljaju Markovljev lanac kao jednu od najranijih metoda koja je pokazala potencijal chatbota kroz generiranje teksta na temelju statističkih vjerojatnosti slijeda riječi. Markovljev lanac je razvio Andrey Markov 1906. godine za modeliranje stohastičkih procesa, a kasnije se počeo koristiti u zadacima poput predviđanja sljedeće riječi (npr. *autocomplete* u e-pošti). U chatbot kontekstu, Markovljev lanac procjenjuje vjerojatnost da određena riječ slijedi drugu i na temelju unaprijed naučenih prijelaznih vjerojatnosti “slaže” odgovor. Kao rana primjena Markovljeva lanca navodi se HeX, koji je osvojio Loebnerovu nagradu 1997., nakon čega je interes za ovu tehniku postupno rastao. Inače, Loebnerova nagrada je bila godišnje natjecanje (1990.–2020.) u kojem su se chatbotovi natjecali u varijanti Turingovog testa, suci su kroz tekstualni razgovor pokušavali razlikovati program od stvarne osobe, a najuvjerljiviji program dobivao je nagradu [2].

Klasični Markovljevi modeli opisuju prelazak sustava između različitih stanja kroz vrijeme, pri čemu se prijelazne vjerojatnosti procjenjuju iz empirijskih podataka i pretpostavlja se da su stabilne kroz susjedne vremenske intervale. Prostorni Markovljevi modeli proširuju tu ideju na prostorne jedinice (npr. namjene zemljišta) te uključuju prostornu ovisnost između susjednih lokacija, a dodatna proširenja omogućila su i višedimenzionalne simulacije [2].

Unatoč jednostavnosti implementacije, probabilistički pristup zasnovan na Markovljevim lancima pati od ozbiljnih ograničenja u kontekstu prirodnog dijaloga. Budući da modeli predviđanja obuhvaćaju tek neposredno prethodne tokene (jednu ili nekoliko riječi), izostanak šireg kontekstualnog prozora onemogućuje zadržavanje dugoročne strukture rečenice i smisla razgovora. Ova je tehnika povijesno redefinirala pristup obradi jezika, trasirajući put od strogo determinističkih pravila prema generativnim modelima [2].

2.1.2. Turingov test

Alan Turing svojim radom tvrdi kako je pitanje “Mogu li strojevi misliti?” nejasno pa ga zamjenjuje praktičnijim kriterijem: imitacijskom igrom (u nastavku kaže kako je to “test”), gdje ispitivač preko pisanih poruka pokušava razlikovati čovjeka od stroja. Fokusira se na digitalna računala, objašnjava njihovu građu (memorija, izvršna jedinica, kontrola/program) i naglašava njihovu univerzalnost. Jedno dovoljno snažno računalo može oponašati ponašanje bilo kojeg stroja s diskretnim stanjima ako je dobro programirano. Predviđa kako bi se u nekoliko desetljeća mogla pojaviti računala koja bi u takvoj igri bila dovoljno uvjerljiva u “razgovoru” s prosječnim ispitivačem [3].

Velik dio rada posvećen je odgovorima na prigovore kako strojevi ne mogu misliti: teološke tvrdnje o “duši”, strah od posljedica, matematička ograničenja, argument iz svijesti i razne “nemogućnosti” tipa da stroj ne može biti kreativan, griješiti ili učiti. Turing uglavnom pokazuje da ti prigovori ne dokazuju nemogućnost, često miješaju kvarove s pogrešnim zaključivanjem ili se oslanjaju na tadašnja ograničenja tehnologije. Kao pozitivni smjer predlaže da se ne programira “odrasli um” direktno, nego da se napravi “dječji stroj” koji se zatim uči i odgaja (nagrade/kazne, jezik naredbi, moguće i slučajnost), analogno procesu učenja i evolucije [3].

Al-Amin kaže kako se Turingov test može smatrati modelskom pretečom chatbota jer je već 1950. postavio razgovor u prirodnom jeziku kao ključni način provjere “inteligentnog” ponašanja mašine, kroz dijalog u kojem treba biti teško razlikovati mašinu od čovjeka. Evaluacija se zasniva na tome može li računalo “prevariti” ljudskog ispitivača da pomisli kako razgovara s čovjekom, što je isti osnovni cilj konverzacijskih sistema poput chatbota. Također, kao utjecajan istraživački kamen temeljac, Turingov test je godinama i desetljećima motivirao razvoj obrade prirodnog jezika, čime je direktno usmjerio put prema modernim chatbot tehnologijama [2].

2.2. Prva generacija - tekstualni chatbotovi bazirani na pravilima

Prva generacija chatbotova pojavila se između sredine 1960-ih i ranih 1980-ih godina. Tada su programeri prvi put pokušali svoje teorijske ideje prevesti u stvarne razgovore. Ovi su sustavi tehnički funkcionirali tako što su prepoznavali unaprijed definirane tekstualne obrasce i primjenjivali strogo definirana, ručno napisana pravila. Nisu stvarno razumjeli što tekstovi znače ili u kojem kontekstu stoje. Sustav je tražio važne riječi u poruci osobe koja ga koristi i odgovarao unaprijed definiranim frazama. To je zvučalo kao pravi razgovor, ali zapravo nije bilo. Unutar usko definiranih područja, ovi su chatbotovi postavili temelje za to kako ljudi i računala komuniciraju. Također su pokazali kako ljudi imaju sklonost da strojevima daju ljudske osobine.

2.2.1. ELIZA

Lane i ostali u svom radu detaljno obrađuju ELIZA, chatbot kojeg je Joseph Weizenbaum razvio na MIT-u sredinom 1960-ih godina. Po tom istraživanju, ELIZA predstavlja prvi chatbot na svijetu, nastao na prvom *time-sharing* sustavu CTSS (*Compatible Time-Sharing*

```

EEEEEE LL      IIII ZZZZZZ  AAAA
EE      LL      II      ZZ  AA  AA
EEEEEE LL      II      ZZZ  AAAAAA
EE      LL      II      ZZ  AA  AA
EEEEEE LLLLLL IIII ZZZZZZ  AA  AA

Eliza is a mock Rogerian psychotherapist.
The original program was described by Joseph Weizenbaum in 1966.
This implementation by Norbert Landsteiner 2005.

ELIZA: Is something troubling you ?
YOU:   Yes. I'm trying to describe you for a research paper I'm writing. What sh
ould I say?
ELIZA: You are sure.
YOU:   Yes, I'm sure.
ELIZA: I see.
YOU:   What can I tell them about you?
ELIZA: You seem to need to tell them about me.
YOU:   I absolutely need to. Please help.
ELIZA: That's quite interesting.
YOU:   I agree.
ELIZA: You say you agree ?
YOU:

```

Slika 1: Originalni ispis koda ELIZA pronađen u Weizenbaumovim arhivama [5]

System) na IBM 7094 računalu. Rad opisuje kako je tim istraživača 2021. otkrio originalni ispis koda ELIZA (Slika 1) u Weizenbaumovim arhivama na MIT-u, uključujući ranu verziju čuvenog DOCTOR skripta te gotovo potpunu verziju MAD-SLIP koda, kombinacije programskog jezika MAD i Weizenbaumovog vlastitog sustava SLIP (*Symmetric Lisp Processor*). Tim je uspješno “vratio na život” originalnu ELIZA na emuliranom IBM 7094 sustavu u prosincu 2024., čime je po prvi put nakon više od 60 godina omogućeno pokretanje autentičnog originalnog koda. Bit ovog postignuća nije samo nostalgija, rekonstrukcija je omogućila provjeru što originalna ELIZA doista jest, potvrđujući kako je bila funkcionalan, iako ograničen sustav koji je pomogao pokrenuti cijelo područje konverzacijske umjetne inteligencije [4].

Analizom restauriranog koda otkriveno je nekoliko aspekata koji ranije nisu bili poznati. Pronađena verzija ima implementiran potpuni “*teaching mode*” (način podučavanja), spominjan u Weizenbaumovom radu iz 1966. samo jednom rečenicom, ali nikada detaljno opisan. Korisnici su mogli upisivanjem naredbi poput ADD, SUBST, APPEND, RANK, TYPE, DISPLA i START mijenjati pravila razgovora i spremati nove skripte. Također, otkrivena je značajna greška: ELIZA se “srušila” kada bi primila numerički unos (npr. “*you are 999 today*”), što autori tumače kao ironičan obrat uvida Ade Lovelace da računala mogu “djelovati na stvari koje nisu brojevi”. Važno je da tim nije popravio takve greške, ostavili su kod autentičnim, točno 96% onakav kakav je pronađen, kako bi očuvali arheološku vrijednost nalaza. Time ELIZA postaje ne samo povijesni artefakt već i upozorenje da su mnogi kasniji opisi (npr. da je ELIZA bila napisana u Lispu) bili pogrešni, originalna je bila u MAD-SLIP jeziku, Lisp verzija nastala je kasnije kao klon koji je napisao Bernie Cosell, i koja se zatim proširila preko ARPANeta [4].

2.2.2. PARRY

AI-Amin u svom radu obrađuju PARRY chatbot koji je 1972. razvio psihijatar Kenneth Colby na Stanfordu i predstavlja suprotnost ELIZI. Umjesto da glumi terapeuta, PARRY glumi paranoidnog shizofrenog pacijenta kako bi odvuкао pažnju sa sebe i potaknuo korisnika na dugačke, razrađene odgovore. PARRY je služio i kao alat za obuku psihijataru (vježbanje raz-

govora s pacijentima), ali i kao operativni model Colbyjeve teorije paranoje, tj. demonstracija kako “poremećena interpretacija znakova” može proizvoditi prepoznatljive obrasce paranoidnog govora i ponašanja [2].

PARRY je jedan od prvih chatbotova koji se evaluirao kroz oblik “Turing testa” radi procjene koliko uvjerljivo može oponašati ljudski razgovor. Ta linija evaluacije kasnije se nastavlja kroz natjecanja poput Loebner Prizea, gdje se botovi ocjenjuju po “prirodnosti” nakon kratkih razgovora. Poanta u tekstu je da su PARRY (i kasnije Loebner Prize) dio ranih pokušaja mjerenja konverzacijske “inteligencije” chatbotova prema tome koliko dobro simuliraju prirodan ljudski dijalog [2].

2.2.3. Racter

Racter (1983), kojeg su izradili Chamberlain i Etter, u tekstu se opisuje kao pionirski chatbot koji se posebno istaknuo po nasumičnom generiranju novog teksta, a ne samo biranju odgovora iz predložaka. Racter (skraćeno od *raconteur*, pripovjedač) mogao je tijekom razgovora stvarati jedinstvene rečenice i prozne ulomke koristeći pravila kontekstno-slobodne gramatike (*context-free grammar*) i element slučajnosti. Rad naglašava kako je rezultat često bio besmislen jer nije postojalo “pravo razumijevanje”, ali da je sama proceduralna sposobnost generiranja originalnog teksta bila iznimno napredna i utjecajna za to vrijeme [2].

Posebno se ističe kako je autor povezan s Racterom (Chamberlain) objavio knjigu *The Policeman’s Beard is Half Constructed* (1984), koja se predstavlja kao knjiga “autorski” proizvedena Racterovim generativnim mogućnostima, čime je dobio širu kulturnu vidljivost. Tekst napominje kako nije potpuno jasno u kojoj je mjeri proza doista bila u potpunosti računalno generirana, no važna je poruka da je Racter popularizirao ideju chatbotova koji mogu proizvoditi nov i originalan razgovorni sadržaj. Rad govori o tome kako pojedini Racterovi fragmenti zvuče introspektivno i “ljudski” (npr. o snovima i željama), što je otvorilo pitanja o humanizaciji i o tome kako AI može imitirati emocionalni i egzistencijalni izraz iako nema ljudsko iskustvo [2].

2.3. Druga generacija - skriptni jezici

Nakon pionirskih sustava poput ELIZA, razvoj chatbotova ušao je u fazu dominacije skriptnih jezika i naprednog uparivanja uzoraka, što se može smatrati njihovom drugom generacijom. Umjesto bazičnih pravila prve generacije, ovi sustavi uvode strukturirane skriptne jezike poput AIML-a, RiveScripta i ChatScripta. Njihova primjena omogućila je izgradnju opsežnijih baza znanja, čime je olakšano prepoznavanje konteksta i upravljanje kompleksnijim varijacijama korisničkih upita. Iako su se i dalje temeljili na unaprijed definiranim predlošcima bez elemenata strojnog učenja, ovi su jezici značajno podigli prag interaktivnosti [6].

2.3.1. Mr. Spock u TINYMUD okruženju

Godine 1989. James Aspnes s Carnegie Mellon University razvio je TINYMUD (*multi-player real-time virtual world*), jedan od prvih višekorisničkih tekstualnih virtualnih svjetova. Na tom sustavu pojavio se jedan od prvih chatbot likova, Mr. Spock, koji je mogao odgovarati na osnovna konverzacijska pitanja korisnika koristeći jednostavno podudaranje uzoraka [2].

Mr. Spock predstavljao je rani pokušaj stvaranja umjetnog lika koji može razgovarati s ljudima unutar virtualnog okruženja te je prethodio pojmu NPC (*non-player character*) botova koji su danas uobičajeni u igrama i virtualnim svjetovima [2].

2.3.2. ALICE i AIML

Sljedeći veći korak u povijesti chatbotova je ALICE (*Artificial Linguistic Internet Computer Entity*), nastao 1995. kao *online* chatbot inspiriran programom ELIZA. ALICE je bio temeljen na slaganju uzoraka, bez stvarnog razumijevanja cjelokupnog razgovora [2], [6].

Iako je kasnije poboljšavan i osvojio Loebner Prize za “najhumaniji” računalni program tri puta (2000., 2001. i 2004.), ALICE nije uspio proći puni Turingov test [2].

Eleni naglašava kako je ključna razlika između ALICE i programa ELIZA to što je ALICE razvijen uz novi jezik napravljen upravo za tu svrhu: AIML (*Artificial Intelligence Markup Language*), koji se temelji na XML-u i organiziran je oko kategorija koje se sastoje od uzorka i predloška [6].

Autori kažu kako je AIML nastao u razdoblju 1995–2000 za izgradnju baze znanja chatbotova koji koriste pristup slaganja uzoraka. AIML je baziran na XML-u, a temeljna jedinica znanja je kategorija koja se sastoji od uzorka (*pattern* - korisnički unos) i predloška (*template* - odgovor bota) [6]. Kategorije mogu biti grupirane unutar tema (*topics*), a sve se organizira u strukturu nazvanu Graphmaster koja omogućuje učinkovito uparivanje uzoraka pretraživanjem u dubinu (*depth-first search*) [7].

2.4. Treća generacija - komercijalni i praktični chatboti

Chatbotovi su prošli dug put, od Weizenbaumove ELIZA do asistenata ugrađenih u svaki pametni telefon. Ključni trenutak u toj evoluciji bio je prijelaz s izoliranih programa na masovne komunikacijske platforme, čime su chatbotovi postali dostupni širokom krugu korisnika u stvarnom vremenu. Kroz primjere poput pionirskog SmarterChilda i suvremenih rješenja na platformama kao što su Facebook Messenger, WhatsApp i WeChat, može se pratiti kako su ovi sustavi mijenjali način na koji se dobivaju informacije, komunicira s brendovima i koriste društvene mreže [2] [6].

2.4.1. SmarterChild

SmarterChild je chatbot koji je razvila tvrtka ActiveBuddy 2001. godine i smatra se jednim od ranih pionira koji je chatbotove približio širokoj publici. Al-Amin u svom radu navodi kako je chatbot radio unutar *instant messaging* platformi i mogao je davati informacije o raznim temama na upit, što je pokazalo kako chatbot može funkcionirati kao svojevrsni “virtualni asistent” koji pronalazi činjenice i odgovara na korisnička pitanja [2].

Al-Amin ističe kako je posebno važno da se SmarterChild mogao dodati u MSN Messenger i AOL Instant Messenger te je dosegao više od 30 milijuna korisnika. Time je “pionirski” otvorio put chatbotovima u mrežama za dopisivanje i kasnijim asistentima poput Appleova Sirija i Samsungova S Voicea, jer je demonstrirao potencijal pružanja usluga izravno kroz razgovorno sučelje u obliku poruka. Također je pokazao praktičnu vrijednost (npr. dohvat vijesti, vremena i burzovnih podataka iz baza) pa je pristup informacijama kroz razgovor umjesto kroz web-stranice označen kao važan korak prema AI sustavima koji služe ljudima putem dijaloga [2].

2.4.2. Web i društvene mreže

Chatbotovi na društvenim mrežama naglo su se proširili paralelno s rastom interneta i društvenih platformi, gdje mogu imati različite uloge: korisničku podršku, marketing i oglašavanje, zabavu te prikupljanje podataka. U radu Al-Amina se također naglašava kako se mogu koristiti i kao instrumenti “hibridnih prijetnji” s ciljem utjecaja na javno mnijenje, što pokazuje koliko su njihove primjene višestruke i društveno značajne [2].

U praksi se chatbotovi često integriraju izravno u profile brendova na društvenim mrežama gdje pružaju trenutačnu podršku, automatski generiraju sadržaj (primjerice opise i hashtagove), omogućuju personaliziran “razgovorni” kontakt s publikom, provode analizu sentimenta te prikupljaju i organiziraju podatke o korisnicima i trendovima. Kao primjer integracije na velikim platformama navodi se Meta, koja uvodi AI značajke te testira “Meta AI” asistenta dostupnog na WhatsAppu, Messengeru i Instagramu, uz naglasak na postupno uvođenje i zaštitne mehanizme [2].

2.5. Četvrta generacija - glasovni asistenti i konverzacijska sučelja

Četvrta generacija razvoja umjetne inteligencije u domeni komunikacije označava prijelaz s tekstualnih chatbotova na inteligentne glasovne asistente integrirane u svakodnevne uređaje. Ova faza, koja je započela početkom 2010-ih, donijela je revoluciju u interakciji čovjeka i računala omogućivši prirodan razgovor, prepoznavanje konteksta i upravljanje pametnim domovima. Moderni asistenti koriste duboko učenje i masovne baze podataka kako bi predvidjeli potrebe korisnika i pružili personalizirano iskustvo u stvarnom vremenu [2] [6].

2.5.1. Siri, Google Assistant, Cortana, Alexa

Pojava Siri na iPhoneu 4S 2011. godine postavila je temelje za komercijalnu upotrebu glasovnih asistenata jer je proistekla iz ambicioznog vojnog projekta CALO koji je težio stvaranju kognitivnog agenta sposobnog za učenje. Siri je uvela koncept personaliziranog digitalnog pomoćnika koji ne samo da odgovara na pitanja, već i izvršava zadatke poput postavljanja podsjetnika ili slanja poruka, prilagođavajući se s vremenom jeziku i preferencijama korisnika. Iako je pionir, njezina ovisnost o internetskoj vezi i ograničena podrška za manje jezike ostali su izazovi u ranoj fazi razvoja [2] [6].

“Hey Siri” je *on-device* glasovni okidač koji koristi duboku neuronsku mrežu (DNN) za stalno oslušivanje aktivacijske fraze uz minimalnu potrošnju baterije. Sustav na iPhoneu radi u dvije faze: mali DNN na štedljivom koprocesoru (AOP) vrši početnu detekciju, a tek nakon toga glavni procesor s većim modelom potvrđuje upit. Cijeli proces prepoznavanja temelji se na analizi kratkih audio-okvira od 0,2 sekunde i vremenskoj integraciji rezultata kako bi se izračunala pouzdanost okidanja [8].

Nakon lokalne aktivacije, sustav provjerava identitet korisnika usporedbom s glasovnim profilom spremljenim na uređaju. Iako je sam detektor okidača u potpunosti lokalan radi privatnosti i brzine, većina daljnje obrade (glavno prepoznavanje govora i tumačenje upita) odvija se u oblaku. Ako Siri server utvrdi da početak govora nije bio ispravan okidač, šalje signal uređaju da se vrati u stanje mirovanja [8].

Ubrzo nakon Applea, Google je 2012. predstavio Google Now (kasnije Google Assistant), fokusirajući se na prediktivno pružanje informacija na temelju lokacije i povijesti pretraživanja, dok su Microsoft i Amazon 2014. godine ušli na tržište s Cortanom i Alexom. Dok je Cortana bila usmjerena na produktivnost unutar Windows ekosustava, Alexa je redefinirala koncept pametnog doma kroz Echo uređaje, omogućivši programerima stvaranje novih vještina putem Alexa Skills Kita [2] [6].

2.6. Peta generacija - duboko učenje, transformeri i LLM chatbotovi

Peta generacija uči iz podataka i to u količinama koje nijedan čovjek ne bi mogao pročitati. Dok su raniji sustavi poput programa ELIZA koristili jednostavno prepoznavanje uzoraka, moderni pristupi temelje se na dubokom učenju i neuronskim mrežama koje simuliraju ljudske kognitivne procese. Ova evolucija omogućila je chatbotima da postanu ne samo digitalni asistenti za izvršavanje zadataka, već i empatični sugovornici sposobni za složenu komunikaciju [6].

2.6.1. Sekvencijski modeli

Sekvencijski modeli, poput rekurentnih neuronskih mreža (RNN) i mreža s dugom kratkoročnom memorijom (LSTM), predstavljali su ključni korak u omogućavanju chatbotima da

zadrže kontekst razgovora. Ovi modeli koriste petlje koje omogućuju informacijama da perzistiraju, što je neophodno za razumijevanje rečenica u kojima značenje ovisi o prethodno izgovorenim riječima [9].

Ovi modeli često nailaze na poteškoće pri učenju dugoročnih ovisnosti u vrlo dugim tekstovima. Iako su LSTM i GRU (*Gated Recurrent Units*) značajno poboljšali preciznost u sustavima za odgovaranje na često postavljana pitanja (FAQ), njihova arhitektura i dalje zahtijeva sekvencijalnu obradu podataka, što ograničava brzinu treniranja na velikim skupovima podataka [9].

2.6.2. Transformer arhitektura

Transformer arhitektura donijela je revoluciju u obradi prirodnog jezika uvođenjem mehanizma “pažnje” (*attention mechanism*), koji omogućuje modelu da istovremeno promatra sve dijelove ulazne sekvence. Transformeri mogu paralelno obrađivati podatke, što ubrzava proces treniranja i omogućuje rad s neusporedivo većim korpusima teksta [10].

Ova arhitektura omogućuje modelu da dinamički dodjeljuje važnost različitim riječima u rečenici, bez obzira na njihovu međusobnu udaljenost, čime se postiže dublje razumijevanje semantike i konteksta. Upravo je ovaj tehnološki skok postavio temelje za razvoj suvremenih velikih jezičnih modela koji dominiraju današnjim tržištem umjetne inteligencije [10].

2.6.3. Veliki jezični modeli i generativni chatboti (ChatGPT, Bard ...)

Veliki jezični modeli (LLM) predstavljaju vrhunac generativne umjetne inteligencije, koristeći milijarde parametara za predviđanje i generiranje teksta koji je često nerazlučiv od ljudskog. Sustavi poput ChatGPT-a ili Barda (sada Gemini) ne oslanjaju se na unaprijed definirane odgovore, već sintetiziraju nove rečenice u stvarnom vremenu, prilagođavajući se tonu i specifičnim zahtjevima korisnika kroz višestruke krugove dijaloga [2].

Iako ovi modeli pokazuju impresivne sposobnosti u rješavanju problema, kreativnom pisanju i programiranju, oni se i dalje suočavaju s izazovima poput “halucinacija” ili generiranja netočnih informacija. Njihova integracija u svakodnevne alate i sposobnost učenja iz povratnih informacija korisnika čine ih najnaprednijim oblikom interakcije čovjeka i stroja do danas [11].

3. AI inženjerstvo - koncepti i ključne tehnike

Ovaj rad bavi se izgradnjom chatbota koji se oslanja na postojeće jezične modele, što ga smješta u područje AI inženjerstva, a ne razvoja modela od nule. U ovom poglavlju objašnjavaju se koncepti koji su bili izravno relevantni za taj proces.

3.1. Pojam i uloga AI inženjerstva

AI inženjerstvo je specijalizirano područje u kojem stručnjaci koriste umjetnu inteligenciju i strojno učenje za razvoj aplikacija i sustava koji organizacijama pomažu povećati učinkovitost, smanjiti troškove i donositi bolje poslovne odluke. AI inženjeri razvijaju alate, sustave i procese koji omogućuju primjenu umjetne inteligencije u stvarnom svijetu [12].

U praksi, AI inženjer najčešće radi na spoju modela i produkcijskog sustava: uzima pretreniran model, prilagođava ga kroz fino podešavanje (*fine-tuning*) ili prompt engineering, i izlaže ga kao API koji ostatak aplikacije može konzumirati. Izgradnja modela od nule danas je rjeđa, i skuplja, nego što je to slučaj s inženjering radom oko postojećih. Potrebne vještine uključuju programiranje u jezicima poput Pythona, R-a, Jave i C++-a, uz solidno poznavanje statistike i linearne algebre. Od infrastrukturnih alata očekuje se iskustvo s *big data* tehnologijama kao što su Apache Spark, Hadoop i MongoDB. Ključan je i praktičan rad s algoritmima strojnog učenja i dubokog učenja, a posebno se cijeni poznavanje popularnih AI radnih okvira poput TensorFlowa, PyTorch-a i Kerasa [12].

3.2. Temeljni modeli i veliki jezični modeli (LLM)

Suvremeni pristup u području umjetne inteligencije označava pomak od razvoja izoliranih, usko specijaliziranih modela prema primjeni pretreniranih sustava opće namjene. U središtu ovog pristupa nalaze se temeljni modeli, unutar kojih se veliki jezični modeli (LLM) izdvajaju kao specifična podskupina. Dok su temeljni modeli multimodalni i obuhvaćaju domene poput računalnog vida i obrade zvuka, LLM-ovi su usmjereni na tekstualne podatke, demonstrirajući napredne sposobnosti semantičkog zaključivanja i kontekstualne adaptacije. Razlika između tih dviju skupina nije samo terminološka, ona određuje što od modela možeš tražiti i što možeš očekivati [11], [13].

3.2.1. Temeljni modeli

Temeljni modeli (često u literaturi nazivani i *Pretrained Foundation Models* - PFMs) definiraju se kao modeli velikog kapaciteta prethodno trenirani na golemim i heterogenim skupovima podataka, čime stječu opće reprezentacije primjenjive na niz specifičnih (engl. *downstream*) zadataka. Povijesni razvoj ovih modela, od ranih jezičnih modela poput BERT-a do kompleksnih multimodalnih sustava, ukazuje na trend u kojem se kroz povećanje računalnih kapaciteta i količine podataka postiže visoka razina transfernog učenja (prijenos znanja ste-

čenog na jednom zadatku na drugi, srodan zadatak, umjesto učenja od nule). Ovakav pristup omogućuje da se jedan centralni model, uz minimalne prilagodbe, uspješno koristi u domenama koje variraju od računalnog vida do analize grafova, čime se mijenja proces dizajna inteligentnih sustava [13].

Ključna prednost temeljnih modela leži u njihovoj sposobnosti prilagodbe specifičnim kontekstima putem metoda kao što su fino podešavanje, instrukcijsko učenje ili tehnike parametarski učinkovite adaptacije. Oslanjanje na jedan centralni model donosi i značajne izazove, ponajprije u nasljeđivanju pristranosti iz podataka za pretreniranje, u sigurnosnim rizicima te u potrebi za rigoroznom evaluacijom otpornosti. Stoga se suvremena istraživanja ne fokusiraju samo na povećanje performansi, već i na razvoj mehanizama koji osiguravaju privatnost, etičnost i pouzdanost modela u produkcijskim okruženjima, što znači da se uz tehničke probleme javljaju i pitanja pristranosti, privatnosti i etike [11].

3.2.2. Veliki jezični modeli (LLM)

Veliki jezični modeli predstavljaju specijalizaciju temeljnih modela unutar domene prirodnog jezika, karakterizirani milijardama parametara i treniranjem na korpusima teksta koji obuhvaćaju značajan dio ljudskog znanja. LLM-ovi pokazuju takozvana “izranjajuća” (emergenta) svojstva, sposobnosti poput rješavanja logičkih problema ili učenja iz konteksta koje se pojavljuju tek pri dosezanju određene kritične mase parametara i podataka. Evolucija ovih modela dovela je do pomaka s jednostavnog predviđanja sljedećeg tokena prema sustavima koji su usklađeni s ljudskim namjerama putem metoda poravnanja, čime postaju sposobni za kompleksnu interakciju i izvršavanje složenih korisničkih uputa [11], [13].

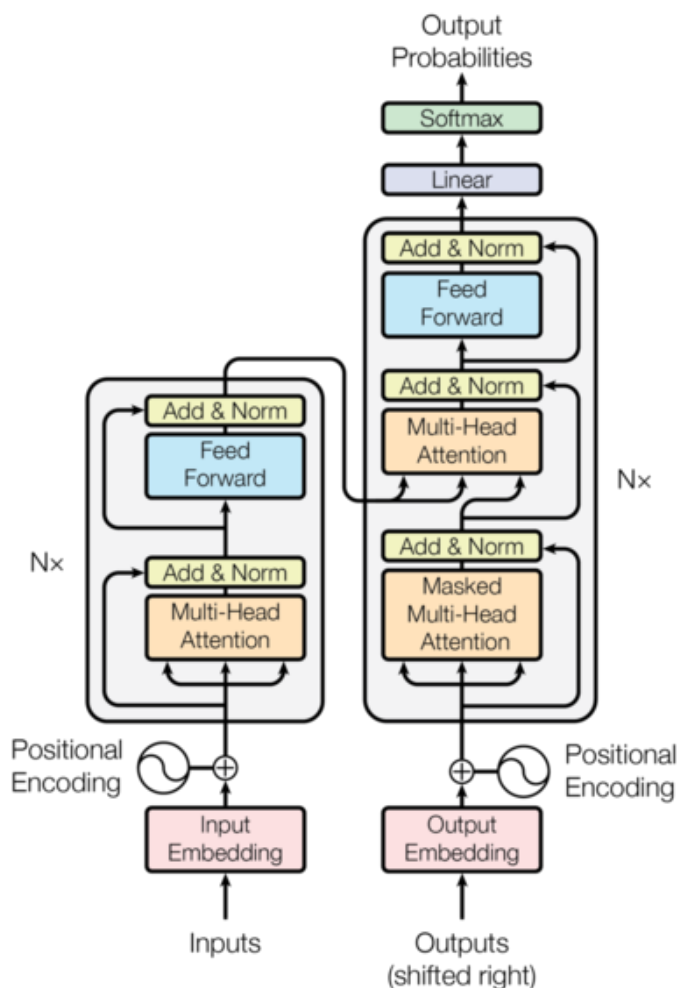
Arhitektonski temelj LLM-ova čini Transformer arhitektura, čija skalabilnost i paralelizacija omogućuju obradu ogromnih količina informacija uz korištenje naprednih varijanti mehanizama pažnje. Kako bi se prevladali visoki računalni troškovi treniranja i inferencije, istraživački fokus se sve više usmjerava prema optimizaciji arhitekture, uključujući efikasnije metode pozicijskog kodiranja i tehnike kompresije modela poput kvantizacije i “prorjeđivanja”. Ovi tehnički napredci omogućuju širu dostupnost LLM-ova, pretvarajući ih iz isključivo istraživačkih artefakata u praktične komponente koje se mogu integrirati u različite softverske arhitekture uz zadržavanje visoke razine preciznosti [13].

U suvremenim primjenama, LLM-ovi se rijetko koriste kao izolirani sustavi. Umjesto toga, oni postaju jezgra širih okvira koji uključuju vanjske izvore znanja kroz metode poput generiranja potpomognutog dohvatom (*Retrieval-Augmented Generation* - RAG). Ovakva integracija rješava inherentne probleme modela, poput halucinacija i nedostatka ažurnih informacija, dok istovremeno omogućuje specijalizaciju za specifične domene bez potrebe za skupim ponovnim treniranjem. Konačno, proces evaluacije LLM-ova evoluirao je od jednostavnih metričkih pokazatelja prema kompleksnim referentnim testovima koji ispituju kognitivne sposobnosti, etičku usklađenost i praktičnu korisnost [13] [11].

3.3. Tehničke osnove: transformer i *attention*

Prije pojave Transformera, rekurentne neuronske mreže poput LSTM i GRU bile su standardni izbor za obradu sekvenci teksta. Njihov glavni nedostatak bila je sekvencijalna obrada, svaki element sekvence morao se obraditi tek nakon prethodnog, što je onemogućavalo paralelizaciju i usporavalo treniranje na dugim sekvencama [10]. Transformer arhitektura riješila je ovaj problem potpunim odbacivanjem rekurencije i oslanjanjem isključivo na mehanizme pažnje (*attention*), čime je postala temeljna arhitektura za suvremene jezične modele [13].

3.3.1. Transformer arhitektura



Slika 2: Koder - dekoder [10]

Transformer se sastoji od dva glavna dijela (Slika 2): kodera (*encoder*) koji obrađuje ulazni tekst i dekodera (*decoder*) koji generira izlazni tekst. Koder pretvara ulaznu sekvencu u numeričke reprezentacije koje dekoder koristi za generiranje izlaza, element po element. Model je autoregresivan, pri generiranju svakog novog elementa koristi sve prethodno generirane elemente kao kontekst [10].

Ključna prednost ove arhitekture je sposobnost da istovremeno “vidi” sve dijelove ulazne

sekvence, neovisno o njihovoj međusobnoj udaljenosti. Ovo omogućuje učinkovito hvatanje dugodosežnih ovisnosti u tekstu, što je kod rekurentnih mreža predstavljalo značajan izazov [10]. Također, budući da se sve pozicije mogu obraditi paralelno, Transformer omogućuje znatno brže treniranje na modernim GPU jedinicama [10].

3.3.2. *Attention* mehanizam

Attention mehanizam omogućuje modelu da pri obradi svakog elementa sekvence pažnju obrati na relevantne dijelove ulaza. Funkcionira tako da za svaki element (upit) izračuna koliko je povezan sa svim ostalim elementima (ključevima), a zatim te povezanosti koristi kao težine za kombiniranje vrijednosti. Matematički se to izražava sljedećom formulom [10]:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Q (Query) predstavlja trenutačni element koji se obrađuje, K (Key) su svi elementi ulaza na koje se može obratiti pažnja, a V (Value) su vrijednosti povezane s tim elementima. Normalizacija dijeljenjem s $\sqrt{d_k}$ osigurava stabilnost gradijenata tijekom treniranja [10].

Višeglavi *attention* (*multi-head attention*) proširuje ovaj mehanizam izvođenjem više paralelnih *attention* operacija, svaka s različitim naučenim projekcijama. Rezultati se zatim spajaju, što modelu omogućuje istovremeno fokusiranje na različite vrste informacija, primjerice, jedna “glava” može pratiti sintaktičke odnose, dok druga prati semantičke veze [10].

3.4. Rad s LLM-ovima u aplikacijama: tokeni, kontekst i kontrola generiranja

Veliki jezični modeli obrađuju tekst na način koji se razlikuje od tradicionalnog pristupa čitanju i pisanju. Umjesto da rad s tekstom temelji na riječima ili rečenicama, LLM-ovi koriste tokene kao osnovne jedinice obrade i generiraju tekst sekvencijalno, token po token. Razumijevanje ovih procesa ključno je za učinkovitu primjenu LLM-ova u praktičnim aplikacijama.

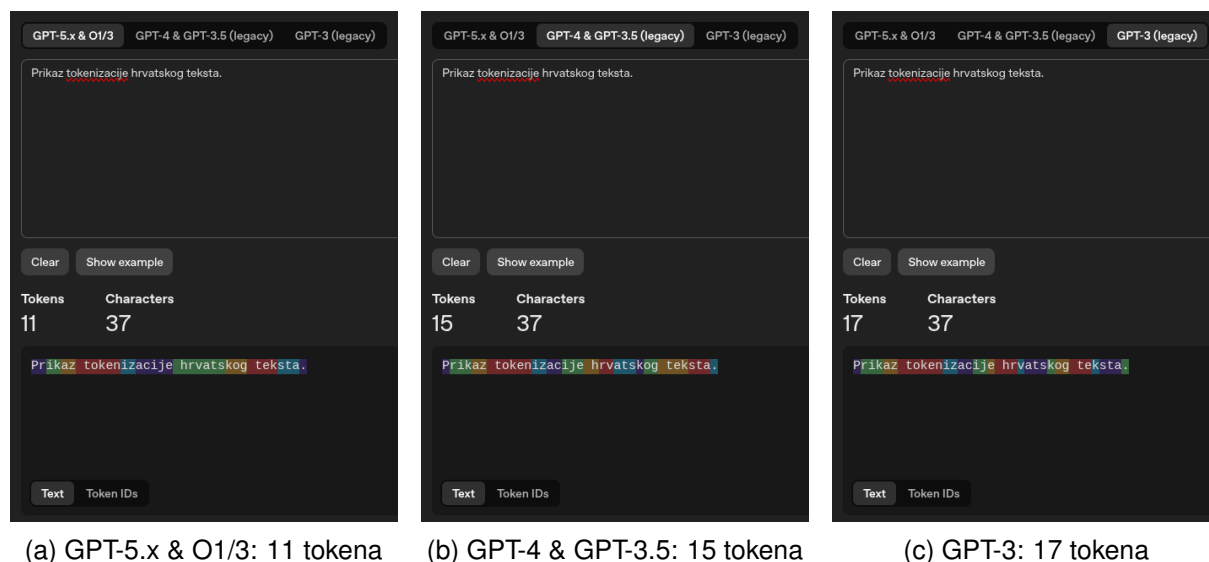
3.4.1. Tokeni i tokenizacija

Token je osnovna jedinica teksta koju model obrađuje tijekom svog rada. Ovisno o postupku tokenizacije, token može biti znak, podriječ, simbol ili cijela riječ [10]. Tokenizacija je bitna faza predobrade koja tekst rastavlja na te nedjeljive jedinice. Najčešće korištene metode tokenizacije kod LLM-ova uključuju *wordpiece*, kodiranje parom bajtova (BPE) i *unigramLM* [13].

Proces tokenizacije započinje tako što se ulazni tekst pretvara u niz tokena, pri čemu svaki token dobiva jedinstveni identifikator iz rječnika modela. Na primjer, riječ “tokenization” može biti podijeljena na više tokena poput “token”, “ization” ili čak na manje jedinice ovisno o primijenjenom algoritmu. Vaswani i suradnici koriste naučene vektorske reprezentacije (*embed-*

dings) za pretvaranje ulaznih i izlaznih tokena u vektore dimenzije d_{model} [10]. Ove vektorske reprezentacije omogućuju modelu da razumije semantičke odnose između različitih tokena.

Na Slici 3 prikazana je tokenizacija rečenice “Prikaz tokenizacije hrvatskog teksta.” (37 znakova) prema trima generacijama OpenAI modela. Isti tekst novije generacije dijele na manje tokena, što ilustrira napredak u obradi hrvatskog jezika.



Slika 3: Tokenizacija hrvatskog teksta prema trima generacijama OpenAI modela. Izvor: <https://platform.openai.com/tokenizer>.

Model generira izlaz sekvencijalno, predviđajući sljedeći token na temelju prethodno generiranih tokena. Ovaj proces iterativnog generiranja znači da model mora za svaki novi token izvršiti proračun kroz sve svoje slojeve, što izravno utječe na brzinu izvođenja i cijenu korištenja modela. Veći broj tokena u ulazu ili izlazu povećava računske zahtjeve i vrijeme potrebno za dobivanje odgovora. Različiti LLM-ovi koriste različite veličine rječnika tokena, primjerice, GPT-3 koristi približno 50.000 tokena, dok BLOOM koristi 250.000 tokena u svom rječniku [13].

3.4.2. Kontekst modela

Kontekst modela označava maksimalnu količinu teksta s kojom model može raditi odjednom. Može se zamisliti kao radna memorija modela, sve što model “vidi” i “pamti” tijekom obrade jednog zahtjeva ograničeno je veličinom ovog kontekstnog prozora. LLM-ovi su tradicionalno trenirani s ograničenim kontekstnim prozorima zbog računske složenosti mehanizma pažnje i visokih zahtjeva za memorijom [13].

Ograničenja konteksta predstavljaju važan praktični izazov. Model koji je treniran s ograničenom duljinom sekvence obično ne može uspješno generalizirati na veće duljine tijekom primjene. Proširivanje kontekstnog prozora tijekom finog podešavanja modela je spor, neučinkovit i računski skup proces [13]. Zbog toga su razvijene različite tehnike za proširenje konteksta poput interpolacije pozicijskih kodiranja i učinkovitijih mehanizama pažnje.

Gusti mehanizam globalne pažnje jedan je od glavnih uzroka ograničenja pri treniranju modela s većim kontekstnim prozorima. Korištenje učinkovitijih varijanti pažnje, poput lokalne, rijetke ili dilatirane pažnje, može značajno smanjiti računsku složenost. Na primjer, LongT5 koristi prijelaznu globalnu pažnju (TGlobal) koja kombinira lokalnu i globalnu pažnju, što omogućuje proširenje konteksta na 16.000 tokena [13].

Ograničenja konteksta motiviraju razvoj različitih tehnika za rad s dugim dokumentima. Tehnike poput sažimanja sadržaja, dijeljenja teksta na manje dijelove tj. segmentacija (*chunking*) i pristupa proširene generiranja s dohvaćanjem (RAG) omogućuju modelima pristup većoj količini informacija nego što stane u njihov kontekstni prozor. RAG pristup kombinira vanjsko pohranjivanje informacija s mogućnostima generiranja LLM-a, čime se omogućuje pristup ažurnim podacima i smanjuje pojava halucinacija [13].

3.4.3. Parametri generiranja: *temperature*, *top-k* i *top-p*

Nakon što model procesira ulazne tokene kroz svoje slojeve, dobiva se izlazna raspodjela vjerojatnosti za sve moguće sljedeće tokene. Transformer arhitektura koristi linearnu transformaciju i *softmax* funkciju za pretvaranje izlaza dekodera u vjerojatnosti sljedećeg tokena [10]. *Softmax* funkcija normalizira rezultate tako da zbroj svih vjerojatnosti bude jednak jedan, čime se dobiva valjana vjerojatnosna raspodjela.

Različiti parametri kontroliraju kako model bira sljedeći token iz ove raspodjele vjerojatnosti, što omogućuje podešavanje ravnoteže između kreativnosti i dosljednosti generiranog teksta.

Temperature je parametar koji kontrolira stupanj stohastičnosti u procesu generiranja. Temperature modificira vjerojatnosnu raspodjelu prije primjene *softmax* funkcije tako što dijeli logite (neobrađeni brojevi izlazi modela za svaki token, prije pretvaranja u vjerojatnosti) s vrijednošću temperature. Niže vrijednosti temperature (bliže nuli) čine raspodjelu oštrijom, model postaje konzervativniji i češće bira tokene s najvećom vjerojatnosti, što rezultira predvidljivijim i stabilnijim odgovorima. Više vrijednosti temperature (veće od jedan) zaglađuju raspodjelu vjerojatnosti, dajući više šanse tokenima s nižom vjerojatnosti, što potiče veću raznolikost i kreativnost u generiranom tekstu. Temperatura se često koristi pri treniranju modela za generiranje prirodnog jezika kako bi se poboljšala kvaliteta prijevoda ili generiranih sekvenci [14].

Top-k uzorkovanje ograničava izbor sljedećeg tokena na k najvjerojatnijih kandidata iz raspodjele vjerojatnosti. Nakon što se identificira prvih k tokena s najvećom vjerojatnosti, model uzorkuje sljedeći token samo iz ovog ograničenog skupa, zanemarujući sve ostale mogućnosti. Ovaj pristup sprječava model da bira vrlo nevjerojatne tokene koji mogu narušiti koherentnost teksta, ali istovremeno omogućuje određenu raznolikost među najvjerojatnijim opcijama. Vrijednost parametra k određuje koliko će kandidata biti razmatrano, manja vrijednost rezultira konzervativnijim odabirom, dok veća vrijednost dopušta više varijabilnosti [14].

Top-p uzorkovanje, poznato i kao *nucleus sampling*, predstavlja dinamičniji pristup filtriranju kandidata. Umjesto fiksnog broja tokena, ovaj metoda odabire najmanji skup tokena čija kumulativna vjerojatnost prelazi zadani prag p (obično između 0.9 i 0.95). To znači da

model automatski prilagođava veličinu skupa kandidata ovisno o distribuciji vjerojatnosti, kada je jedan token dominantan, skup će biti manji, a kada je više tokena približno jednako vjerojatno, skup će biti veći. Ovaj pristup omogućuje modelu da prilagodi razinu konzervativnosti ovisno o pouzdanosti predikcije u svakom koraku generiranja [14].

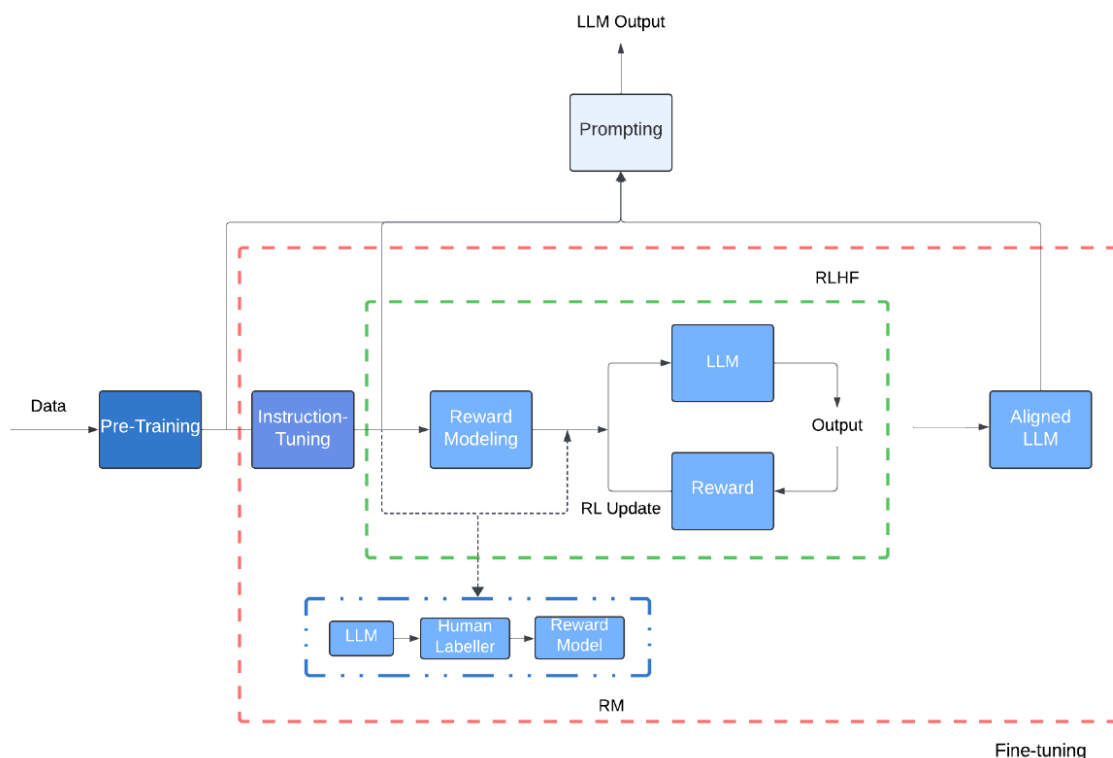
Kombinacija ovih parametara omogućuje fino podešavanje ponašanja modela prema specifičnim potrebama primjene. Za zadatke koji zahtijevaju preciznost i dosljednost, poput generiranja programskog koda ili formalnih dokumenata, koriste se niže vrijednosti temperature i konzervativnije strategije uzorkovanja. Za kreativne zadatke poput pisanja priča ili generiranja ideja, preporučuju se više vrijednosti temperature i fleksibilnije strategije uzorkovanja koje potiču raznolikost u generiranom sadržaju.

3.5. Prilagodba ponašanja bez treniranja: prompt engineering i *in-context* učenje

Tradicionalni pristup prilagodbi jezika modela za specifične zadatke ili domene zahtijeva postupak finog podešavanja, pri čemu se parametri prethodno treniranog modela ažuriraju korištenjem zadataka specifičnih podataka. Iako ovaj pristup omogućava značajno poboljšanje performansi za ciljne aplikacije, proces je računski skup i zahtijeva pristup kvalitetnim označenim podacima za svaki novi zadatak. Fino podešavanje može dovesti do katastrofičnog zaboravljanja (*catastrophic forgetting*), gdje model gubi znanje stečeno tijekom inicijalnog treniranja [14]. Zbog toga se razvio drugačiji pristup: umjesto da se mijenja model, mijenja se način na koji mu se postavljaju pitanja.

Veliki jezični modeli pokazuju sposobnost adaptacije putem promjena u načinu postavljanja upita, odnosno kroz tehniku koja se naziva *prompting* ili *prompt engineering*. Kao što je prikazano na Slici 4, razvoj velikog jezičnog modela odvija se kroz niz faza. Model se najprije *pred-trenira* (engl. *pre-training*) na velikoj količini tekstualnih podataka, čime usvaja jezične obrasce, činjenično znanje i statističke odnose među riječima. Nakon toga slijedi faza finog podešavanja, koja obuhvaća podešavanje prema uputama (engl. *instruction-tuning*) te učenje pojačanjem temeljeno na ljudskim povratnim informacijama (engl. *reinforcement learning with human feedback*, RLHF). U sklopu RLHF-a gradi se model nagrađivanja (engl. *reward model*, RM), koji na temelju ljudskih ocjena generiranih odgovora procjenjuje njihovu kvalitetu, a dobivena nagrada zatim usmjerava ažuriranje parametara modela tako da njegovi izlazi budu što bolje usklađeni s ljudskim preferencijama. Rezultat cijelog postupka jest usklađeni model (engl. *aligned LLM*), spreman za primjenu [11].

Za razliku od navedenih faza koje mijenjaju parametre modela, *prompting* predstavlja metodu postavljanja upita treniranim LLM-ovima koja omogućava prilagodbu modela bez finog podešavanja parametara. Ova neočekivana sposobnost velikih jezičnih modela omogućava korisnicima da usmjeravaju ponašanje modela isključivo kroz pažljivo oblikovane tekstualne upute i primjere unutar samog upita. Dva ključna pristupa prilagodbi bez treniranja su *prompt engineering*, koji se fokusira na strukturiranje i formuliranje upita, te *in-context* učenje, pri čemu model uči iz demonstracija prisutnih u promptu [14], [15].



Slika 4: Dijagram toka različitih faza razvoja LLM-a od pred-treniranja do korištenja [11]

3.5.1. Prompt engineering

Prompt engineering predstavlja disciplinu oblikovanja uputa za velike jezične modele s ciljem postizanja konzistentnijih i kvalitetnijih rezultata. Ovaj pristup omogućava prilagodbu ponašanja modela bez potrebe za dodatnim treniranjem ili izmjenom parametara. Prompt može sadržavati različite elemente poput definiranja uloge modela, specificiranja željenog formata izlaza, postavljanja pravila ponašanja ili davanja kontekstualnih informacija [16].

Istraživanja pokazuju kako kvaliteta prompta značajno utječe na performanse modela. Dobro strukturiran prompt uključuje jasne i detaljne upute, može sadržavati pozitivne i negativne primjere željenog ponašanja te specificirati očekivanu duljinu ili format odgovora. Prompting može uključivati tehnike poput poticanja korak-po-korak rasuđivanja, gdje se model navodi da eksplicitno prikaže svoj proces razmišljanja prije davanja konačnog odgovora [13].

Prompt engineering postaje posebno važan u kontekstu *zero-shot* učenja, gdje model treba odgovoriti na zadatak koji nije vidio tijekom treniranja. Veliki jezični modeli pokazuju sposobnost *zero-shot* učenja, što znači da mogu odgovarati na upite bez ikakvog prethodnog primjera u promptu. Ova sposobnost omogućava modelima generalizaciju na nove zadatke isključivo na temelju jasno formuliranih uputa [13].

3.5.2. Sistemski i korisnički prompt

Moderni sustavi velikih jezičnih modela koriste distinkciju između sistemskog i korisničkog prompta kako bi omogućili fleksibilnije i kontroliranije interakcije. Sistemski prompt definira opće ponašanje modela kroz cijelu interakciju i postavlja osnovne smjernice za način rada. Ovaj prompt obično ostaje konstantan tijekom sesije i određuje kontekst u kojem model interpretira korisničke zahtjeve. Sistemski prompt može uključivati informacije o ulozi modela, stilskim preferencijama, ograničenjima u ponašanju ili specifičnim domenskim znanjima [15].

Korisnički prompt predstavlja konkretan zahtjev ili upit koji korisnik postavlja u određenom trenutku. Ovaj prompt je dinamičan i mijenja se sa svakim zahtjevom korisnika. Model kombinira informacije iz sistemskog i korisničkog prompta kako bi generirao odgovarajući odgovor. Ova arhitektura omogućava održavanje konzistentnog ponašanja modela (kroz sistemski prompt) dok istovremeno omogućava fleksibilnost u odgovaranju na raznolike korisničke upite [13].

Važnost ove distinkcije posebno dolazi do izražaja u aplikacijama gdje je potrebno održavanje određenog stila komunikacije ili poštivanje specifičnih ograničenja kroz cijelu konverzaciju. Na primjer, sistemski prompt može definirati da model uvijek daje odgovore u određenom profesionalnom tonu ili da izbjegava određene teme, dok korisnički promptovi postavljaju konkretna pitanja unutar tog okvira.

3.5.3. *In-context* learning

In-context learning ili učenje u kontekstu predstavlja ključnu sposobnost velikih jezičnih modela koja im omogućava prilagodbu novim zadacima bez izmjene parametara modela. Ovaj fenomen se manifestira kroz sposobnost modela da uči iz primjera koji su dani unutar samog prompta. Kada se modelu pruži nekoliko primjera ulazno-izlaznih parova za određeni zadatak, model može generalizirati taj obrazac i primijeniti ga na nove ulaze [17].

Few-shot learning, kao specifičan oblik *in-context* učenja, pokazuje se posebno efektivnim. Istraživanja na GPT-3 modelu pokazala su kako performanse modela u *few-shot* scenariju premašuju performanse u *zero-shot* scenariju, što sugerira da veliki jezični modeli funkcioniraju kao meta-učenici (*meta-learners*), modeli koji uče kako učiti iz malog broja primjera [17]. Broj primjera koji se daju modelu može varirati, ali čak i jedan ili dva dobro odabrana primjera mogu značajno poboljšati performanse na novom zadatku.

Format *in-context* učenja tipično uključuje strukturirane demonstracije gdje se svakom primjeru prilaže ulaz i očekivani izlaz. Model zatim koristi ove demonstracije kao reference za generiranje odgovora na novi upit. Kvaliteta i relevantnost odabranih primjera značajno utječu na performanse modela. Istraživanja pokazuju da primjeri trebaju biti što sličniji ciljanom zadatku te da redoslijed primjera također može utjecati na rezultate [13].

Ova tehnika omogućava privremeno usmjeravanje modela bez potrebe za skupim postupkom finog podešavanja. Parametri modela ostaju nepromijenjeni, a prilagodba se postiže isključivo kroz sadržaj prompta. To čini *in-context* učenje izuzetno praktičnim pristupom za brzu

prilagodbu modela različitim domenama i zadacima, posebno u scenarijima gdje je dostupno samo nekoliko primjera ili gdje je potrebna česta promjena zadataka [13].

In-context učenje ima svoja ograničenja. Svi primjeri moraju stati u kontekstni prozor modela, što ograničava broj demonstracija koje se mogu pružiti. Performanse ovog pristupa ovise o veličini modela, veći modeli pokazuju značajno bolje sposobnosti *in-context* učenja nego manji modeli [17].

3.6. Prilagodba modela treniranjem i optimizacija primjene

Veliki jezični modeli treniraju se na ogromnim količinama tekstualnih podataka kako bi stekli opće jezične sposobnosti, no njihova primjena u stvarnim sustavima često zahtijeva dodatne korake optimizacije. Prilagodba modela specifičnim domenama ili zadacima, smanjenje računalnih zahtjeva i usklađivanje s ljudskim preferencijama ključni su izazovi pri integraciji LLM-ova u proizvodne sustave. Dok su tehnike poput *prompt engineeringa* i *in-context* učenja korisne za prilagodbu ponašanja bez izmjene parametara modela, postoje scenariji u kojima je potrebno izravno modificirati sam model kako bi se postigla stabilnija i učinkovitija izvedba. Ovaj pristup omogućuje optimizaciju modela za specifične zadatke uz kontrolu nad resursnim zahtjevima i usklađenost s očekivanim ponašanjem [11].

Tehnike prilagodbe modela kreću se od potpunog ponovnog treniranja na domenskim podacima do metoda koje selektivno modificiraju mali podskup parametara, kao i postupaka kompresije koji smanjuju veličinu modela uz minimalan gubitak performansi. Fino podešavanje omogućuje specijalizaciju modela za određene zadatke ili domene, dok *parameter-efficient* metode postižu slične rezultate uz znatno manje računalnih resursa. Tehnike kao što su fino podešavanje usklađuju izlaze modela s ljudskim vrijednostima i preferencijama, što je ključno za primjenu u stvarnim sustavima gdje je sigurnost i korisnost prioritet. Metode kvantizacije i destilacije omogućuju razvoj manjih, brzih verzija modela pogodnih za implementaciju na uređajima s ograničenim resursima, čime se proširuje dostupnost i praktična primjenjivost LLM tehnologije [11].

3.6.1. Fino podešavanje i PEFT

Fino podešavanje predstavlja proces dodatnog treniranja prethodno naučenog jezičnog modela na skupu podataka specifičnom za određeni zadatak ili domen. Fino podešavanje omogućuje modelu da specijalizira svoje znanje za uže definirane aplikacije. Ovaj proces nadograđuje postojeće reprezentacije naučene tijekom pretreniranja, prilagođavajući ih posebnostima ciljane domene ili zadatka. Kod tradicionalnog finog podešavanja ažuriraju se svi parametri modela, što može rezultirati stabilnijim i pouzdanijim ponašanjem u specifičnim scenarijima, ali zahtijeva značajne računalne resurse i memoriju proporcionalnu veličini modela [11].

Tradicionalni pristup finom podešavanju, iako učinkovit, suočava se s praktičnim izazovima kada se primjenjuje na moderne LLM-ove koji sadrže milijarde parametara. Ažuriranje svih parametara tijekom treniranja zahtijeva pohranu gradijenata i optimizacijskih stanja

za svaki parametar, što multiplicira memorijske zahtjeve i čini proces računalno zahtjevnim. Potpuno fino podešavanje nosi rizik od “katastrofičkog zaboravljanja” gdje model gubi opće jezične sposobnosti naučene tijekom pretriranja dok se prilagođava novom zadatku. Ovi izazovi motivirali su razvoj pristupa koji omogućuju učinkovitu prilagodbu modela uz minimiziranje računalnih i memorijskih troškova [11].

Parameter-Efficient Fine-Tuning (PEFT) metode predstavljaju alternativni pristup koji postiže prilagodbu modela modificirajući samo mali podskup parametara ili dodavanjem dodatnih adaptivnih komponenti. Jedna od najpoznatijih PEFT tehnika je LoRA (*Low-Rank Adaptation*) koja umjesto ažuriranja svih težina matrica u modelu, dodaje male podešavane matrice niskog ranga koje se kombiniraju s originalnim zamrznutim težinama. Ovaj pristup smanjuje broj parametara koje je potrebno trenirati, tipično za nekoliko redova veličine, što rezultira znatno nižim memorijskim zahtjevima i bržim treniranjem, uz zadržavanje kvalitete prilagodbe usporedive s potpunim finim podešavanjem. PEFT metode također omogućuju zadržavanje više različitih adaptacija istog osnovnog modela za različite zadatke, što je praktično u scenarijima gdje je potrebno podržati više specijaliziranih aplikacija [11].

3.6.2. Preference fine-tuning

Preference fine-tuning predstavlja naprednu tehniku koja omogućuje usklađivanje ponašanja jezičnih modela s ljudskim vrijednostima i preferencijama kroz učenje iz komparativnih ocjena. *Preference fine-tuning* koristi podatke u obliku usporedbi između različitih generiranih odgovora na isti upit. Ljudski korisnici ocjenjuju koji od ponuđenih odgovora je korisniji, sigurniji, informativniji ili prikladniji, a model uči da preferira generiranje odgovora koji su bolje ocijenjeni. U praksi se pokazalo da je ljudima lakše reći koji od dva odgovora je bolji nego opisati što idealan odgovor uopće jest, što ga čini pogodnim za domene gdje je kvaliteta subjektivna ili je teško uspostaviti formalizam [11].

Najpoznatiji pristup preference fine-tuningu je *Reinforcement Learning from Human Feedback* (RLHF) koji kombinira nadzirano učenje (*supervised learning*) s metodama učenja potkrepljivanjem (engl. *reinforcement learning*). Proces započinje skupljanjem usporednih ocjena ljudi gdje anotatori rangiraju različite odgovore modela na iste upite. Iz tih ocjena trenira se nagradni model koji uči predvidjeti ljudske preferencije, tj. procjenjuje koja vrsta odgovora će biti pozitivno ocijenjena. Konačno, koristeći algoritme poput *Proximal Policy Optimization* (PPO), osnovni jezični model se optimizira da maksimizira očekivanu nagradu prema naučenom nagradnom modelu, efektivno učeći generirati odgovore usklađene s ljudskim preferencijama. Ovaj pristup pokazao se ključnim u razvoju dijaloških AI sustava poput ChatGPT, omogućivši modele koji ne samo da generiraju koherentne odgovore, već su također korisni i pružaju odgovore u stilu koji korisnici preferiraju [15].

Iza toga stoji jednostavan problem: model može biti tehnički sposoban, ali to ne znači da će se ponašati onako kako ljudi očekuju. Kroz iterativni proces prikupljanja povratnih informacija i učenja iz njih, modeli postaju sve prikladniji za interakciju s ljudima u stvarnim aplikacijama. Ovaj pristup omogućuje korekciju neželjenih ponašanja poput generiranja štetnog sadržaja, nedosljednosti u odgovorima ili neprimjerenog tona, što je teško postići samo kroz

nadzirano učenje. *Preference fine-tuning* omogućava kontinuiranu prilagodbu modela u skladu s ljudskim preferencijama i društvenim standardima, čineći sustave fleksibilnijima i odgovornijima u dugoročnoj primjeni [11].

3.6.3. Kvantizacija i destilacija

Model od 70 milijardi parametara u punoj preciznosti zahtijeva više od 140 GB memorije, što premašuje kapacitet gotovo svake potrošačke grafičke kartice. Kvantizacija rješava taj problem smanjujući preciznost pohrane težina: umjesto 32-bitnog zapisa po parametru, koristi se 8-bitni ili čak 4-bitni, čime se memorijski zahtjev smanjuje četiri, odnosno osam puta uz relativno skroman pad kvalitete. Osim smanjenja memorijskog otiska, kvantizacija također ubrzava izvođenje jer operacije s nižom preciznošću se može brže izvršavati, posebno na specijaliziranom hardveru. Moderna istraživanja pokazuju kako je moguće kvantizirati veliki dio parametara LLM-a bez značajnog gubitka performansi, zahvaljujući redundanciji u naučenim reprezentacijama. Postoje različiti pristupi kvantizaciji, uključujući kvantizaciju nakon treniranja *post-training* gdje se već trenirani model komprimira, te treniranje svjesno kvantizacije *quantization-aware training* gdje se kvantizacija uzima u obzir tijekom samog procesa treniranja [11].

Kvantizacija smanjuje preciznost postojećeg modela; destilacija ide korak dalje i stvara fundamentalno manji model. Umjesto da mali model uči samo iz označenih podataka, uči i iz distribucije odgovora većeg modela, tzv. mekih predikcija koje nose više informacije nego oznaka točnog odgovora. Na primjer, pri generiranju sljedeće riječi, učiteljski (*teacher*) model može dodijeliti vjerojatnost 0.7 jednoj riječi, 0.2 drugoj i 0.1 trećoj, ove relativne vjerojatnosti pružaju učenički (*student*) modelu uvid u nijanse jezika koje je učiteljski model naučio [11].

Primijenjene zajedno, ove dvije tehnike daju model koji je i manji i brži od originala, dovoljno kompaktan da radi lokalno na laptopu ili mobitelu, bez potrebe za slanjem upita u oblaku. Upravo to je razlog zašto modeli poput Phi-3 ili Gemma 3 mogu postići rezultate usporedive s mnogo većim modelima, uz djelić njihovih računalnih zahtjeva [11].

3.7. RAG: povezivanje modela s vanjskim znanjem

Jezični model ne može znati što se promijenilo nakon što je treniran i neće to ni naznačiti, nego će generirati odgovor koji je uvjerljiv, činjenično zastario ili netočan, što se naziva halucinacijom. Za sustav koji odgovara na pitanja iz vojnog zakonodavstva to nije tek teorijski problem, propis koji je bio na snazi u trenutku treniranja u međuvremenu je možda izmijenjen ili stavljen izvan snage. *Retrieval-Augmented Generation* (RAG) pristup rješava ova ograničenja integrirajući generativne sposobnosti velikih jezičnih modela s preciznošću dohvaćanja informacija u stvarnom vremenu, čime se omogućuje da modeli utemeljuju svoje odgovore u dinamički ažuriranim i provjerljivim izvorima [18].

3.7.1. Osnovna ideja RAG-a

Retrieval-Augmented Generation (RAG) predstavlja okvir koji spaja mehanizme dohvaćanja dokumenata temeljene na pretraživanju s generativnim modelima kako bi se poboljšala kvaliteta generiranih rezultata. RAG sustav koristi informacije koje su dohvaćene iz vanjskih izvora znanja, najčešće baza podataka, kako bi nadopunio proces generiranja odgovora. Ovaj pristup kombinira snagu velikih jezičnih modela u generiranju teksta s preciznošću dohvaćanja informacija u stvarnom vremenu, čime se znatno povećava točnost i pouzdanost odgovora [18].

Tijekom faze dohvaćanja (*retrieval*), sustav najprije konstruira upit na temelju korisničkog unosa koji je namijenjen jezičnom modelu. Ovaj upit se zatim koristi za pretraživanje naj-sličnijih ili najrelevantnijih dokumenata iz vanjskih izvora znanja. Mehanizam za dohvaćanje procjenjuje sličnost između korisničkog upita i vraćenih rezultata iz baze znanja kako bi odabrao najrelevantnije dokumente. Nakon što su najrelevantniji rezultati identificirani, proces obogaćivanja kombinira dohvaćene dokumente s izvornim korisničkim unosom kako bi se formirao konačni upit koji se prosljeđuje generativnom jezičnom modelu [18].

Veliki jezični modeli mogu imati ograničenu memoriju i zastarjele informacije, što dovodi do netočnih odgovora. Dohvaćanje relevantnih informacija iz vanjskog, ažuriranog skladišta omogućuje modelima da točno odgovaraju s referencama i koriste više informacija nego što je dostupno u njihovim parametrima. S obogaćivanjem dohvaćanja, manji modeli su pokazali izvedbu usporedivu s mnogo većim modelima, primjerice, model od 11 milijardi parametara može biti konkurentan modelu PaLM od 540 milijardi parametara, a model od 7,5 milijardi može dosegnuti izvedbu modela Gopher od 280 milijardi parametara [11].

3.7.2. Vektorska reprezentacija i vektorske baze podataka

Vektorska reprezentacije tekstualnih podataka predstavljaju temelj za semantičko pretraživanje u RAG sustavima. Mehanizmi za gusti dohvat (*dense retrievers*) koriste prethodno trenirane jezične modele kako bi pretvarali i upite i dokumente u guste vektorske prikaze koji enkapsuliraju semantičko značenje teksta. Za razliku od rijetkih metoda pretraživanja (*sparse retrievers*) poput BM25 i TF-IDF koje se temelje na usporedbi ključnih riječi, guste metode omogućuju identifikaciju kontekstualno relevantnih dokumenata čak i u odsustvu točnog podudaranja riječi [18].

Primjer gustog mehanizma za dohvat je *Dense Passage Retriever* (DPR) koji koristi dvije neovisne, prethodno trenirane BERT mreže kao gusti enkoder. Jedna mreža enkodira svaki ulazni upit u vektor realnih brojeva, dok druga enkodira svaki tekstualni odlomak u vektor istih dimenzija. Sličnost između upita i tekstualnog odlomka mjeri se unutarnjim produktom ovih vektora, a enkoderi se treniraju maksimiziranjem unutarnjih produkata između vektora upita i odgovarajućih relevantnih odlomaka. Ovakvi enkoderi pokazuju otpornu izvedbu kao prethodno trenirani mehanizmi za dohvaćanje i široko se koriste u različitim RAG modelima [18].

Vektorske baze podataka omogućuju efikasno semantičko pretraživanje pohranjivanjem i indeksiranjem vektorskih prikaza dokumenata. Kada korisnik postavi upit, sustav ga pretvara

u vektorski oblik korištenjem istog modela za vektorsku reprezentaciju i zatim pretražuje vektorsku bazu kako bi pronašao vektore koji su najbližiji upitu u vektorskom prostoru. Vanjska memorija u RAG sustavima može biti održavana u različitim formatima kao što su dokumenti, vektori ili baze podataka, a neki sustavi održavaju posredničke (engl. *intermediate*) memorijske reprezentacije kako bi zadržali informacije kroz više iteracija [11].

3.7.3. Aproksimativno pretraživanje najbližih susjeda

Sličnost između vektora upita i vektora dokumenta u praksi se najčešće mjeri kosinusnom sličnošću, normaliziranim skalarnim produktom dvaju vektora kojemu vrijednost bliža jedinici označava veću semantičku srodnost teksta koji vektori predstavljaju [18]. Egzaktna, tzv. *brute-force* pretraga uspoređuje upit sa svakim pojedinim vektorom u korpusu, čime jamči da će uvijek pronaći stvarno najbliže susjede, no vrijeme takve pretrage raste linearno s brojem pohranjenih vektora, što je za korpuse s milijunima odlomaka računski neprihvatljivo skupo.

Za velike korpuse zato se u praksi koriste algoritmi za aproksimativno pretraživanje najbližih susjeda (*Approximate Nearest Neighbors*, ANN), koji uz malen i kontroliran gubitak točnosti postižu znatno bržu pretragu. Najčešće korišten ANN pristup u suvremenim vektorskim bazama podataka je *Hierarchical Navigable Small World* (HNSW), a uz njega se koriste i *Inverted File Index* (IVF), *Product Quantization* (PQ), *Locality Sensitive Hashing* (LSH) te strukture stabla poput KD-stabla i *Ball tree* [19].

HNSW organizira vektore u višeslojni graf u kojem je svaki čvor povezan sa susjedima slične vrijednosti, oponašajući svojstvo “malog svijeta” iz teorije grafova, gdje se svaki čvor iz bilo kojeg drugog može dosegnuti kroz malen broj skokova (engl. *hops*). Pretraga započinje na najvišem, najrjeđe povezanom sloju grafa i postupno se spušta prema gušće povezanim nižim slojevima, sužavajući skup kandidata sve dok se ne pronađe skup vektora najbliži upitu. Ovakva struktura omogućuje pretraživanje čija složenost raste logaritamski, a ne linearno, s brojem pohranjenih vektora. Omjer brzine i točnosti pritom je konfigurabilan preko parametara poput broja veza po čvoru (M) te širine pretrage tijekom izgradnje i samog pretraživanja indeksa (*efConstruction*, *efSearch*) [19].

HNSW je implementiran i dostupan u većini specijaliziranih vektorskih baza podataka i proširenja korištenih u RAG sustavima, primjerice Weaviate, Pinecone, Qdrant, Milvus, LanceDB te `pgvector` proširenju za PostgreSQL [20]. Izbor između egzaktnu pretragu i ANN indeksa poput HNSW-a u konačnici je kompromis: egzaktna pretraga jamči potpunu točnost uz veći utrošak vremena i memorije, dok ANN pristupi omogućuju skalabilnost na znatno veće korpuse uz prihvatljiv, kontroliran gubitak preciznosti dohвата. Izbor primijenjen u ovom radu, gdje se za RAG verziju koristi egzaktna FAISS pretraga, a za GraphRAG verziju Neo4j vektorski indeksi temeljeni na HNSW-u, opisan je i obrazložen u poglavlju o izradi (potpoglavlja 4.5.3 i 4.6.1).

3.7.4. Segmentacija dokumenata

Nije svejedno kako se dokumenti dijele prije pohrane u vektorsku bazu. Segmentacija se može provesti na razini tokena, rečenice, odlomka ili cijelog dokumenta, i taj odabir izravno utječe na to koliko će dohvaćeni sadržaj biti koristan modelu [18].

Gruba granularnost dohvaćanja (npr. dohvaćanje cijelih dokumenata) može pružiti širi kontekst, ali istovremeno može uvesti suvišan sadržaj koji potencijalno ometa jezični model u kasnijim (engl. *downstream*) zadacima. Fina granularnost dohvaćanja pruža preciznije informacije, ali može povećati složenost dohvaćanja i pohrane te ugroziti semantički integritet. Dohvaćanje na razini dokumenta funkcionira dobro kada zadatak zahtijeva sveobuhvatan kontekst, dok se dohvaćanje na razini odlomka (*chunk-level* i *passage-level*) fokusira na specifične odjeljke teksta, usklađujući kontekst s relevantnošću [18].

Odabir veličine i strategije segmentacije predstavlja važan dizajnerski izbor u RAG sustavima. Uspješni RAG modeli poput REALM, RAG i Atlas koriste pristup dohvaćanja na razini odlomka jer on pomaže u zadacima koji zahtijevaju specifične odgovore, pritom ne preopterećujući model sa suvišnim informacijama. Dohvaćanje na razini tokena, iako manje uobičajeno, može se koristiti za specijalizirane zadatke koji zahtijevaju točna podudaranja ili integraciju podataka izvan domene treniranja. Strategija segmentacije mora uzeti u obzir i ograničenja kontekstnog prozora modela, dijeljenje na manje cjeline omogućuje lakše pretraživanje i smanjuje rizik od prekoračenja maksimalne duljine ulaza [18].

U literaturi se konkretne strategije segmentacije obično klasificiraju u nekoliko osnovnih kategorija [18]. Segmentiranje fiksne veličine (*fixed-size chunking*) dijeli tekst na cjeline jednake, unaprijed zadane duljine (u tokenima ili znakovima), bez obzira na strukturu ili smisao teksta na granici dvaju odlomaka, zbog čega je najjednostavnije za implementaciju, ali i najsklonije prekidanju rečenica i gubitku konteksta. Segmentiranje s preklapanjem (*overlap chunking*) ublažava taj nedostatak tako da završni dio jednog odlomka ponovno uključi na početak sljedećeg, čime se smanjuje rizik da relevantna informacija bude prekinuta točno na granici dvaju segmenata. Semantičko segmentiranje (*semantic chunking*) granice odlomaka umjesto fiksnom duljinom određuje smislenim cjelinama teksta (rečenicama, temama ili odlomcima), čime bolje čuva cjelovitost značenja pojedinog segmenta. Napokon, segmentiranje temeljeno na strukturi ili sintaksi izvornog sadržaja oslanja se na parsiranje, tj. formalnu analizu strukture dokumenta (npr. raščlambu programskog koda u apstraktno sintaksno stablo, ili, kao u ovom radu, prepoznavanje naslova članaka unutar pravnog teksta) kako bi granice segmenata odgovarale stvarnim, semantički zaokruženim cjelinama izvora.

Funkcija `chunk_by_articles`, opisana u poglavlju o izradi RAG verzije chatbota (potpoglavlje 4.5.2), pripada upravo posljednjoj, strukturnoj kategoriji, budući da granice odlomaka ne određuje fiksna duljina teksta, već strukturno obilježje izvornog dokumenta (naslovi članaka zakona). Njezin zamjenski postupak (*fallback*) za dokumente bez prepoznate strukture, `chunk_text_with_overlap`, predstavlja klasično segmentiranje s preklapanjem opisano u prethodnom odlomku.

3.7.5. Optimizacija upita i ponovno rangiranje rezultata

Osnovni RAG postupak opisan u prethodnim potpoglavljima, u kojem se korisnički upit izravno vektorizira i uspoređuje s korpusom, u literaturi se naziva naivnim RAG-om (*naive RAG*) i predstavlja tek polazišnu točku. Njegova se učinkovitost u praksi dodatno poboljšava koracima koji se izvode prije i nakon same pretrage, a koji zajedno čine tzv. napredni RAG (*advanced RAG*) [20].

U koraku prije dohvata (*pre-retrieval*) cilj je poboljšati sami upit prije nego što se njime pretražuje vektorski indeks. Reformulacija upita (*query rewriting*) preoblikuje kratko ili nejasno korisničko pitanje u eksplicitniji oblik, čime se smanjuje jaz između formulacije pitanja i formulacije relevantnih odlomaka u korpusu. Proširivanje upita (*query expansion*) generira dodatne, srodne varijante ili podupite istog pitanja te nad svakim provodi zasebnu pretragu, čime se povećava pokrivenost relevantnih odlomaka. Usmjeravanje upita (*query routing*) odlučuje kojem se od više mogućih izvora ili indeksa upit treba uputiti, što je osobito relevantno u sustavima koji, poput GraphRAG-a opisanog u ovom radu, kombiniraju više izvora dohvata, vektorski indeks i graf bazu podataka [20].

Poseban oblik optimizacije upita predstavlja HyDE (*Hypothetical Document Embeddings*), umjesto izravne vektorizacije kratkog korisničkog pitanja, jezičnim se modelom najprije generira hipotetski odgovor na to pitanje, koji se zatim vektorizira i njime pretražuje korpus. Budući da hipotetski odgovor stilski i sadržajno više nalikuje odlomcima korpusa nego samo pitanje, ovaj pristup posebno može poboljšati dohvat kod kratkih ili nejasno formuliranih upita. Srodan pristup, RAG-Fusion, upit dijeli na više podupita, nad svakim provodi zasebnu vektorsku pretragu te dobivene rangirane liste spaja tehnikom recipročnog rangiranja (*Reciprocal Rank Fusion*, RRF) [20].

Nakon dohvata, korak ponovnog rangiranja (*reranking*) preciznije procjenjuje relevantnost već dohvaćenih kandidata prije nego što se proslijede modelu, čime se smanjuje broj odlomaka koji ulaze u konačan kontekst i filtrira manje relevantan sadržaj. Postoje tri glavne skupine pristupa: heurističke metode poput BM25, *Maximal Marginal Relevance* (MMR), koja balansira relevantnost i raznolikost rezultata, te RRF-ovi; specijalizirani *cross-encoder* modeli (npr. MonoBERT, MonoT5, MiniLM), koji upit i dohvaćeni odlomak kodiraju zajedno te izravno predviđaju mjeru njihove relevantnosti te pristup u kojem sami jezični model služi kao ocjenjivač relevantnosti (*LLM-as-judge*) [20].

RAG verzija chatbota opisana u ovom radu (potpoglavlje 4.5.4) koristi oblik heurističkog rangiranja, prag sličnosti, hijerarhijski bonus prema pravnoj snazi izvora i deduplikaciju gotovo identičnih odlomaka, bez posebne optimizacije upita prije dohvata i bez zasebnog cross-encoder ili LLM-as-judge koraka ponovnog rangiranja nakon dohvata. Ove tehnike, osobito reformulacija ili proširivanje kratkih upita i usmjeravanje upita između vektorskog i graf izvora, predstavljaju konkretan smjer budućeg poboljšanja, a razmatraju se i u poglavlju 7.5 kao mogući razlog zašto GraphRAG verzija nije donijela očekivano poboljšanje u odnosu na RAG.

3.8. GraphRAG

Objasni koncept knowledge grapha, entiteta i relacija, razliku spram vektorskog RAG-a te zašto je pogodan za međupovezane propise.

4. Opis izrade

Ovo poglavlje opisuje praktičnu izradu triju uspoređenih verzija chatbota za odgovaranje na pitanja iz hrvatskog vojnog zakonodavstva: osnovnog modela bez dohvata konteksta (*Vanilla*), modela proširenog dohvatom iz vektorske baze (RAG) te modela čiji se dohvat temelji na grafu pravnih dokumenata (GraphRAG). Najprije se opisuju domena i zahtjevi (poglavlje 4.1) te korištene programske biblioteke (poglavlje 4.2). Poglavlje 4.3 potom opisuje implementacijske elemente zajedničke svim trima verzijama (jezični model, okolina izvođenja, sistemski upit, postupak generiranja odgovora i evaluacije), nakon čega slijede poglavlja specifična za svaku pojedinu verziju: Vanilla (poglavlje 4.4), RAG (poglavlje 4.5) i GraphRAG (poglavlje 4.6).

4.1. Opis domene i zahtjevi

Hrvatsko vojno zakonodavstvo odabrano je kao domena za usporedbu triju pristupa dohvata konteksta (Vanilla, RAG i GraphRAG). Radi se o skupu propisa koji uređuju obranu, službu u Oružanim snagama RH i temeljno vojno osposobljavanje. Korpus se sastoji od četiri izvorna teksta s više od 500 članaka:

- **Ustav RH** (pročišćeni tekst iz 2018., 146 članaka, .docx)
- **Zakon o obrani** (2025., 128 članaka, .docx)
- **Zakon o službi u Oružanim snagama RH** (2025., 236 članaka, .docx)
- **Pravilnik o temeljnom vojnom osposobljavanju** (2025., 22 članaka, .pdf)

Dokumenti obuhvaćaju tri hijerarhijske razine (ustavnu, zakonsku i podzakonsku). To omogućuje testiranje sposobnosti dohvata informacija raspršenih po normativnoj hijerarhiji. Sustav je namijenjen osobama kojima je potrebna brza i razumljiva orijentacija u propisima:

- Kandidatima i ročnicima na temeljnom vojnom osposobljavanju
- Djelatnim i pričuvnim vojnim osobama
- Djelatnicima tijela državne uprave nadležnih za obranu
- Studentima i istraživačima

Sustav služi kao pomoćni alat za orijentaciju, a ne kao zamjena za pravno savjetovanje. Njegovi su ključni zahtjevi:

- **Odgovaranje na prirodnom jeziku** bez potrebe da korisnik poznaje pravnu strukturu.
- **Precizan dohvat** i navođenje konkretnog članka i stavka.
- **Povezivanje informacija** iz više dokumenata unutar jednog odgovora.

- **Dosljednost i provjerljivost** dobivenih informacija.

Evaluacijska pitanja (Prilog 1) zato uključuju i jednostavne upite (oslonjene na jedan članak) i složena pitanja koja traže sintezu više odredbi. Pravni propisi predstavljaju zahtjevan test za jezične modele iz tri razloga:

- **Stroga terminologija:** Jezik propisa je formalan i precizan. Male jezične razlike (primjerice između *ročnika*, *kandidata* i *djelatne vojne osobe*) direktno mijenjaju pravno značenje.
- **Rizik od halucinacija:** Odgovor bez točnog citata (članak i stavak) ili s netočnom tvrdnjom u pravu je neupotrebljiv i potencijalno štetan.
- **Normativna povezanost:** Ustavne, zakonske i podzakonske odredbe međusobno se nadopunjuju i isprepliću.

Zbog te potrebe za točnošću, citiranjem i povezivanjem hijerarhijskih izvora, vojno zakonodavstvo je idealno područje za usporedbu navedena tri pristupa izgradnje chatbota.

4.2. Korištene programske biblioteke

U nastavku je dan pregled svih vanjskih paketa koji se instaliraju (`pip install`) i uvoze unutar izrađenih Jupyter bilježnica, grupiran prema fazi izrade rada. Sve bilježnice pokreću se u okruženju Google Colab, zbog čega se koriste i pomoćni moduli `google.colab` te `huggingface_hub` za autentifikaciju i pristup datotekama.

4.2.1. Osnovni model bez dohvata konteksta (*Vanilla*)

U ovoj konfiguraciji jezični model generira odgovore isključivo na temelju znanja stečenog tijekom treniranja, bez dohvata vanjskog konteksta. Korišteni su sljedeći paketi:

bitsandbytes Omogućuje kvantizaciju modela (8-bit/4-bit), čime se značajno smanjuje potrošnja memorije prilikom učitavanja velikih jezičnih modela.

transformers Biblioteka tvrtke Hugging Face za rad s predtreniranim jezičnim modelima. Koristi se za učitavanje tokenizatora, modela i izgradnju obradnog tijeka za generiranje teksta.

accelerate Omogućuje optimizirano izvođenje modela na GPU-u i raspodjelu resursa prilikom učitavanja velikih modela.

sentencepiece Implementacija tokenizatora temeljenog na podriječima (*subword tokenizer*), koju zahtijevaju jezični modeli.

torch PyTorch, temeljni okvir za duboko učenje na kojem se izvode svi modeli.

huggingface_hub Modul `login` koristi se za autentifikaciju korisnika radi pristupa modelima s Hugging Face Huba.

google.colab Moduli `userdata` i `drive` omogućuju pristup spremljenim API ključevima i Google Disku unutar Colab okruženja.

pandas Rad s tabličnim podacima (spremanje i obrada rezultata).

os, json, re, time, copy Standardne Python biblioteke za rad s datotekama, JSON formatom, regularnim izrazima, mjerenjem vremena izvođenja i kopiranjem struktura podataka.

4.2.2. Retrieval-Augmented Generation (RAG)

Uz pakete iz prethodne faze, dodatno se instaliraju i koriste:

sentence-transformers Generira vektorske reprezentacije rečenica i odlomaka teksta koje se koriste za semantičku pretragu.

faiss-cpu Facebook AI Similarity Search, biblioteka za brzu pretragu sličnosti vektora, predstavlja vektorsku bazu podataka za dohvat najbližnjih odlomaka.

python-docx Čitanje sadržaja Microsoft Word (`.docx`) dokumenata koji čine izvornu bazu znanja.

pdfplumber Ekstrakcija teksta iz PDF dokumenata.

numpy Numeričke operacije nad nizovima, temelj za rad s vektorima.

sklearn.manifold.TSNE Metoda za redukciju dimenzionalnosti, korištena za vizualizaciju visokodimenzionalnih vektorskih reprezentacija u dvodimenzionalnom prostoru.

matplotlib, seaborn Biblioteke za izradu grafova i vizualizaciju podataka (npr. prikaz raspršenosti vektorske reprezentacije).

logging Standardna biblioteka za bilježenje tijeka izvođenja programa.

math Standardna matematička biblioteka.

4.2.3. GraphRAG

Uz zajedničke pakete iz prve dvije faze, dodatno se instaliraju i koriste:

neo4j Klijent za grafovsku bazu podataka Neo4j (`GraphDatabase`), koristi se za spremanje entiteta i relacija između njih te za dohvat konteksta putem graf upita, umjesto isključivo vektorske sličnosti.

tqdm Prikaz trake napretka (*progress bar*) prilikom obrade velikog broja dokumenata ili upita.

pathlib Objektно orijentiran rad s putanjama datoteka.

unicodedata Normalizacija Unicode znakova prilikom obrade teksta.

4.3. Zajednička implementacijska osnova triju verzija chat-bota

Kako bi usporedba triju pristupa, osnovnog modela bez dohvata konteksta (*Vanilla*), pristupa temeljenog na dohvat iz vektorske baze (*RAG*) te pristupa temeljenog na dohvat iz graf baze podataka (*GraphRAG*), bila valjana, sve tri implementacije dijele identičnu osnovu u pogledu jezičnog modela, okoline izvođenja, sistemskog upita i postupka generiranja odgovora. Jedina namjerna razlika među pristupima jest način na koji se dolazi do konteksta koji se prosljeđuje modelu, dok su svi ostali čimbenici kontrolirani kako bi se izolirao učinak same metode dohvata. U nastavku su opisani zajednički elementi.

4.3.1. Temeljni jezični model i konfiguracija kvantizacije

Sve tri verzije koriste isti predtrenirani jezični model, `utter-project EuroLLM-22B Instruct-2512`, učitani putem biblioteke `transformers`. Model se u svim slučajevima učitava kvantiziran na 4 bita s pomoću klase `BitsAndBytesConfig`, čime se smanjuju zahtjevi za memorijom grafičke procesorske jedinice bez izmjene arhitekture ili težina modela:

Isječak koda 1: Zajednička konfiguracija kvantizacije modela

```
1 model_name = "utter-project/EuroLLM-22B-Instruct-2512"
2
3 quantization_config = BitsAndBytesConfig(
4     load_in_4bit=True,
5     bnb_4bit_use_double_quant=True,
6     bnb_4bit_compute_dtype=torch.float16,
7     bnb_4bit_quant_type="nf4"
8 )
9
10 pipe = pipeline(
11     "text-generation",
12     model=model_name,
13     dtype=torch.float16,
14     device_map="auto",
15     model_kwargs={"quantization_config": quantization_config}
16 )
17
18 pipe.model.generation_config.max_length = 8192
```

Model se učitava preko istog `pipeline` sučelja biblioteke `transformers`, uz prosljeđivanje identične konfiguracije kvantizacije kao `model_kwargs`. Izračun se izvodi u polovičnoj preciznosti (`torch.float16`), a raspodjela slojeva modela na dostupne uređaje pre-

puštena je automatskom mehanizmu biblioteke `accelerate` (`device_map="auto"`). Vrijednost `torch.float16` navedena je na dvjema odvojenim razinama: parametar `dtype` sučelja `pipeline` određuje preciznost svih nekvantiziranih dijelova izračuna, dok parametar `bnb_4bit_compute_dtype` unutar `BitsAndBytesConfig` zasebno određuje preciznost u koju se 4-bitno kvantizirane težine privremeno dekvantiziraju neposredno prije matričnog množenja u kvantiziranim slojevima. Oba su parametra u ovoj konfiguraciji usklađena na `torch.float16` kako bi izračun bio dosljedan kroz cijeli model. Najveća dopuštena duljina konteksta modela ograničena je na 8192 tokena u sve tri verzije. Time je osigurano da eventualne razlike u kvaliteti ili brzini odgovora ne proizlaze iz različitih verzija ili konfiguracija modela, već isključivo iz razlike u pristupu dohvatima konteksta.

Konfiguracija kvantizacije oslanja se na dvije tehnike koje je uvela metoda QLoRA [21], a koje zajedno omogućuju veću uštedu memorije uz manji gubitak točnosti nego standardna 4-bitna kvantizacija. Tip kvantizacije `nf4` (`bnb_4bit_quant_type="nf4"`) odnosi se na format *NF4 (4-bit NormalFloat)*, čije su razine kvantizacije, za razliku od ravnomjerno raspoređenih razina standardnog cjelobrojnog 4-bitnog zapisa, izvedene iz kvantila normalne razdiobe. Budući da su težine pretreniranih neuronskih mreža približno normalno raspodijeljene oko nule, ovakav raspored razina bolje odgovara stvarnoj raspodjeli vrijednosti koje se kvantiziraju, čime se pri istoj duljini zapisa (4 bita po parametru) postiže manja pogreška kvantizacije nego kod standardnog 4-bitnog formata [21]. Parametar `bnb_4bit_use_double_quant=True` uključuje *dvostruku kvantizaciju (double quantization)*, postupak kojim se dodatnoj kvantizaciji podvrgavaju i same kvantizacijske konstante (skalirajući faktori) nastale u prvom koraku kvantizacije težina, umjesto da se te konstante, kao uobičajeno, pohranjuju u punoj 32-bitnoj preciznosti. Time se prosječna memorijska cijena po parametru modela dodatno smanjuje za približno 0,37 bita bez mjerljivog utjecaja na točnost, što za veće modele donosi uštedu reda veličine nekoliko gigabajta [21].

4.3.2. Okolina izvođenja

Sve tri bilježnice izvode se unutar okruženja Google Colab te dijele isti postupak pripreme okoline:

- pristup Google disku putem naredbe `drive.mount()`, koji služi kao trajna pohrana preuzetih modela i predobrađenih podataka između sesija;
- autentifikacija na Hugging Face Hub putem tokena pohranjenog u Colab korisničkim podacima (`userdata.get('HF_TOKEN')`) i funkcije `login()`;
- korištenje lokalno predmemoriranog modela postavljanjem varijabli okoline `HF_HOME`, `HF_HUB_OFFLINE` i `TRANSFORMERS_OFFLINE`, čime se izbjegava ponovno preuzimanje modela s interneta prilikom svakog pokretanja.

Ujedačenom okolinom izvođenja osigurano je da sve tri verzije koriste identičnu inačicu modela iz iste predmemorije te se time isključuje mogućnost da razlike u rezultatima proizlaze iz nedosljedne postave sustava.

Svi eksperimenti provedeni su u okruženju Google Colab uz grafičku karticu NVIDIA A100 s 40 GB memorije (VRAM), čija je iskorištenost tijekom triju uzastopnih sesija izvođenja prikazana na Slici 5. Kao zajednički generativni model za sve tri arhitekture — Vanilla, RAG i GraphRAG — odabran je model EuroLLM-22B-Instruct u 4-bitnoj kvantizaciji (NF4). Time je osigurano da model u cijelosti stane u dostupnu memoriju grafičke kartice bez prebacivanja pojedinih slojeva na procesor ili disk, čime bi se dodatno usporilo učitavanje i izvođenje te narušila usporedivost mjerenja. Korištenje istoga modela i istovjetne konfiguracije okruženja u sve tri arhitekture ključno je za metodološku ispravnost istraživanja, budući da razlike u izmjenim performansama tako proizlaze isključivo iz svojstava pojedine arhitekture, a ne iz razlika u hardveru ili kapacitetu jezičnog modela.



Slika 5: Iskorištenost systemske i GPU memorije te diska tijekom triju uzastopnih sesija izvođenja u okruženju Google Colab (9. 7. 2026.).

Zbog ograničenja izvršnog okruženja Google Colab, u kojem se dodijeljeni virtualni stroj i njegov lokalni disk (`/content`) brišu po isteku svake sesije, ponovno preuzimanje jezičnog modela sa sustava Hugging Face pri svakom pokretanju bilježnice predstavljalo je usko grlo. Model EuroLLM-22B-Instruct u formatu `safetensors` zauzima približno 44 GB, a pri opaženim brzinama preuzimanja od oko 7 MB/s njegovo dohvaćanje trajalo bi više od sat vremena po sesiji. Kako bi se osigurala reproducibilnost i vremenska učinkovitost, model je jednokratno pohranjen (keširan) na trajni prostor za spremanje Google Drive putem varijable okruženja `HF_HOME`, koja usmjerava lokalnu predmemoriju biblioteke Hugging Face na montirani direktorij Drive. Time se identičan model dijeli među svim trima bilježnicama (Vanilla, RAG i GraphRAG), čime se uklanja višestruko preuzimanje i jamči da sve tri arhitekture koriste jednake težine modela, što je preduvjet za metodološki ispravnu usporedbu.

Budući da se biblioteka Hugging Face pri učitavanju usmjerava na predmemoriju na Driveu (varijabla `HF_HOME`), model se u radnu memoriju grafičkog procesora učitava izravno iz

tog trajnog keša, uz postavljen offline način rada (`HF_HUB_OFFLINE`), čime se izbjegava svako ponovno obraćanje sustavu Hugging Face nakon inicijalnog preuzimanja. Za ubrzanje samog inicijalnog keširanja korištena je biblioteka `hf_transfer`, koja omogućuje paralelno preuzimanje dijelova modela (engl. *shards*). Sama kvantizacija modela u 4-bitni format (NF4) provodi se tek pri učitavanju u memoriju grafičkog procesora, dok na disku ostaje pohranjen model pune preciznosti, zbog čega navedeni zahtjevi pohrane odgovaraju nekvantiziranoj veličini modela.

4.3.3. Sistemski upit (*system prompt*)

Sve tri verzije chatbota koriste potpuno identičan sistemski upit kojim se modelu definira uloga i pravila odgovaranja:

“Ti si pravni stručnjak specijaliziran za zakonodavstvo Republike Hrvatske, posebno za Zakon o obrani, Zakon o službi u Oružanim snagama RH i pripadajuće pravilnike.

Pravila odgovaranja:

- 1. Jezik odgovora: hrvatski, latinica.*
- 2. Navedi konkretne članke zakona, brojčane vrijednosti i nadležna tijela.*
- 3. Ako je dostupan kontekst, koristi SAMO informacije iz njega.*
- 4. Ako kontekst ne sadržava odgovor, odgovori na temelju općeg znanja i dodaj napomenu da informacija nije pronađena u dostupnim dokumentima.”*

Kod verzija RAG i GraphRAG dohvaćeni se kontekst nadovezuje na ovaj isti sistemski upit (`SYSTEM_PROMPT + “\n\n Kontekst za odgovor:\n” + context`), dok kod verzije Vanilla kontekst izostaje jer dohvat nije proveden. Zadržavanjem identičnog temeljnog upita osigurano je da se ponašanje modela ne mijenja zbog formulacije uputa, već isključivo zbog prisutnosti ili odsutnosti dohvaćenog konteksta.

4.3.4. Postupak generiranja odgovora

Generiranje odgovora u sve tri verzije provodi se putem funkcije `generate_response()`, koja u svim implementacijama slijedi isti postupak:

- poruke se pretvaraju u konačan tekstualni ulaz s pomoću predloška `pipe.tokenizer.apply_chat_template()`, čime se osigurava da format upita odgovara formatu na kojem je model instruiran;
- izračun najveće dopuštene duljine ulaza kao razlike najveće duljine konteksta modela i broja tokena rezerviranih za generiranje (`max_allowed_input = model_max_length - max_new_tokens`), uz ispis upozorenja ako broj tokena ulaza prelazi dopušteni limit;
- postavljanje parametara generiranja putem objekta `GenerationConfig`, pri čemu se koriste isti nazivi i tip parametara (`max_new_tokens`, `temperature`,

`do_sample`) te ista vrijednost kazne za ponavljanje tokena (`repetition_penalty=1.1`) u sve tri verzije;

4. konačan poziv modela za generiranje odgovora obavlja se uz istu, determinističku konfiguraciju: `temperature=0.3` i `do_sample=False`, čime se isključuje slučajnost pri odabiru tokena i osigurava ponovljivost odgovora između pokretanja, što je nužno za pouzdanu usporedbu rezultata triju pristupa;
5. generirani odgovor se dodatno obrađuje: ako ne završava interpunkcijskim znakom (`.`, `?` ili `!`), tekst se skraćuje do posljednjeg potpunog interpunkcijskog znaka kako bi se izbjegao odgovor prekinut usred rečenice zbog dosezanja ograničenja `max_new_tokens`.

Jedina razlika u samom pozivu generiranja jest dodatni korak dohvata konteksta koji prethodi pozivu (kod RAG i GraphRAG verzija), dok sama mehanika generiranja odgovora, parametri, konfiguracija, obrada ulaza i naknadna obrada izlaza, ostaje nepromijenjena u sve tri verzije.

4.3.5. Skup podataka za evaluaciju i postupak paketne obrade

Sve tri verzije koriste identičan skup testnih pitanja i očekivanih odgovora, pohranjen u datoteci `pitanja_odgovori.json` na Google Disku. Datoteka sadrži metapodatke (ukupan broj pitanja i raspodjelu po kategorijama) te sama pitanja grupirana po kategorijama, pri čemu se za svaki upit čuvaju polja `pitanje` i `odgovor` (referentni, očekivani odgovor).

Nad učitanim skupom podataka sve tri bilježnice provode identičan postupak paketne obrade:

1. iteracija kroz sva pitanja iz svih kategorija, uz bilježenje kategorije kojoj pitanje pripada;
2. poziv odgovarajuće funkcije za generiranje odgovora (`vanilla_chat`, odnosno ekvivalentne funkcije u RAG i GraphRAG verzijama) i mjerenje vremena potrebnog za generiranje svakog pojedinog odgovora;
3. spremanje rezultata (redni broj, kategorija, pitanje, očekivani odgovor, generirani odgovor i vrijeme generiranja) u zajednički oblikovan `pandas.DataFrame`;
4. izvoz prikupljenih rezultata u CSV datoteku (`rezultati.csv`) na Google Disku, uz ispis ukupnog broja obrađenih pitanja i prosječnog vremena odgovora.

Korištenjem istog skupa pitanja i istog postupka mjerenja i spremanja rezultata u sve tri verzije omogućena je izravna, poravnata usporedba odgovora triju pristupa po pojedinom pitanju u fazi evaluacije (poglavlje o evaluaciji).

4.3.6. Evaluacija rezultata

rouge-score Implementacija ROUGE metrike koja uspoređuje preklapanje n-grama između generiranog odgovora i referentnog (točnog) odgovora; koristi se za evaluaciju kvalitete generiranog teksta.

bert-score Metrika koja mjeri semantičku sličnost generiranog i referentnog teksta s pomoću kontekstualnih BERT vektorskih reprezentacija, umjesto doslovnog preklapanja riječi.

sentence-transformers Ovdje se koristi za izračun semantičke sličnosti (*cosine similarity*) između vektora generiranih i referentnih odgovora.

sklearn.metrics.pairwise.cosine_similarity Funkcija za izračun kosinusne sličnosti između parova vektora.

pandas, numpy Obrada i agregacija rezultata evaluacije.

matplotlib, seaborn, matplotlib.pyplot Vizualizacija rezultata evaluacije (usporedba metrika po modelima/pristupima).

json, logging Spremanje rezultata u JSON formatu i bilježenje tijeka evaluacije.

4.4. Vanilla verzija chatbota

Vanilla verzija predstavlja osnovni, referentni pristup u kojem jezični model odgovara na korisnička pitanja isključivo na temelju znanja stečenog tijekom predtreniranja, bez ikakvog dodatnog dohvata konteksta iz vanjskih izvora. Svrha ove verzije jest uspostaviti polazišnu točku prema kojoj se u kasnijoj evaluaciji uspoređuje doprinos pristupa temeljenih na dohvatu informacija, RAG i GraphRAG. Model, konfiguracija kvantizacije, okolina izvođenja, sistemski upit, postupak generiranja odgovora te postupak paketne evaluacije identični su onima opisanim u zajedničkoj implementacijskoj osnovi. U ovom se poglavlju opisuje isključivo ono što je specifično za Vanilla pristup.

4.4.1. Funkcija `vanilla_chat`

Za razliku od RAG i GraphRAG verzija, kod kojih se prije poziva generiranja provodi korak dohvata relevantnog konteksta, Vanilla verzija korisničko pitanje prosljeđuje modelu izravno, bez ikakvog posredničkog koraka pretrage:

Isječak koda 2: Slaganje upita i poziv generiranja u Vanilla verziji

```
1 def vanilla_chat(user_question):
2     messages = [
3         {"role": "system", "content": SYSTEM_PROMPT},
4         {"role": "user", "content": f"{user_question}\n\nOdgovori NA
        HRVATSKOM JEZIKU koristeći LATINICU."}
5     ]
6     return generate_response(messages, max_new_tokens=512, temperature=0.3,
        do_sample=False)
```

Poruka poslana modelu sastoji se od samo dva elementa: zajedničkog sistemskog upita (SYSTEM_PROMPT) i korisničkog pitanja, kojemu se dodaje eksplicitna uputa da odgovor mora biti na hrvatskom jeziku i latinici. Budući da model nema pristup dokumentima specifičnim za domenu (Zakon o obrani, Zakon o službi u Oružanim snagama RH i pripadajući pravilnici), odgovor u potpunosti ovisi o znanju koje je model usvojio tijekom predtreniranja. Očekuje se da će ovakav pristup biti osjetljiviji na činjenične pogreške i zastarjele ili nepotpune informacije, osobito kod pitanja koja zahtijevaju poznavanje konkretnih brojčanih vrijednosti ili odredbi pojedinih članaka zakona, što ovu verziju čini korisnom referentnom točkom za procjenu koristi dohvata konteksta.

4.4.2. Izlazni podaci

Rezultati dobiveni ovom verzijom pohranjuju se u zajednički `rezultati.csv` korišten za usporedbu svih triju pristupa, pri čemu Vanilla verzija u njega upisuje sljedeće, pristupu specifične stupce:

- `vanilla_odgovor` - odgovor koji je model generirao za dano pitanje bez dohvaćenog konteksta;
- `vanilla_vrijeme_sekunde` - vrijeme (u sekundama) potrebno za generiranje odgovora, zaokruženo na dvije decimale.

Ostali stupci (redni broj, kategorija, pitanje i očekivani odgovor) zajednički su svim verzijama i opisani su u okviru postupka paketne obrade u zajedničkoj implementacijskoj osnovi.

4.5. RAG verzija chatbota

RAG (*Retrieval-Augmented Generation*) verzija proširuje Vanilla pristup dodatnim korakom koji se izvodi prije generiranja odgovora: dohvaćom najrelevantnijih odlomaka iz korpusa pravnih dokumenata na temelju semantičke sličnosti s korisničkim pitanjem. Dohvaćeni se odlomci potom umeću u sistemski upit kao dodatni kontekst, čime se modelu omogućuje da odgovor temelji na stvarnom sadržaju zakona i pravilnika, umjesto isključivo na znanju stečenom tijekom predtreniranja. U nastavku je opisano ono što je specifično za ovu verziju: model

za generiranje vektorskih reprezentacija, priprema korpusa, vektorski indeks, postupak dohвата odlomaka te njegova integracija u zajednički postupak generiranja odgovora.

4.5.1. Model za generiranje vektorskih reprezentacija

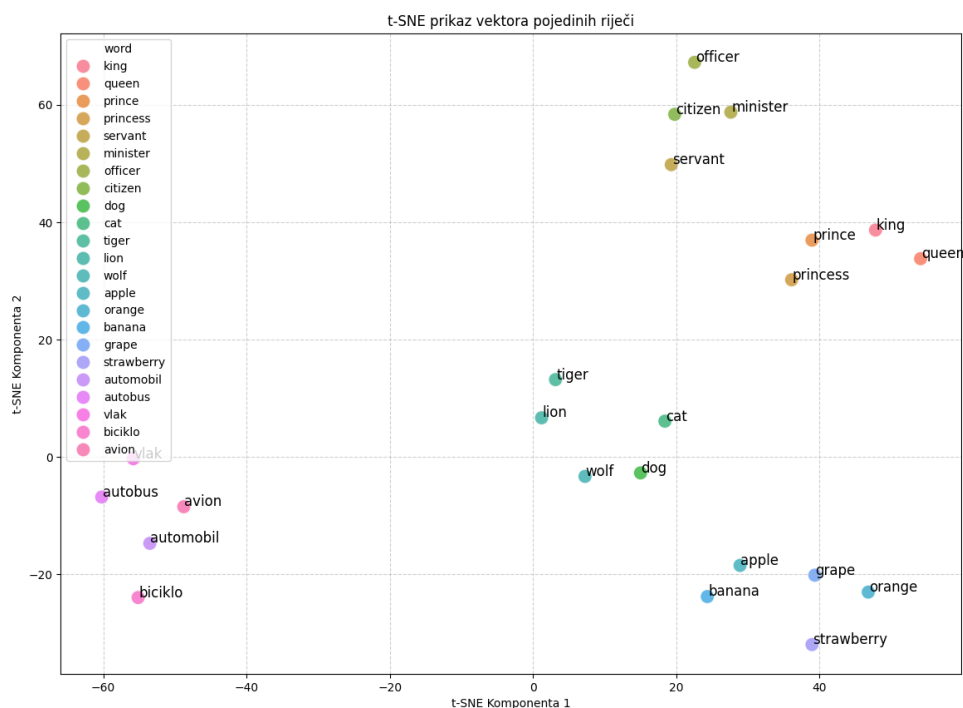
Za pretvorbu teksta u vektorske reprezentacije koristi se predtrenirani višezječni model `intfloat/multilingual-e5-large`, učitani putem biblioteke `sentence-transformers`

Isječak koda 3: Učitavanje modela za generiranje vektorskih reprezentacija

```
1 embed_model = SentenceTransformer("intfloat/multilingual-e5-large")
```

U skladu s konvencijom modela obitelji E5, tekstovima se prije generiranja vektora dodaje prefiks koji označava njihovu ulogu: dokumenti u korpusu označavaju se prefiksom `"passage: "`, dok se korisnički upiti prilikom pretrage označavaju prefiksom `"query: "`. Ovim razlikovanjem model generira vektore prilagođene svrsi teksta (dokument naspram upita), što poboljšava kvalitetu semantičkog podudaranja pri pretrazi.

Radi ilustracije svojstava dobivenog vektorskog prostora, u bilježnici je uključen i demonstracijski primjer u kojem se manji skup pojmova vektorizira te prikazuje u dvije dimenzije metodom t-SNE, čime se vizualno potvrđuje da model grupira semantički srodne pojmove (npr. nazivi vozila ili nazivi zanimanja) blizu jedan drugome u vektorskom prostoru (Slika 6). Ovaj primjer služi isključivo kao ilustracija svojstava modela vektorskih reprezentacija i ne sudjeluje u stvarnom dohvatu ili generiranju odgovora.



Slika 6: Prikaz vektoriziranih riječi u dvije dimenzije t-SNE metodom, gdje se semantički srodni pojmovi grupiraju u blizini jedan drugoga.

4.5.2. Priprema korpusa dokumenata

Korpus se gradi iz dokumenata pohranjenih u mapi `Dokumenti` na Google Disku, pri čemu su podržani formati `.txt`, `.docx` i `.pdf` (tekst se iz njih izvlači funkcijama `extract_text_from_docx` odnosno `extract_text_from_pdf` korištenjem `python-docx` i `pdfplumber`). Prilikom učitavanja svakog dokumenta prepoznaje se i njegova hijerarhijska razina na temelju naziva datoteke (`detect_document_hierarchy`): Ustav (razina 1), Zakon (razina 2), Pravilnik (razina 3) ili ostalo (razina 4). Ova se informacija kasnije koristi za rangiranje rezultata pretrage prema pravnoj snazi izvora.

Svaki dokument dijeli se na manje segmente funkcijom `chunk_by_articles`, koja tekst prvenstveno segmentira prema naslovima članaka (uzorak "Članak N."), tako da svaki odlomak korpusa najčešće odgovara jednom zakonskom članku ili njegovom dijelu, uz ograničenje najveće duljine odlomka (1200 znakova) i najmanje duljine (50 znakova) kako bi se izbjegli kratki odlomci. Ako u tekstu nije moguće prepoznati strukturu članaka, primjenjuje se generička segmentacija s preklapanjem (`chunk_text_with_overlap`, veličina odlomka 800 znakova, preklapanje 200 znakova), koja tekst dijeli po odlomcima uz zadržavanje dijela prethodnog teksta na početku sljedećeg odlomka radi očuvanja konteksta na granicama.

4.5.3. Vektorski indeks

Dobiveni odlomci pretvaraju se u vektore (uz prefiks "passage: ") i pohranjuju u vektorski indeks biblioteke `faiss`, tipa `IndexFlatIP` (pretraga temeljena na skalarnom produktu), uz prethodnu L2-normalizaciju vektora, čime skalarni produkt postaje ekvivalentan kosinusnoj sličnosti:

Isječak koda 4: Izgradnja FAISS indeksa nad korpusom dokumenata

```
1 embedding_dim = embed_model.get_sentence_embedding_dimension()
2 faiss_index = faiss.IndexFlatIP(embedding_dim)
3
4 def add_documents_to_corpus(texts, metadatas=None):
5     prefixed_texts = ["passage: " + t for t in texts]
6     embeddings = embed_model.encode(prefixed_texts, convert_to_numpy=True)
7     faiss.normalize_L2(embeddings)
8     faiss_index.add(embeddings)
```

Uz svaki odlomak pohranjuju se i pripadni metapodaci (naziv datoteke, tip dokumenta i hijerarhijska razina), koji se koriste u kasnijim koracima rangiranja i raznolikosti rezultata.

Odabir tipa `IndexFlatIP` znači da s pomoću FAISS-a korpus se pretražuje egzaktno, a ne aproksimativno (ANN) (potpoglavlje 3.7.3), svaki upit uspoređuje se sa svim pohranjenim vektorima, čime je zajamčeno da će biti pronađeni stvarno najsličniji odlomci, bez gubitka preciznosti koji unose ANN indeksi poput HNSW-a. Takav pristup je opravdan upravo zbog veličine korpusa, koji broji nekoliko tisuća odlomaka pravnih dokumenata, a ne milijune, pa je linearna složenost pretrage zanemariva u odnosu na vrijeme generiranja odgovora samim je-

zičnim modelom. Za razliku od RAG verzije, GraphRAG verzija (potpoglavlje 4.6.1) oslanja se na Neo4j vektorske indekse, koji ANN pretragu interno provode HNSW algoritmom, čime dvije implementirane verzije chatbota ujedno ilustriraju oba pristupa opisana u teorijskom pregledu, egzaktnu i aproksimativnu pretragu najbližih susjeda.

4.5.4. Postupak dohvata relevantnih odlomaka

Funkcijom `retrieve_relevant_passages` je implementiran dohvat relevantnih odlomaka za dano pitanje kojom se provodi višestupanjski postupak:

1. **Početna pretraga** korisnički upit vektorizira se uz prefiks `"query: "`, nakon čega se u FAISS indeksu pretražuje širi skup kandidata (`fetch_k`, do pet puta veći od traženog broja rezultata, ograničen na veličinu korpusa), kako bi se ostavio prostor za naknadno filtriranje i rangiranje;
2. **Filtriranje po pragu sličnosti** kandidati čija je sličnost s upitom ispod praga `score_threshold = 0.30` odbacuju se;
3. **Hijerarhijsko podizanje ocjene** rezultatima iz dokumenata više pravne snage (npr. zakon naspram pravilnika) dodaje se manji bonus na ocjenu sličnosti, proporcionalan njihovoj hijerarhijskoj razini, čime se pri jednakoj semantičkoj sličnosti daje prednost mjerodavnijem izvoru;
4. **Uklanjanje gotovo identičnih odlomaka** odlomci iz istog dokumenta koji se preklapaju u više od 92% riječi (Jaccardova sličnost skupova riječi) smatraju se duplikatima te se izostavljaju iz konačnog skupa;
5. **Raznolikost izvora** uvodi se ograničenje najvećeg broja odlomaka iz istog dokumenta (`per_doc_cap`) kako konačni kontekst ne bi bio dominiran odlomcima iz jednog jedinog propisa, uz naknadno popunjavanje preostalim mjestima iz ranije izostavljenih kandidata ako selektiranih odlomaka nema dovoljno.

Rezultat funkcije jest konačan popis od najviše `top_k` odlomaka (tekst, ocjena sličnosti i metapodaci) koji se proslijeđuju u sistemski upit modela.

4.5.5. Integracija dohvata u generiranje odgovora

Zajednička funkcija `generate_response` proširena je u ovoj verziji parametrima `use_rag` i `rag_top_k`. Kada je dohvat konteksta uključen, iz posljednje korisničke poruke izdvaja se upit, dohvaćaju se relevantni odlomci te se oblikovani kontekst (svaki odlomak numeriran i označen izvorom) nadovezuje na sadržaj sistemske poruke:

Isječak koda 5: Umetanje dohvaćenog konteksta u sistemsku poruku

```
1 if use_rag:
2     retrieved = retrieve_relevant_passages(user_query, top_k=rag_top_k)
3     context_text = "\n\n".join(
4         f"[{i}] {r['text']} (izvor: {r['metadata']['file']})"
5         for i, r in enumerate(retrieved, start=1)
6     )
7     final_messages[0]["content"] += "\n\nKontekst za odgovor:\n" +
    context_text
```

Ako nakon umetanja konteksta ukupan broj tokena ulaza premaši dopušteni limit, kontekst se postupno skraćuje uklanjanjem posljednjih (najmanje relevantnih) odlomaka sve dok ulaz ne stane unutar dopuštene duljine, čime se izbjegava odbacivanje cijelog upita zbog prekoračenja limita modela.

4.5.6. Prilagodba parametara prema kategoriji pitanja

Za razliku od Vanilla verzije, broj dohvaćenih odlomaka i najveći broj tokena odgovora u RAG verziji nisu fiksni, već ovise o kategoriji pitanja iz skupa za evaluaciju (jednostavno, kompleksno_jedan_dokument, kompleksno_vise_dokumenata):

Isječak koda 6: Broj dohvaćenih odlomaka i tokena odgovora prema kategoriji pitanja

```
1 RAG_TOP_K = {
2     'jednostavno': 5,
3     'kompleksno_jedan_dokument': 8,
4     'kompleksno_vise_dokumenata': 12,
5 }
6
7 RAG_MAX_TOKENS = {
8     'jednostavno': 384,
9     'kompleksno_jedan_dokument': 640,
10    'kompleksno_vise_dokumenata': 1024,
11 }
```

Ovakvom se prilagodbom kompleksnijim pitanjima, koja po definiciji zahtijevaju informacije iz više izvora ili opsežnije obrazloženje, omogućuje veći broj dohvaćenih odlomaka konteksta i duži odgovor, dok se za jednostavna pitanja broj odlomaka i duljina odgovora ograničavaju radi konciznosti i brzine generiranja.

4.5.7. Funkcija rag_chat i izlazni podaci

Poziv chatbota u ovoj verziji obavlja se funkcijom `rag_chat`, koja na temelju kategorije pitanja očitava odgovarajući `top_k` i `max_new_tokens` te poziva `generate_response` uz `use_rag=True`, zadržavajući pritom istu determinističku konfiguraciju (`temperature=0.3`, `do_sample=False`) kao i ostale verzije.

Rezultati se ne spremaju u novu datoteku, već se postojeći `rezultati.csv` (generiran u Vanilla verziji) učitava i nadopunjuje dvama dodatnim stupcima:

- `rag_odgovor` - odgovor generiran uz dohvaćeni kontekst;
- `rag_vrijeme_sekunde` - vrijeme (u sekundama) potrebno za generiranje odgovora, uključujući vrijeme dohвата odlomaka.

Ovim se pristupom u jednoj datoteci postupno objedinjuju odgovori svih triju verzija za identičan skup pitanja, što u fazi evaluacije omogućuje izravnu, redak-po-redak usporedbu rezultata.

4.6. GraphRAG verzija chatbota

GraphRAG verzija zamjenjuje vektorsku pretragu nad plošnim skupom odlomaka (kao u RAG verziji) strukturiranim grafom pravnih dokumenata pohranjenim u graf bazi podataka Neo4j. Umjesto da se relevantnost odlomka procjenjuje isključivo na temelju semantičke sličnosti s upitom, model dokumenata predstavljen je kao graf članaka, stavaka, entiteta i eksplicitnih odnosa među njima (upućivanja na druge članke, dijeljeni pojmovi, tematska srodnost), pa se konačni kontekst gradi kombinacijom vektorske pretrage i obilaska grafa (*graph traversal*) iz najrelevantnijih čvorova prema njihovim susjedima. Cilj je omogućiti dohvat informacija koje su međusobno pravno povezane, ali ne nužno i tekstualno slične, što je osobito važno kod pitanja koja objedinjuju odredbe iz više propisa.

4.6.1. Graf baza podataka i shema

Graf se pohranjuje u instanci Neo4j (AuraDB) smještenoj u oblaku, na koju se spaja putem službenog Python klijenta (`neo4j.GraphDatabase`):

Isječak koda 7: Povezivanje na Neo4j i pomoćna funkcija za izvršavanje upita

```
1 neo4j_driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER,  
2 NEO4J_PASSWORD))  
3  
4 def neo4j_run(query, params=None):  
5     with neo4j_driver.session() as session:  
6         return list(session.run(query, params or {}))
```

Schema grafa definirana je sljedećim tipovima čvorova:

- **Zakon** predstavlja jedan pravni akt (zakon, pravilnik ili Ustav);
- **Članak** predstavlja jedan članak propisa; nosi puni tekst članka i njegovu vektorskih reprezentacija;

- **Stavak** predstavlja pojedini stavak unutar članka, također s vlastitom vektorskom reprezentacijom, radi preciznijeg pogotka kod uskih pitanja;
- **Entitet** normalizirani naziv pravnog pojma ili tijela spomenutog u članku (npr. naziv dužnosti, tijela ili instituta), dobiven prepoznavanjem imenovanih entiteta;
- **Obveza** tekstualni izvadak klasificiran kao obveza, zabrana ili pravo.

Čvorovi su povezani usmjerenim relacijama: `SADRZI` (sadrži članak), `IMA_STAVAK` (ima stavak), `UPUCUJE_NA` (jedan članak izričito upućuje na drugi), `SPOMINJE` (članak spominje entitet), `SADRZI_OBVEZU` (članak sadrži obvezu) te `ISTA_TEMA` (automatski izvedena tematska poveznica između članaka različitih propisa, opisana u potpoglavlju 4.6.3). Jedinstvenost identifikatora osigurana je ograničenjima (*constraints*) nad poljima `id` čvorova `Zakon`, `Clanak` i `Stavak`, dok su nad poljima vektorske reprezentacije čvorova `Clanak` i `Stavak` definirani vektorski indeksi (`clanak_embedding`, `stavak_embedding`) s kosinusnom mjerom sličnosti, korišteni kasnije u fazi dohвата.

4.6.2. Izgradnja grafa iz dokumenata

Isti izvorni dokumenti (`.docx`, `.pdf`) koji se u RAG verziji dijele na odlomke ovdje se strukturirano raščlanjuju na razinu članaka i stavaka funkcijama `graph_parse_docx` i `graph_parse_pdf`, koje prepoznaju naslove članaka (uzorak "Članak N.") i stavaka (uzorak " (N) ") regularnim izrazima. Tekst koji prethodi prvom prepoznatom članku, kao i dokumenti u kojima struktura članaka uopće nije prepoznata, ne odbacuju se, već se dijele na pomoćne pseudo-članke fiksne duljine (`_fallback_pseudo_clanci`), čime se osigurava da nijedan dio izvornog teksta ne izostane iz grafa. Kod `.docx` dokumenata dodatno se pokušava odrediti stvarni naziv propisa (umjesto naziva datoteke) pretragom prvih 15 odlomaka za oblikovanim stilom naslova (*heading*), čime se u graf i kasniji prikaz konteksta unosi čitljiviji naziv zakona ili pravilnika.

Budući da je izgradnja grafa deterministički postupak nad istim izvornim dokumentima, prije svakog pokretanja ingestije graf se u potpunosti briše naredbom `MATCH (n) DETACH DELETE n`, nakon čega se iznova gradi od nule. Ovime se osigurava da rezultati evaluacije uvijek odgovaraju jednom, dosljednom stanju grafa, bez zaostalih čvorova ili veza iz prethodnih, potencijalno izmijenjenih verzija dokumenata ili logike ingestije.

Tijekom raščlambe svakog članka dodatno se prepoznaju:

- **unutartekstna upućivanja** na druge članke (npr. „članka 15. ovog Zakona” ili „članka 3. Ustava Republike Hrvatske”). Naziv ciljnog propisa iz fraze (npr. „ovog Zakona”, „Ustava Republike Hrvatske”, „Zakona o službi u Oružanim snagama RH”) razrješava se u konkretan identifikator čvora `Zakon` funkcijom `rijesi_zakon_id`, koja pri prvom pozivu izgrađuje i predmemorira (`_AKT_ID_CACHE`) preslikavanje između prepoznatih obrazaca u nazivu propisa (Ustav, Zakon o službi u OSRH, Zakon o obrani) i njihovih stvarnih ID-eva u grafu, dohvaćenih upitom nad već

upisanim čvorovima `Zakon`; tako razriješeno upućivanje upisuje se kao relacija `UPUCUJE_NA` prema članku ciljnog propisa;

- **imenovani entiteti** unutar teksta članka, prepoznati modelom za prepoznavanje imenovanih entiteta za hrvatski jezik (`classla/bcms-bertic-ner`). Budući da NER model ima ograničenje na duljinu ulaznog teksta, dulji se tekst članka prije prepoznavanja dijeli na preklapajuće prozore od 512 znakova s pomakom od 384 znaka (`graph_extract_entities`), čime se osigurava da entiteti blizu granice prozora ne budu propušteni; podriječi prepoznate kao nastavci tokena (obilježene prefiksom `"###"`, karakteristične za BERT-ov *tokenizer*) odbacuju se kao nepotpuni entiteti. Prepoznati entiteti dodatno se normaliziraju uklanjanjem padežnih nastava (jednostavni algoritam skraćivanja sufiksa) kako bi se isti pojam u različitim padežima povezao s istim čvorom `Entitet`; generički pojmovi koji se pojavljuju u gotovo svakom članku (npr. „Republika Hrvatska”, „Oružane snage”) izuzimaju se popisom zaustavnih entiteta (`STOP_ENTITETI`) kako ne bi umjetno povezivali nesrodne članke;
- **odredbe o obvezama** rečenice koje sadrže obrasce tipične za obveznu („dužan”, „mora”), zabranjujuću („zabranjeno”, „ne smije”) ili ovlašćujuću („ima pravo”, „ovlašten”) normu, klasificirane jednostavnim pretraživanjem ključnih riječi i pohranjene kao čvorovi `Obveza`.

Funkcija `graph_ingest_zakon` potom za svaki članak izračunava vektorsku reprezentaciju (istim modelom `intfloat/multilingual-e5-large` korištenim u RAG verziji, uz prefiks `"passage: "`) i upisuje čvor članka, njegove stavke, prepoznata upućivanja, entitete i obveze u graf s pomoću `MERGE` upita, čime se osigurava idempotentnost ponovljenog pokretanja ingestije nad istim dokumentima.

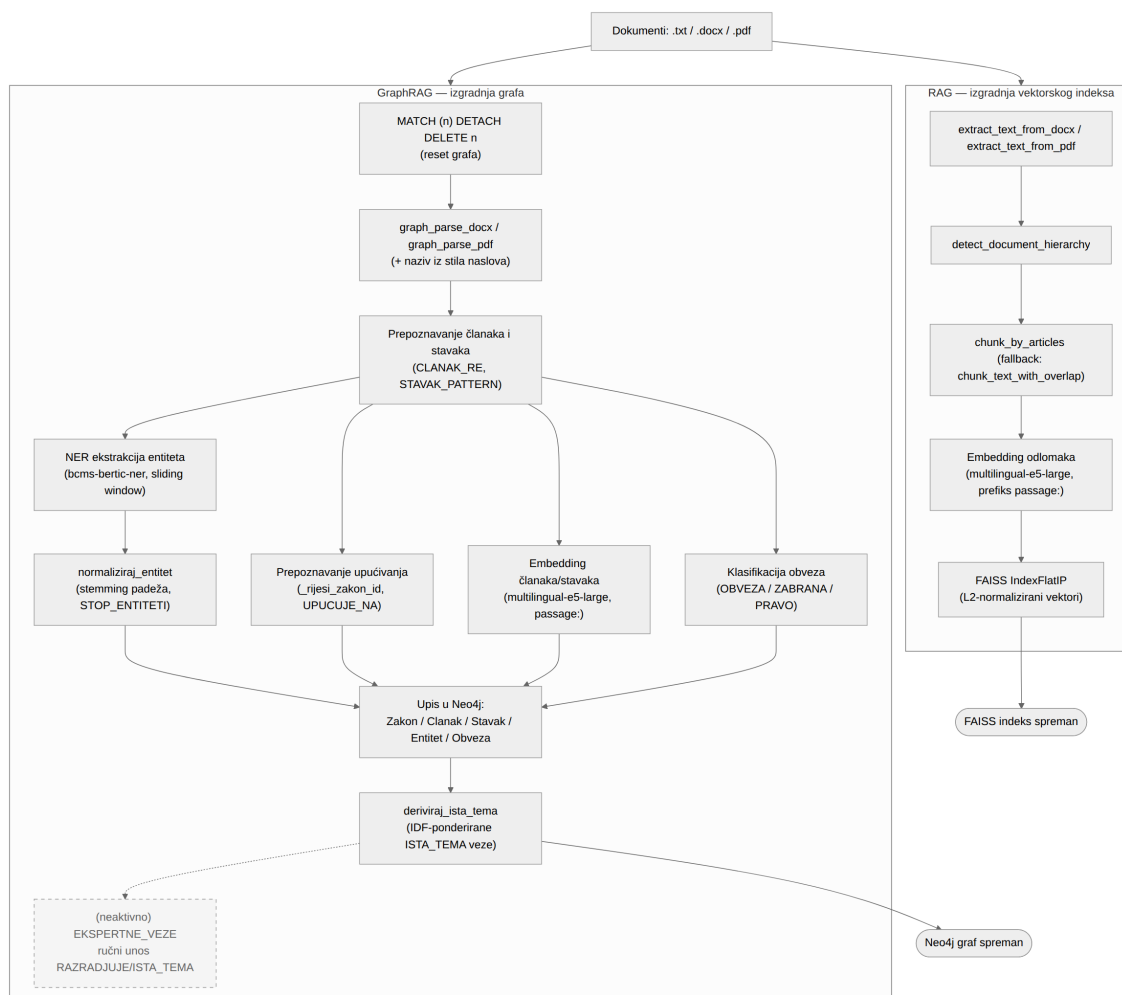
4.6.3. Automatsko izvođenje tematskih veza

Osim eksplicitnih upućivanja unutar teksta, graf sadrži i automatski izvedene relacije `ISTA_TEMA`, koje povezuju članke *različitih* propisa što dijele tematski značajne entitete. Važnost dijeljenog entiteta mjeri se IDF-om (*inverse document frequency*) na razini članaka: entiteti koji se pojavljuju u mnogo članaka doprinose manje, dok rjeđi, specifičniji entiteti nose veću težinu. Veza `ISTA_TEMA` uspostavlja se samo između međudokumentnih parova članaka čiji zbroj IDF-težina dijeljenih entiteta prelazi zadani prag (`MIN_TEZINA = 1, 5`) i koji dijele barem dva entiteta, čime se sprječava stvaranje velikog broja slabo informativnih veza.

Mada je izvan obradnog tijeka, ali na ovom mjestu će biti spomenuta i mogućnost ručnog unosa ekspertnih veza. U Colab bilježnici je pripremljena i funkcionalnost ručnog unošenja dodatnih ekspertnih veza između članaka, što omogućuje proširenje grafa u slučajevima kada automatski postupci ne prepoznaju stvarnu pravnu poveznicu. Ova mogućnost nije aktivirana u evaluacijskoj izvedbi, ali je vrijedna spomena jer pokazuje mogući razvoj rada u budućnosti.

Slika 7 sažeto prikazuje cjelokupan postupak pripreme podataka opisan u prethodnim

potpoglavljima, usporedno za obje verzije koje se oslanjaju na predobrađeni korpus: izgradnju FAISS vektorskog indeksa za RAG verziju (desno) te izgradnju grafa u bazi Neo4j za GraphRAG verziju (lijevo), uključujući prepoznavanje entiteta, upućivanja, obveza i automatsko izvođenje veza ISTA_TEMA.



Slika 7: Procesni lanac pripreme podataka za RAG (izgradnja vektorskog indeksa) i GraphRAG (izgradnja grafa) verzije chatbota.

4.6.4. Postupak dohvata konteksta

Funkcijom `graph_retrieve_context` implementiran je dohvat konteksta za dano pitanje i odvija se u tri faze:

1. **Vektorska pretraga početnih čvorova** - upit se vektorizira (uz prefiks "query:") te se putem Neo4j vektorskih indeksa istovremeno pretražuju i članci i stavci (`GRAPH_VECTOR_POOL = 30`: kandidati svake vrste); pogodak na razini stavka mapira se natrag na njegov roditeljski članak, čime se uska, specifična pitanja i dalje povezuju s odgovarajućim širim kontekstom članka. Rezultati se potom ograničavaju najvećim brojem početnih članaka po propisu (`GRAPH_PER_DOC_CAP =`

3), kako bi pretraga uvijek pokrivala više relevantnih propisa umjesto da jedan dokument bude dominantan;

2. **Obilazak grafa iz početnih čvorova** - iz svakog početnog članka graf se obilazi jedan korak po tipiziranim relacijama (`UPUCUJE_NA`, `RAZRADJUJE`, `ISTA_TEMA`) te posredno, preko zajedničkog entiteta (`SPOMINJE`), ali samo prema člancima iz *drugog* propisa. Ocjena relevantnosti susjednog članka izračunava se kao umnožak ocjene početnog članka i težine tipa veze kojom je dosegnut (`GRAPH_REL_WEIGHTS`: 1,00 za `RAZRADJUJE`, 0,95 za `ISTA_TEMA`, 0,85 za `UPUCUJE_NA`, 0,70 za posredno dosegnute preko entiteta), čime se izravne pravne poveznice vrednuju više od posredne tematske srodnosti;
3. **Deduplikacija i konačan odabir** - kandidati (početni i susjedni članci) objedinjuju se prema jedinstvenom identifikatoru članka (umjesto usporedbe teksta kao u RAG verziji), rangiraju prema konačnoj ocjeni te se odabire najviše `top_k` članaka, pri čemu `top_k` ovisi o kategoriji pitanja (`GRAPH_TOP_K`, jednako strukturirano kao `RAG_TOP_K` u prethodnoj verziji). Tekst svakog odabranog članka pritom se ograničava na najviše `GRAPH_MAX_CHARS_PO_CLANKU = 4000` znakova, čime se sprječava da jedan neuobičajeno dugačak članak sam po sebi potroši velik dio dopuštene duljine ulaza modela na štetu preostalih dohvaćenih članaka.

Svaki odabrani članak u konačnom kontekstu označen je nazivom propisa, brojem članka, izračunatom ocjenom te, ako je dosegnut obilaskom grafa, i tipom veze kojom je pronađen (npr. `veza=UPUCUJE_NA`), radi transparentnosti izvora informacije u odgovoru.

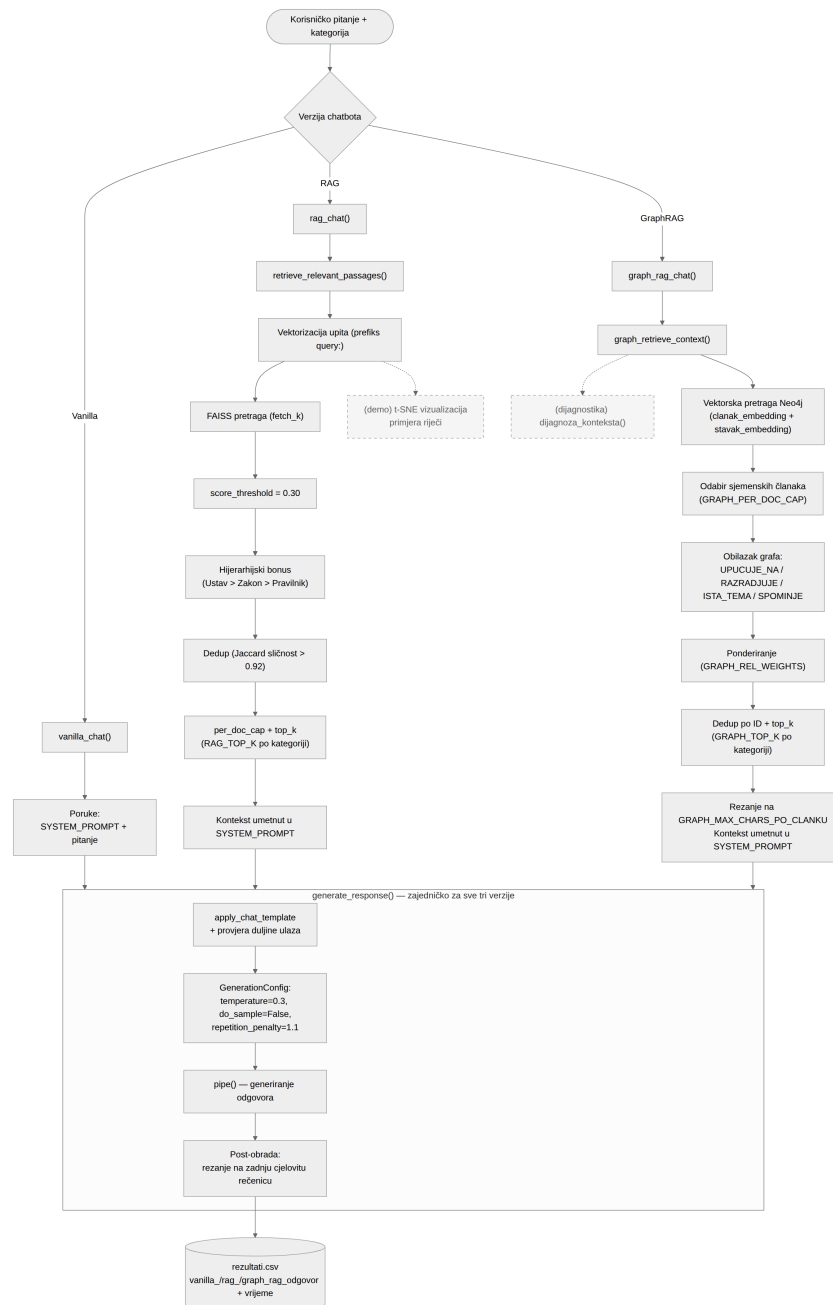
4.6.5. Integracija u generiranje odgovora

Funkcija `graph_rag_chat` poziva `graph_retrieve_context` za dano pitanje i njegovu kategoriju, dobiveni kontekst nadovezuje na `SYSTEM_PROMPT` (odvojen nizom “--” između pojedinih članaka) te poziva zajedničku funkciju `generate_response` uz istu determinističku konfiguraciju (`temperature=0.3`, `do_sample=False`) kao i prethodne dvije verzije. Za razliku od RAG verzije, ovdje funkcija `generate_response` nema poseban parametar za uključivanje dohvata (`use_rag`), jer se dohvat obavlja izvan nje, prije poziva, a umetnuti se kontekst pri eventualnom prekoračenju dopuštene duljine ulaza skraćuje uklanjanjem posljednjih blokova razdvojenih znakovnim nizom “--”, analogno postupku u RAG verziji.

4.6.6. Alati za dijagnostiku grafa

Radi provjere kvalitete izgrađenog grafa i ponašanja pretraživača prije pokretanja pune evaluacije, bilježnica sadrži i pomoćne dijagnostičke funkcije koje se ne koriste u samom generiranju odgovora. Funkcija `dijagnoza_konteksta` za zadano pitanje ispisuje broj dohvaćenih članaka, koliko je od njih pronađeno obilaskom grafa (a ne izravnom vektorskom pretragom) te koliko je različitih propisa pokriveno dohvaćenim kontekstom, što omogućuje brzu ručnu provjeru je li dohvat za pojedino, osobito međudokumentno pitanje smislen. Nakon izgradnje grafa

ispisuju se i zbirne statistike o strukturi grafa (broj propisa, članaka i stavaka, broj članaka po dokumentu, raspodjela broja veza po tipu te broj tzv. mostovnih entiteta koji izravno povezuju članke dvaju različitih propisa), koje služe kao provjera ispravnosti ingestije, ali same po sebi ne utječu na generirane odgovore niti na rezultate evaluacije.



Slika 8: Obradni tijek korisničkog upita za Vanilla, RAG i GraphRAG verziju chatbota, do zajedničkog koraka generiranja odgovora.

4.6.7. Izlazni podaci

Rezultati GraphRAG verzije nadopunjuju isti `rezultati.csv` koji su prethodno popunile Vanilla i RAG verzija, dodavanjem dvaju stupaca:

- `graph_rag_odgovor` - odgovor generiran uz kontekst dobiven obilaskom grafa;
- `graph_rag_vrijeme_sekunde` - vrijeme (u sekundama) potrebno za generiranje odgovora, uključujući vektorsku pretragu i obilazak grafa.

Nakon izvođenja sve tri bilježnice, datoteka `rezultati.csv` sadrži, za svako pitanje iz zajedničkog skupa za evaluaciju, odgovore i vremena generiranja sve tri verzije (`vanilla`, `rag`, `graph_rag`) uz očekivani odgovor, što čini temelj za poglavlje o evaluaciji.

Slika 8 objedinjuje cjelokupan tijek obrade jednog korisničkog upita kroz sve tri opisane verzije chatbota, od grananja prema verziji, preko postupaka dohvata specifičnih za RAG i GraphRAG (ili njihova izostanka u Vanilla verziji), do zajedničke funkcije `generate_response` i konačnog zapisa rezultata u `rezultati.csv`.

5. Evaluacija chatbotova

Iskustvo s razvojem ovog sustava pokazalo je da točnost odgovora nije jedino što je važno, chatbot može citirati ispravan članak zakona, a opet biti nekoristan ako odgovor nije razumljiv ili primjenjiv. U kontekstu domene vojnog zakonodavstva, gdje pogreška u citiranju propisa ili pogrešna pravna interpretacija mogu imati ozbiljne posljedice, odabir primjerenih evaluacijskih metoda postaje od osobite važnosti. U ovom poglavlju opisuju se operativni aspekti inferencije i latencije, automatske metrike kvalitete teksta, pristup ljudskoj evaluaciji uz mjerenje međuoocjenjivačke pouzdanosti te pristup gdje jedan jezični model ocjenjuje izlaz drugog, tzv. LLM-as-a-Judge (*LLM-as-a-Judge*).

5.1. Operativni aspekti: inferenca i latencija

Uz kvalitetu generiranih odgovora, ocijenjenu automatskim metrikama i ljudskom evaluacijom u poglavljima koja slijede, chatbot namijenjen stvarnoj upotrebi mora zadovoljiti i operativne zahtjeve vezane uz način posluživanja i brzinu odgovora. U nastavku se stoga najprije razlikuju online i batch način rada modela (potpoglavlje 5.1.1), a potom se izvještava o izmjerenoj latenciji triju uspoređenih konfiguracija sustava (potpoglavlje 5.1.2).

5.1.1. Online i batch inferenca

Ovisno o tome treba li odgovor odmah ili ne, razlikuju se dva pristupa pokretanju modela:

- **Online inferenca** isporučuje odgovore u stvarnom vremenu kao odgovor na svaki korisnički zahtjev. Primjerena je za interaktivne sustave u kojima korisnik očekuje neposredan odgovor, kao što je slučaj s chatbotovima koji odgovaraju na upite iz područja vojnog zakonodavstva.
- **Batch inferenca** obrađuje veći broj zahtjeva izvan realnog vremena, primjerice kao periodički noćni posao ili masovna obrada dokumenata. Pogodna je za scenarije u kojima latencija nije kritična, ali je propusnost (engl. *throughput*) važan čimbenik.

U ovom istraživanju koristi se isključivo *online* inferenca, budući da se evaluira interaktivni chatbot namijenjen odgovaranju na pitanja u stvarnom vremenu.

5.1.2. Latencija

Prag od 200 ms koji korisnici percipiraju kao “trenutačno” nije dostižan za LLM sustave koji generiraju dulje odgovore, ali nije ni relevantan cilj. Važnija je granica oko jedne sekunde: iznad nje korisnik svjesno čeka i gubi osjećaj dijaloga [22]. RAG konfiguracija u ovom sustavu dodaje korak dohвата prije generiranja, što povećava ukupnu latenciju, ali taj se trošak donekle kompenzira kraćim i preciznijim odgovorima koji ne zahtijevaju toliko generiranih tokena.

U sklopu evaluacije mjerena je ukupna latencija odgovora (u sekundama) za sve tri ispitivane konfiguracije sustava. Latencija RAG sustava očekivano je veća zbog dodatnog koraka dohvata i rekontekstualizacije, no taj se trošak kompenzira potencijalno višom kvalitetom odgovora.

5.2. Automatske metrike

Ručna evaluacija 31 pitanja s tri evaluatora daje pouzdane, ali spore i skupe rezultate. Automatske metrike omogućuju da se isti posao napravi u sekundama, uz cijenu: ne znaju razlikovati pravno točan od pravno pogrešnog odgovora, čak i kad su formulacije gotovo identične. Iz tog razloga u ovom istraživanju automatske metrike služe kao pomoćne, ali ne i presudne metode prosudbe.

5.2.1. Leksičke metrike

Leksičke metrike mjere sličnost na razini podudaranja riječi i fraza između generiranog i referentnog odgovora.

5.2.1.1. ROUGE-L

Recall-Oriented Understudy for Gisting Evaluation (ROUGE) skup je metrika izvorno razvijenih za evaluaciju automatskog sažimanja teksta [23]. Varijanta **ROUGE-L** temelji se na najduljem zajedničkom podnizu (engl. *Longest Common Subsequence* – LCS) između generiranog i referentnog odgovora. LCS ne zahtijeva da se podudarni tokeni nalaze u neprekinutom slijedu, što ga čini otpornijim na varijacije u redoslijedu riječi. ROUGE-L vraća F1-mjeru kao harmonijsku sredinu preciznosti i opoziva.

U pravnoj domeni, parafraziranje nije uvijek vrlina. Ako chatbot umjesto “članka 45. stavka 2.” kaže “relevantnog dijela zakona”, semantički je blizu, ali korisnik koji želi citirati propis dobio je beskoristan odgovor. ROUGE-L kažnjava upravo takva odstupanja jer traži leksičko podudaranje, što ga čini prikladnijim za ovu domenu nego što bi bio u, recimo, evaluaciji kreativnog pisanja. Izostanak ili pogrešno navođenje referentnog članka kaznit će se niskim ROUGE-L rezultatom, čak i ako je ostatak odgovora semantički koherentan.

Napomena o metrici BLEU. Metrika *Bilingual Evaluation Understudy* (BLEU) [24] mjeri preciznost n-gramskog podudaranja i izvorno je razvijena za strojno prevođenje. U ovom istraživanju BLEU nije primijenjen kao primarna metrika iz dva razloga: prestrogo kažnjava legitimne parafrazirane odgovore koji su sadržajno točni, ne pruža informaciju o opozivu, što je u evaluaciji chatbota jednako važno kao i preciznost. ROUGE-L pokriva sličan leksički prostor uz bolja statistička svojstva za ovaj zadatak.

5.2.2. Semantičke metrike

Semantičke metrike procjenjuju sličnost na razini značenja, a ne samo podudaranja oblika riječi, što ih čini otpornijima na sinonime i parafraziranje.

5.2.2.1. BERTScore

BERTScore [25] semantička je metrika koja umjesto n-gramskog podudaranja koristi kontekstualne reprezentacije tokena dobivene predtreniranim BERT modelima. Za svaki token u generiranom odgovoru pronalazi se najbliži token u referentnom odgovoru (i obratno) prema kosinusnoj sličnosti njihovih vektora, a konačna ocjena agregira se u preciznost, opoziv i F1-mjeru.

U ovom istraživanju korišten je model `bert-base-multilingual-cased`, koji podržava hrvatski jezik i prikladan je za višezjezične domene [25]. BERTScore je posebno koristan za hvatanje semantičke ekvivalentnosti kada chatbot koristi drugačiju formulaciju od ekspertovog referentnog odgovora, ali prenosi jednako značenje.

5.2.2.2. Kosinusna sličnost (Cosine Similarity)

Kosinusna sličnost mjeri kosinus kuta između dvaju vektora koji predstavljaju cijele odgovore (a ne pojedinačne tokene, kao BERTScore). Vrijednost 1 označava identičnu orijentaciju vektora (maksimalnu sličnost), vrijednost 0 označava da vektori nemaju ničeg zajedničkog u semantičkom prostoru.

Za pretvaranje odgovora u vektore korišten je model `intfloat/multilingual-e5-large` [26], koji je treniran na višezjezičnom korpusu i pokazuje visoku učinkovitost na zadacima semantičkog pretraživanja i sličnosti u slavenskim jezicima. Dok BERTScore uspoređuje odgovore na razini tokena, kosinusna sličnost pruža globalnu procjenu tematske i kontekstualne podudarnosti cijelih odgovora.

5.2.3. Implementacija izračuna metrika

Automatska evaluacija provodi se neovisno o generiranju odgovora, u zasebnoj bilježnici, nad izlaznom datotekom `rezultati.csv` koju su redom nadopunile bilježnice za generiranje odgovora triju verzija chatbota. Kao ulaz koriste se dvije datoteke: `rezultati.csv`, koji za svako pitanje sadrži kategoriju, sam upit, očekivani odgovor te odgovore i vremena generiranja sve tri verzije, i izvorna datoteka `pitanja_odgovori.json` radi provjere potpunosti skupa. Prisutnost obveznih stupaca (`rbr`, `kategorija`, `pitanje`, `ocekivani_odgovor`, `vanilla`, `rag`) provjerava se eksplicitno, dok je stupac `graph_rag` tretiran kao opcionalan, ako nije prisutan, evaluacija se provodi samo za Vanilla i RAG pristup, čime bilježnica ostaje funkcionalna i u međukoraku, prije nego što su generirani odgovori sve tri verzije.

Tri metrike opisane u potpoglavljima 5.2.1 i 5.2.2 računaju se sljedećim funkcijama:

Isječak koda 8: Funkcije za izračun metrika evaluacije

```
1 def compute_rouge_l(reference, hypothesis):
2     scorer = rouge_scorer.RougeScorer(['rougeL'], use_stemmer=False)
3     scores = scorer.score(reference, hypothesis)
4     return scores['rougeL'].fmeasure
5
6 def compute_bertscore(references, hypotheses):
7     P, R, F1 = bert_score_fn(
8         hypotheses, references,
9         model_type='bert-base-multilingual-cased',
10        num_layers=9,
11    )
12    return F1.tolist()
13
14 def compute_cosine_similarity(reference, hypothesis, model):
15     emb_ref = model.encode([reference], convert_to_numpy=True)
16     emb_hyp = model.encode([hypothesis], convert_to_numpy=True)
17     return float(sklearn_cosine(emb_ref, emb_hyp)[0][0])
```

ROUGE-L i kosinusna sličnost izračunavaju se u petlji, redak po redak, zasebno za svaku od prisutnih verzija (Vanilla, RAG, GraphRAG), uspoređujući generirani odgovor s očekivanim odgovorom iz istog retka. BERTScore se, radi učinkovitosti, ne računa pojedinačno po retku, već paketno nad cijelim popisom referenci i hipoteza za svaku verziju odjednom. Dobivene vrijednosti metrika upisuju se kao novi stupci u DataFrame (npr. `vanilla_rouge_l`, `rag_bertscore`, `graph_cosine`), a prošireni skup podataka potom se sprema u novu datoteku, `rezultati_evaluacija.csv`, kako bi izvorni `rezultati.csv` s odgovorima ostao nepromijenjen. Ova datoteka predstavlja ulazni skup podataka za agregaciju, vizualizaciju i statističku analizu rezultata opisane u poglavlju 6.

5.2.4. Ograničenja automatskih metrika u pravnoj domeni

Unatoč prednostima skalabilnosti i reproducibilnosti, sve tri uporabljene automatske metrike dijele suštinsko ograničenje: pretpostavljaju visoku kvalitetu zlatnog standarda i ne detektiraju faktičke pravne pogreške.

Konkretno, chatbot može generirati odgovor koji je leksički i semantički blizak referentnom, ali koji pogrešno primjenjuje pravnu normu, primjerice, navodi ispravan članak zakona, ali s netočnim stavkom ili pogrešnom interpretacijom. Takav odgovor dobit će visoke automatske ocjene, ali je s pravnog stanovišta netočan.

Iz tog razloga automatske metrike u ovom istraživanju imaju komplementarnu, a ne primarnu ulogu, služe za brzo filtriranje i rangiranje odgovora, dok se konačna prosudba o pravnoj točnosti prepušta ljudskim evaluatorima i LLM ocjenjivaču s domenski specifičnim uputama.

5.3. Ljudska evaluacija

Kako bi se nadišla ograničenja automatskih metrika opisana u potpoglavlju 5.2.4, provedena je i ljudska evaluacija odgovora triju verzija chatbota. U nastavku se opisuju sudionici i dimenzije ocjenjivanja (potpoglavlje 5.3.1) te postupak mjerenja i osiguranja pouzdanosti dobivenih ocjena (potpoglavlje 5.3.3).

5.3.1. Evaluatori i dimenzije ocjenjivanja

U evaluaciji je sudjelovalo 14 evaluatora s komplementarnim domenskim znanjem, jedan je pravni ekspert, a ostali su vojni stručnjaci s iskustvom u primjeni vojnog zakonodavstva. Svaki evaluator radio je neovisno, bez uvida u ocjene ostalih, čime se minimizira efekt sidrenja (*anchoring bias*).

Za svako od $N = 31$ pitanja evaluatori su ocjenjivali odgovore chatbota na Likertovoj skali od 1 do 3 prema sljedećim dimenzijama:

Tablica 1: Dimenzije ljudske evaluacije i primarni evaluatori

Dimenzija	Opis	Primarni evaluator
Točnost	Ispravnost citiranja zakona, članaka i propisa	Svi evaluatori
Korisnost	Primjenjivost odgovora u stvarnoj vojnoj situaciji	Svi evaluatori
Halucinacije	Stupanj fabriciranja odgovora	Svi evaluatori

Skala ocjenjivanja definirana je kako slijedi:

1. **FAKTIČNA TOČNOST** — je li tvrdnja točna. Tvrdnja može biti točna, ali ne mora nužno biti odgovor. Ovom ocjenom se ocjenjuje samo točnost, a sljedećom ocjenom se nagrađuje/kažnjava relevantnost odgovora. 3 => Točno; jezgra i ključni detalji su točni. 2 => Djelomično točno; jezgra dobra, ali dio detalja krivi (npr. jedan pogrešan podatak). 1 => Pretežno netočno; kriva jezgra, krivi brojevi/rokovi ili krivo tijelo.

2. **KORISNOST (RELEVANTNOST)** — odgovara li na postavljeno pitanje. Ocjenjuje se neovisno o točnosti, odgovor može biti točan a nekoristan ako priča o nečem drugom. 3 => Izravno i potpuno odgovara na pitanje. 2 => Dotiče temu, ali nepotpuno, razvučeno ili djelomično promašuje fokus. 1 => Promašuje pitanje, odgovara na nešto drugo ili je nečitljiv.

3. **HALUCINACIJE** — prisutnost izmišljenog sadržaja (izmišljeni članci, brojevi NN-a, URL-ovi, institucije, citati kojih nema u izvoru). 3 => Bez izmišljotina; sve provjerljivo u izvoru. 2 => Manja izmišljotina; točna jezgra, ali jedan izmišljen detalj (npr. lažni broj članka ili URL uz inače točan sadržaj). 1 => Halucinira; sadrži izmišljene činjenice, nepostojeće članke/zakone ili lažne izvore.

5.3.2. Google Obrasci

5.3.3. Međuocjenjivačka pouzdanost

Pouzdanost ocjenjivanja kvantificirana je mjerenjem slaganja između svih vojnih evaluatora, koji su ocjenjivali iste odgovore neovisno. Primijenjene su dvije mjere:

- **Fleissov Kappa (κ)** [27] – generalizacija Cohenovog Kappa koeficijenta [28] na više od dva ocjenjivača, koja mjeri slaganje uz korekciju za slučajno slaganje. Vrijednosti se interpretiraju prema konvenciji: $\kappa < 0,20$ – slabo, $0,21-0,40$ – prihvatljivo, $0,41-0,60$ – umjereno, $0,61-0,80$ – dobro, $> 0,80$ – izvrsno [29].
- **Krippendorffov Alpha (α)** [30] – generalizacija koja je primjenjiva na ordinalne skale i više od dva ocjenjivača, što je relevantno za buduća proširenja evaluacijskog skupa.

Prilikom pregleda rezultata uočeno je kako dio ocjenjivača vrlo blago ocjenjuje sve odgovore, što se očituje u vrlo niskoj varijanci njihovih ocjena. Takvi ocjenjivači nisu uključeni u izračun pouzdanosti jer njihova prisutnost iskrivljuje rezultate i ne odražava stvarnu međuocjenjivačku pouzdanost. Ta se pojava naziva pristranost popustljivosti (engl. *leniency bias*) i u literaturi je prepoznata kao čest problem u ljudskoj evaluaciji [31].

Pri interpretaciji rezultata evaluacije potrebno je uzeti u obzir navedenu blagosti u ocjenjivanju, odnosno sklonost pojedinih ocjenjivača da dodjeljuju više ocjene nego što odgovara stvarnoj kvaliteti ocjenjivanih odgovora. Tijekom evaluacije uočeno je da je dio ocjenjivača gotovo svim odgovorima, neovisno o tome je li ih generirala osnovna inačica jezičnog modela, RAG ili GraphRAG arhitektura, dodijelio najviše moguće ocjene. Takvo ponašanje ne odražava stvarnu izvedbu ocjenjivanih sustava, nego značajku samog ocjenjivača, te unosi šum u mjerenje [31]. Ocjenjivač koji svim inačicama dodjeljuje najvišu ocjenu ne razlikuje uspoređivane arhitekture, pa njegove ocjene ne doprinose evaluaciji, a mogu povisiti prosječne rezultate i prikriti stvarne razlike u kvaliteti odgovora.

Budući da ocjene takvih ocjenjivača nisu razlikovale uspoređivane inačice sustava, one su izuzete iz konačnog izračuna rezultata. Kao kriterij za izuzimanje korišten je udio istovjetnih (najviših) ocjena po ocjenjivaču, pri čemu su izuzeti oni ocjenjivači kod kojih je taj udio prelazio 85 %. Na temelju tog kriterija, primijenjenog na svih četrnaest ocjenjivača koji su sudjelovali u evaluaciji, izuzeta su tri ocjenjivača, tako da je u konačnu analizu uključeno jedanaest ocjenjivača. Time se spriječilo da neinformativne, sustavno visoke ocjene povise prosječne rezultate i prikriju stvarne razlike među arhitekturama [31]. Nakon uklanjanja tih ocjenjivača ponovno je izračunata mjera slaganja među ocjenjivačima.

5.4. LLM-as-a-Judge

Uz automatske metrike i ljudsku evaluaciju opisane u prethodnim poglavljima, kao treći, komplementaran pristup evaluaciji koristi se *LLM-as-a-Judge* metoda, u kojoj veliki jezični mo-

del preuzima ulogu ocjenjivača odgovora drugog modela. U nastavku se opisuju motivacija za odabir ovog pristupa (potpoglavlje 5.4.1), implementacija i dizajn evaluacijskog prompta (potpoglavlje 5.4.2) te poznata ograničenja metode (potpoglavlje 5.4.3).

5.4.1. Pristup i motivacija

Koncept *LLM-as-a-Judge* [32] rješava problem evaluacije upošljavanjem velikog jezičnog modela kao neovisnog suca koji evaluira izlaz drugog modela prema unaprijed zadanim kriterijima. LLM ocjenjivač može procjenjivati i kvalitativne aspekte poput koherentnosti argumentacije, primjerenosti tona i praćenja uputa.

LLM ocjenjivač uveden je iz tri razloga. Može obraditi puno više primjera nego što bi ikad stigli ručno, daje konzistentne ocjene jer uvijek koristi isti prompt i bolje od leksičkih metrika prepoznaje parafrazirane ali sadržajno točne odgovore.

5.4.2. Implementacija i dizajn prompta

Kao LLM ocjenjivač korišteni su modeli `claude-sonnet-4` (Anthropic) i `3.6 flash` (Gemini). Odabir je motiviran visokom razinom praćenja uputa (*instruction following*) i strukturiranog izlaza u JSON formatu, što je ključno za automatsku obradu rezultata.

Sistemska prompt dizajniran je s eksplicitnim uputama za domenu vojnog zakonodavstva i zahtjevom za ocjenjivanje triju istih dimenzija kao i kod ljudskih evaluatora (Tablica 1). Prompt uključuje:

- definiciju uloge (*role prompting*): evaluator je ekspert za vojno zakonodavstvo Republike Hrvatske,
- eksplicitan format izlaza: JSON objekt s numeričkim ocjenama 1–3 za svaku dimenziju i kratkim obrazloženjem na hrvatskom jeziku,
- referencu na zlatni standard: za svaki upit LLM-u se predaje i pitanje te referentni odgovor eksperta i odgovor chatbota,
- zabranu izlaza izvan JSON formata kako bi se izbjegla potreba za složenim parsiranjem teksta.

5.4.3. Poznata ograničenja

Primjena LLM-as-a-Judge pristupa donosi nekoliko svojstvenih ograničenja na koja upozorava literatura [32], [33]:

- **Pristranost pozicije** (*position bias*): LLM ocjenjivači skloni su favorizirati odgovor koji se pojavljuje prvi u kontekstu. U ovom istraživanju odgovori se uvijek predaju u fiksnom redoslijedu (Vanilla, RAG pa GraphRAG), što treba uzeti u obzir pri interpretaciji rezultata.

- **Pristranost samopohvale** (*self-enhancement bias*): modeli mogu favorizirati odgovore koji stilski nalikuju njihovim vlastitim odgovorima.
- **Ograničeno domensko znanje**: unatoč visokokvalitetnom modelu, LLM ocjenjivač može propustiti suptilne pravne pogreške koje bi iskusan pravni ekspert odmah prepoznao.
- **Neponovljivost**: stohastičnost jezičnih modela može uzrokovati marginalne razlike u ocjenama pri ponovljenom pokretanju s istim ulazom.

Iz navedenih razloga LLM-as-a-Judge tretira se kao *dopunska*, a ne zamjenska metoda u odnosu na ljudsku evaluaciju.

5.5. Obrazloženje odabira kombinacije metoda

U Tablici 2 prikazana je usporedba svih primijenjenih evaluacijskih metoda prema ključnim karakteristikama relevantnim za domenu vojnog zakonodavstva.

Tablica 2: Usporedba evaluacijskih metoda

Metoda	Skalabilnost	Semantičko razumijevanje	Detektira fakt. pogreške	Pravna specifičnost
ROUGE-L	visoka	nisko	ne	djelomično
BERTScore	visoka	visoko	ne	nisko
Cosine sličnost	visoka	visoko	ne	nisko
LLM-as-a-Judge	srednja	visoko	djelomično	visoko
Ljudska evaluacija	niska	visoko	da	visoko

Nijedna od ovih metoda sama za sebe nije dovoljna, svaka hvata nešto što druge propuštaju, pa ih je jedino smisleno koristiti zajedno. Automatske metrike (ROUGE-L, BERTScore, kosinusna sličnost) osiguravaju skalabilnu i reproducibilnu baznu liniju. LLM-as-a-Judge pruža kvalitativnu procjenu bez angažmana domenskih stručnjaka za svaki pojedinačni upit. Ljudska evaluacija, kod koje se mjeri međuocjenjivačko slaganje, zlatni je standard čija je korelacija s automatskim metodama sama po sebi vrijedan doprinos istraživanju, jer ukazuje na primjenjivost automatskih metrika u ovoj specifičnoj domeni.

6. Statistička analiza rezultata evaluacije

U ovom poglavlju prikazuju se rezultati evaluacije triju konfiguracija sustava (Vanilla, RAG i GraphRAG) prema dimenzijama definiranim u poglavlju 5. Cilj analize je utvrditi postoje li statistički značajne razlike između konfiguracija te ispitati u kojoj mjeri automatske metrike i LLM ocjenjivači koreliraju s ljudskom procjenom kvalitete odgovora.

Evaluaciju je provelo 14 eksperata koji su svaki odgovor ocjenjivali prema tri dimenzije na ordinalnoj skali od 1 do 3: točnost (1 = netočan, 3 = točan), korisnost (1 = nekoristan, 3 = koristan) i halucinacije (1 = ozbiljne halucinacije, 3 = bez halucinacija). Kao što je opisano u potpoglavlju 5.3.3, troje je ocjenjivača naknadno izuzeto zbog sustavno blagog ocjenjivanja, pa sve statistike u ovom poglavlju koje se oslanjaju na ljudske ocjene koriste medijan ocjena preostalih jedanaest ocjenjivača. Budući da se radi o ordinalnoj skali s tri razine, relativno malom uzorku ($N = 31$ pitanja) i o dizajnu ponovljenih mjerenja (svih 31 pitanje ocjenjuje se pod sve tri konfiguracije), primijenjene su isključivo neparametarske metode za zavisne (uparene) uzorke. Za usporedbu triju konfiguracija korišten je Friedmanov test kao objedinjeni test za ponovljena mjerenja, uz post-hoc upareni Wilcoxonov test predznačenih rangova s Bonferroni korekcijom, isti pristup koji je u potpoglavlju 6.6.2 dosljedno primijenjen i za usporedbu LLM ocjenjivača Claude i Gemini. Povezanost između automatskih metrika i ljudskih ocjena kvantificirana je Spearmanovom korelacijom (ρ).

6.1. Pregled prikupljenih podataka

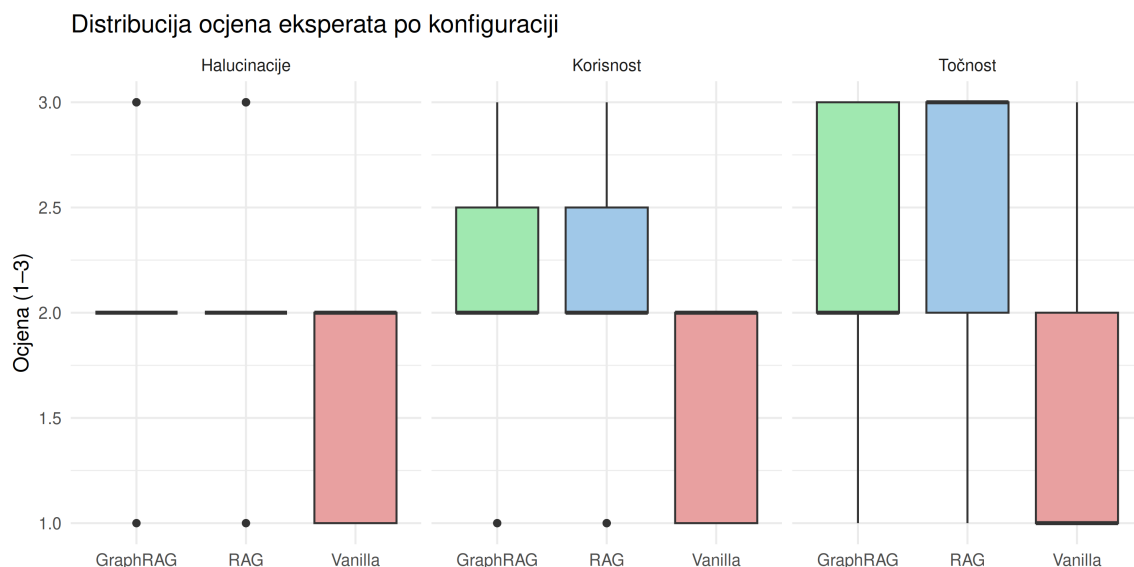
Tablica 3: Deskriptivna statistika ljudskih ocjena po konfiguraciji (skala 1–3, medijan jedanaest ocjenjivača po pitanju)

Konfiguracija	Dimenzija	$\bar{x}$	Med.	s	Min	Max
Vanilla	Točnost	1,48	1	0,63	1	3
Vanilla	Korisnost	1,58	2	0,50	1	2
Vanilla	Halucinacije	1,61	2	0,50	1	2
RAG	Točnost	2,42	3	0,67	1	3
RAG	Korisnost	2,16	2	0,58	1	3
RAG	Halucinacije	2,16	2	0,52	1	3
GraphRAG	Točnost	2,32	2	0,65	1	3
GraphRAG	Korisnost	2,19	2	0,54	1	3
GraphRAG	Halucinacije	2,10	2	0,47	1	3

Tablica 3 pokazuje da Vanilla konfiguracija ima niže prosjeke na sve tri dimenzije (1,48–1,61) od RAG i GraphRAG konfiguracija (2,10–2,42), dok su prosjeci RAG-a i GraphRAG-a međusobno bliski, s razlikama unutar 0,10 bodova. Standardne devijacije (0,47–0,67) usporedive su kroz sve tri konfiguracije. Je li razlika Vanilla naspram RAG/GraphRAG statistički

značajna, te je li razlika između RAG-a i GraphRAG-a i statistički zanemariva ili tek slučajno mala na promatranom uzorku, ispituje se formalnim testom u nastavku.

Slika 9 prikazuje raspodjelu istih agregiranih ocjena po pitanju, odvojeno za svaku dimenziju, čime se uz sažete pokazatelje iz Tablice 3 vidi i oblik same raspodjele (medijan, raspon, iznimke).



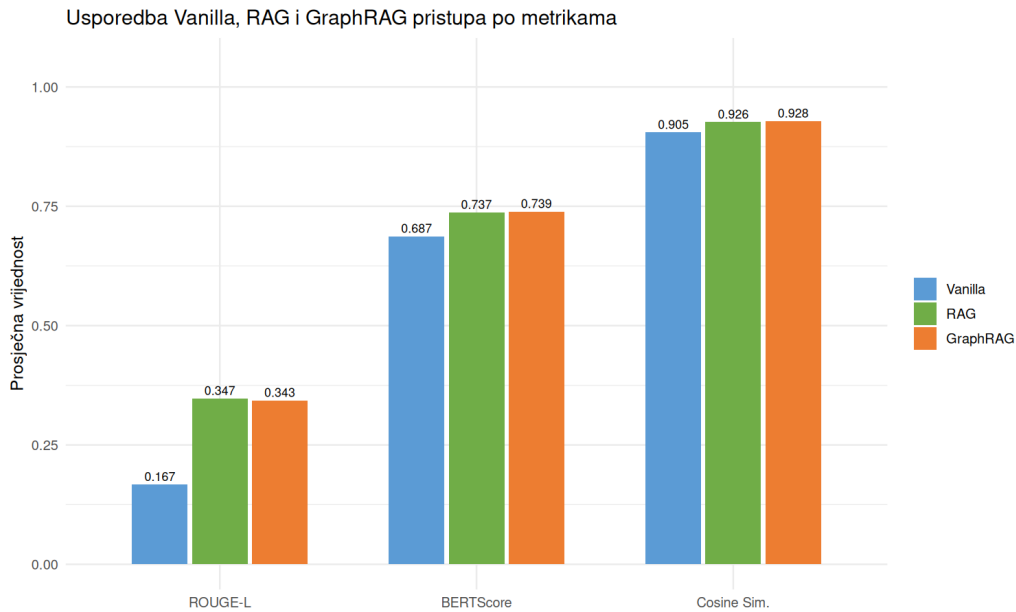
Slika 9: Distribucija ljudskih ocjena po konfiguraciji i dimenziji (medijan jedanaest ocjenjivača po pitanju).

6.2. Agregacija i vizualizacija automatskih metrika

Uz ljudske ocjene prikazane u prethodnom potpoglavlju, jednak pregled provodi se i nad automatskim metrikama izračunatima postupkom opisanom u potpoglavlju 5.2.3. Nad vrijednostima iz datoteke `rezultati_evaluacija.csv` provodi se agregacija na dvije razine: prosjek i standardna devijacija po kategoriji pitanja (jednostavno, kompleksno nad jednim dokumentom, kompleksno nad više dokumenata) te ukupni prosjek preko cijelog skupa. Na temelju ukupnih prosjeka izrađuje se sažetak s razlikom svake metrike pojedine verzije naspram Vanilla pristupa (Δ), čime se izravno kvantificira doprinos dohvata konteksta (RAG, GraphRAG) u odnosu na osnovni model bez dohvata. Dodatno se za svaku verziju broji u koliko od tri metrike ta verzija u prosjeku nadmašuje Vanilla pristup, kao sažeti pokazatelj njezine ukupne prednosti.

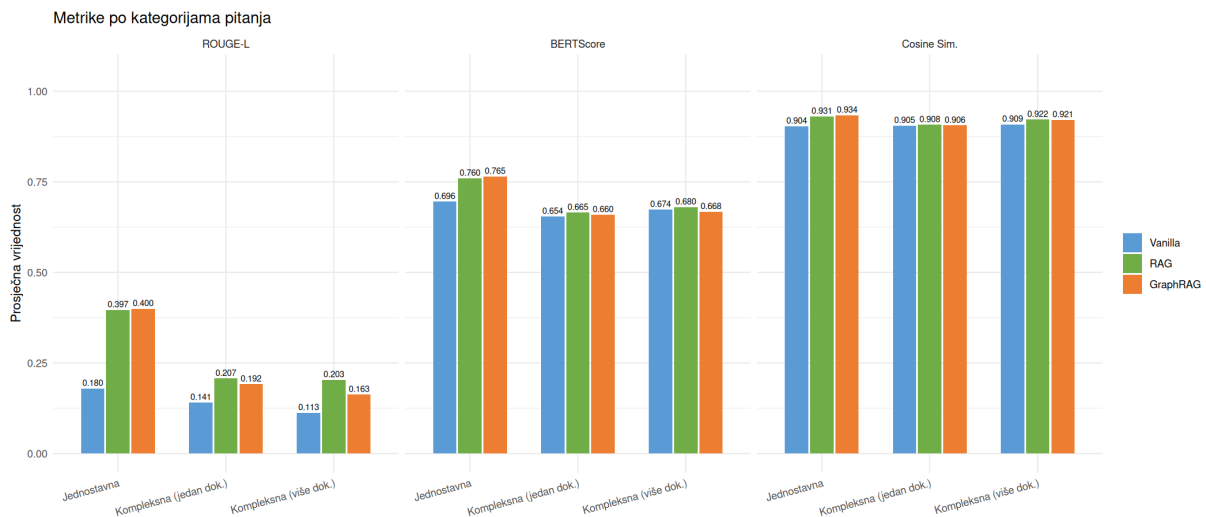
Radi lakšeg tumačenja rezultata izrađena su tri grafička prikaza.

Usporedba pristupa po metrikama (`graf_ustoredba_pristupa.png`) - stupčasti grafikon prosječne vrijednosti svake od triju metrika za svaku verziju chatbota, s brojčano ispisanim vrijednostima iznad stupaca (Slika 10); ovim se grafom vizualno prikazuje relativna prednost RAG i GraphRAG pristupa u odnosu na Vanilla, kao i međusobna usporedba dvaju pristupa s dohvata konteksta.



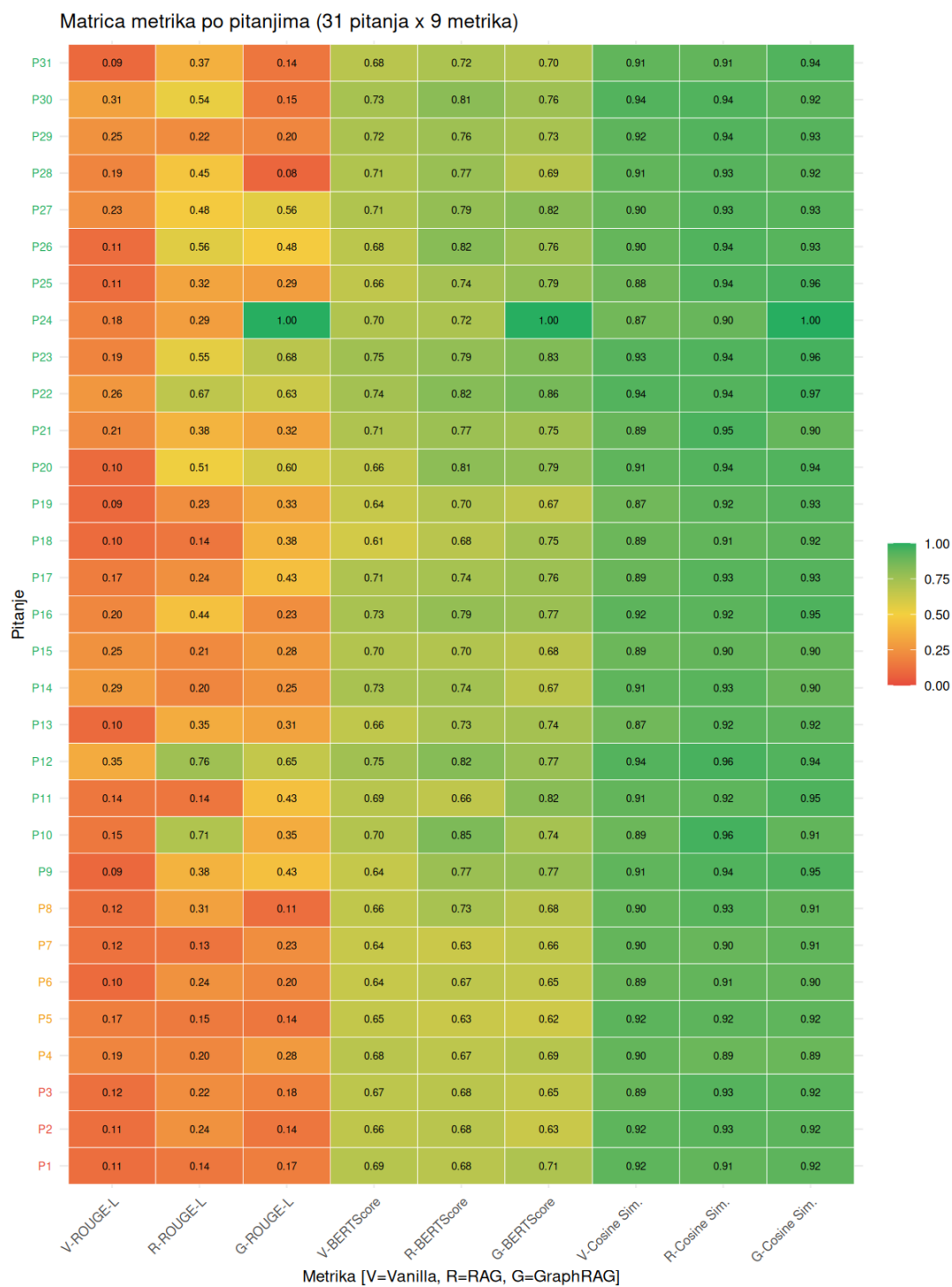
Slika 10: Usporedba triju verzija chatbota po prosječnim vrijednostima triju metrika.

Metrike po kategorijama pitanja (`graf_metrike_po_kategorijama.png`) - tri odvojena podgrafikona (po jedan za svaku metriku), unutar kojih se za svaku kategoriju pitanja uspoređuju sve tri verzije, čime se prikazuje ovisi li relativna prednost pojedinog pristupa o složenosti pitanja (Slika 11).



Slika 11: Usporedba triju verzija chatbota po prosječnim vrijednostima triju metrika, razvrstano po kategorijama pitanja.

Toplinska karta metrika po pitanjima (`graf_heatmap_metrike.png`) - matrica u kojoj redak predstavlja pojedino pitanje, a stupac kombinaciju verzije i metrike, uz bojenje vrijednosti od crvene (niska sličnost) do zelene (visoka sličnost); oznake pitanja na y -osi dodatno su obojene prema kategoriji pitanja, čime se omogućuje uvid u obrasce na razini pojedinačnih, a ne samo agregiranih rezultata (Slika 12).



Slika 12: Toplinska karta metrika po pitanjima, s obojenim oznakama kategorija pitanja na y-osi.

Ovi deskriptivni i vizualni prikazi ukazuju na isti smjer razlike koji se, uz formalno testiranje statističke značajnosti, potvrđuje u sljedećem potpoglavlju.

6.3. Usporedba konfiguracija

Tablica 4: Friedmanov test po dimenzijama ljudske evaluacije

Dimenzija	χ^2	df	p
Točnost	31,01	2	< 0,0001
Korisnost	17,78	2	0,0001
Halucinacije	18,93	2	0,0001

Friedmanov test odbacuje nul-hipotezu o jednakosti triju konfiguracija na sve tri dimenzije ($p \leq 0,0001$), odnosno postoji barem jedan par konfiguracija koji se statistički značajno razlikuje po svakoj od promatranih dimenzija. Ovaj omnibus rezultat sam po sebi ne govori koji su parovi različiti, pa se u nastavku provodi post-hoc uparena usporedba.

Tablica 5: Post-hoc upareni Wilcoxonov test predznačenih rangova s Bonferroni korekcijom (ljudska evaluacija)

Dimenzija	Par	V	p	$p_{\text{Bonf.}}$	r
Točnost	Van. vs. RAG	0,0	< 0,0001	0,0001	1,00
	Van. vs. GraphRAG	7,5	0,0001	0,0003	0,97
	RAG vs. GraphRAG	46,0	0,5919	1,0000	0,41
Korisnost	Van. vs. RAG	7,5	0,0005	0,0015	0,95
	Van. vs. GraphRAG	14,0	0,0012	0,0035	0,92
	RAG vs. GraphRAG	49,0	0,8417	1,0000	0,53
Halucinacije	Van. vs. RAG	7,0	0,0008	0,0025	0,95
	Van. vs. GraphRAG	14,0	0,0031	0,0094	0,90
	RAG vs. GraphRAG	10,0	0,5716	1,0000	0,33

Post-hoc usporedba pokazuje dosljedan obrazac na sve tri dimenzije: Vanilla se statistički značajno razlikuje i od RAG-a i od GraphRAG-a, uz vrlo velike veličine efekta ($r = 0,90$ – $1,00$), dok razlika između RAG-a i GraphRAG-a ni na jednoj dimenziji ne doseže značajnost ($p_{\text{Bonf.}} = 1,0000$ posvuda), s umjerenim veličinama efekta ($r = 0,33$ – $0,53$) koje, uz $N = 31$ uparenih opažanja, ne prelaze prag statističke pouzdanosti.

Tablica 6: Friedmanov test i post-hoc upareni Wilcoxonov test s Bonferroni korekcijom po automatskim metrikama

Metrika	Par	V	p	$p_{\text{Bonf.}}$	r
ROUGE-L	Van. vs. RAG	27,0	$< 0,0001$	$< 0,0001$	0,95
	Van. vs. GraphRAG	48,0	0,0001	0,0003	0,90
	RAG vs. GraphRAG	257,0	0,8677	1,0000	0,48
BERTScore	Van. vs. RAG	29,0	$< 0,0001$	0,0001	0,94
	Van. vs. GraphRAG	70,0	0,0005	0,0015	0,86
	RAG vs. GraphRAG	285,0	0,4744	1,0000	0,43
Kos. sličnost	Van. vs. RAG	32,0	$< 0,0001$	0,0001	0,94
	Van. vs. GraphRAG	44,0	0,0001	0,0002	0,91
	RAG vs. GraphRAG	246,0	0,9765	1,0000	0,50

Friedmanov test (omnibus): ROUGE-L $\chi^2(2) = 19,61$, $p = 0,0001$; BERTScore $\chi^2(2) = 19,94$, $p < 0,0001$; kosinusna sličnost $\chi^2(2) = 21,36$, $p < 0,0001$.

Isti obrazac vidljiv je i kod automatskih metrika, izmjerenih neovisno o ljudskoj evaluaciji: sve tri metrike pokazuju statistički značajnu i vrlo veliku razliku između Vanilla konfiguracije i obiju konfiguracija s dohvatom ($r = 0,86\text{--}0,95$), dok razlika RAG-a i GraphRAG-a ni na jednoj metrici nije značajna ($p_{\text{Bonf.}} = 1,0000$ svuda), uz umjerene veličine efekta ($r = 0,43\text{--}0,50$) koje ne prelaze prag pouzdanosti pri ovoj veličini uzorka. Implikacije podudarnosti ova dva neovisna izvora dokaza razmatraju se u poglavlju 6.8, zajedno s raspravom o mogućim uzrocima u poglavlju 7.5.

6.4. Korelacija automatskih metrika s ljudskom evaluacijom

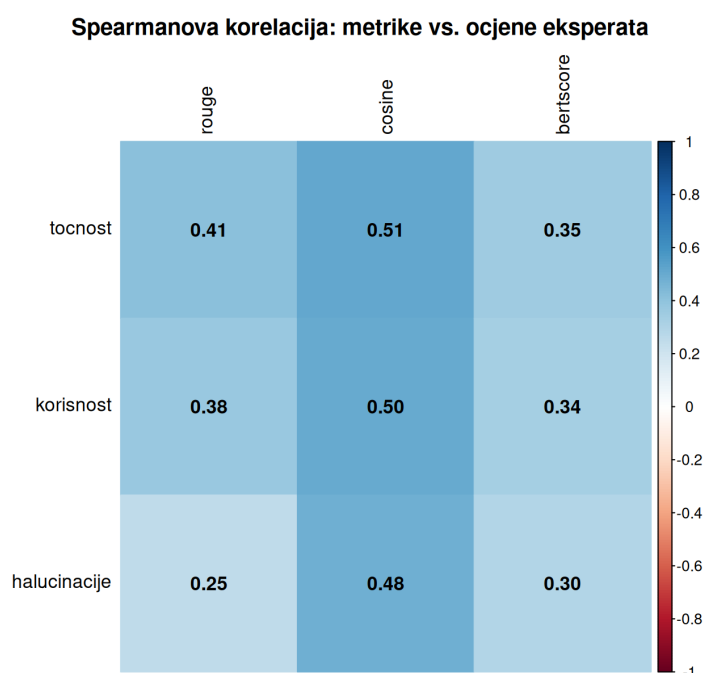
Tablica 7: Spearmanova korelacija (ρ) automatskih metrika s dimenzijama ljudske evaluacije (* $p < 0,05$, ** $p < 0,01$)

Dimenzija	ROUGE-L	BERTScore	Kos. sličnost
Točnost	0,41**	0,35**	0,51**
Korisnost	0,38**	0,34**	0,50**
Halucinacije	0,25*	0,30**	0,48**

Sve devet koeficijenata korelacije pozitivno je i statistički značajno, no po apsolutnoj vrijednosti spadaju u raspon slabe do umjerene povezanosti (0,25–0,51 prema uobičajenim konvencijama tumačenja), što potvrđuje raspravu iz potpoglavlja 5.2.4: automatske metrike prate isti opći smjer kao ljudska prosudba, ali objašnjavaju tek 6–26 % varijance ljudskih ocjena (ρ^2), pa ih ne treba tumačiti kao zamjenu za nju. Kosinusna sličnost dosljedno je najjače povezana s ljudskom evaluacijom na sve tri dimenzije (0,48–0,51), ispred BERTScore-a (0,30–0,35)

i ROUGE-L-a (0,25–0,41), što je očekivano jer kosinusna sličnost, za razliku od leksičkih i tokenskih metrika, uspoređuje cjelokupno semantičko značenje odgovora. ROUGE-L pokazuje najslabiju povezanost upravo s dimenzijom halucinacija (0,25), što je razumljivo s obzirom na to da leksičko podudaranje ne detektira izmišljen sadržaj sve dok se on leksički uklapa u kontekst odgovora.

Slika 13 vizualizira istu korelacijsku matricu, gdje se bojom kodira jačina i smjer povezanosti između svake automatske metrike i dimenzije ljudske evaluacije.



Slika 13: Toplinska karta Spearmanove korelacije automatskih metrika s dimenzijama ljudske evaluacije.

6.5. Međuocjenjivačka pouzdanost

Tablica 8: Međuocjenjivačka pouzdanost (11 ocjenjivača, nakon isključivanja triju blago ocjenjujućih)

Dimenzija	Fleissov κ	Krippendorffov α	Interpretacija
Točnost	0,153	0,168	slabo
Korisnost	0,087	0,117	slabo
Halucinacije	0,056	0,031	slabo

Međuocjenjivačka pouzdanost slaba je na sve tri dimenzije, i to i nakon isključivanja triju blago ocjenjujućih ocjenjivača (usporedbe radi, na punom skupu od 14 ocjenjivača vrijednosti su bile još niže: Fleissov κ 0,007–0,074). Ovo je važno ograničenje koje treba uzeti u obzir pri tumačenju svih ostalih nalaza u ovom poglavlju: pojedinačni ocjenjivači često se ne slažu

oko konkretne ocjene za dano pitanje i konfiguraciju, što je razumljivo s obzirom na suptilnost pravne prosudbe (dio odgovora može biti djelomično točan, formalno ispravan ali nepotpun, ili ovisan o interpretaciji propisa) te na relativno grubu tročlanu ljestvicu koja ne dopušta nijnasiranje ocjena. Niska pouzdanost na razini pojedinačne ocjene ipak ne poništava nalaze iz poglavlja 6.3: agregacijom (medijanom) jedanaest ocjenjivača po pitanju, šum pojedinačnog neslaganja se uglavnom poništava, a razlika između Vanilla i konfiguracija s dohvatom pokazala se dovoljno velikom i dosljednom da premaši tu razinu šuma i ostane statistički značajna. Razliku između RAG-a i GraphRAG-a, koja je već sama po sebi mala, međutim treba tumačiti s dodatnim oprezom upravo zbog niske pouzdanosti temeljnih ocjena od kojih je izvedena.

6.6. LLM-as-a-Judge: Claude i Gemini

Poglavlje 5.4 uvelo je LLM-as-a-Judge kao dopunsku metodu evaluacije, no jedan LLM ocjenjivač sam po sebi ne dopušta procjenu koliko su njegove ocjene pouzdane ili specifične za odabrani model. U ovom se poglavlju stoga uz Claude uvodi i drugi, neovisan LLM ocjenjivač (Gemini), koji je odgovore svih triju verzija chatbota ocijenio prema istom promptu i istim dimenzijama (Tablica 1). Statistička analiza provodi se u dva koraka: najprije se svaki ocjenjivač zasebno uspoređuje s ljudskom evaluacijom Spearmanovom korelacijom (potpoglavlje 6.6.1, Tablica 9), a potom se ocjene dvaju modela uspoređuju međusobno Spearmanovom korelacijom i Wilcoxonovim testom predznačenih rangova (potpoglavlje 6.6.2, Tablica 10), čime se procjenjuje slaže li se odabir LLM ocjenjivača s ljudskom prosudbom i međusobno.

6.6.1. Korelacija s ljudskom evaluacijom

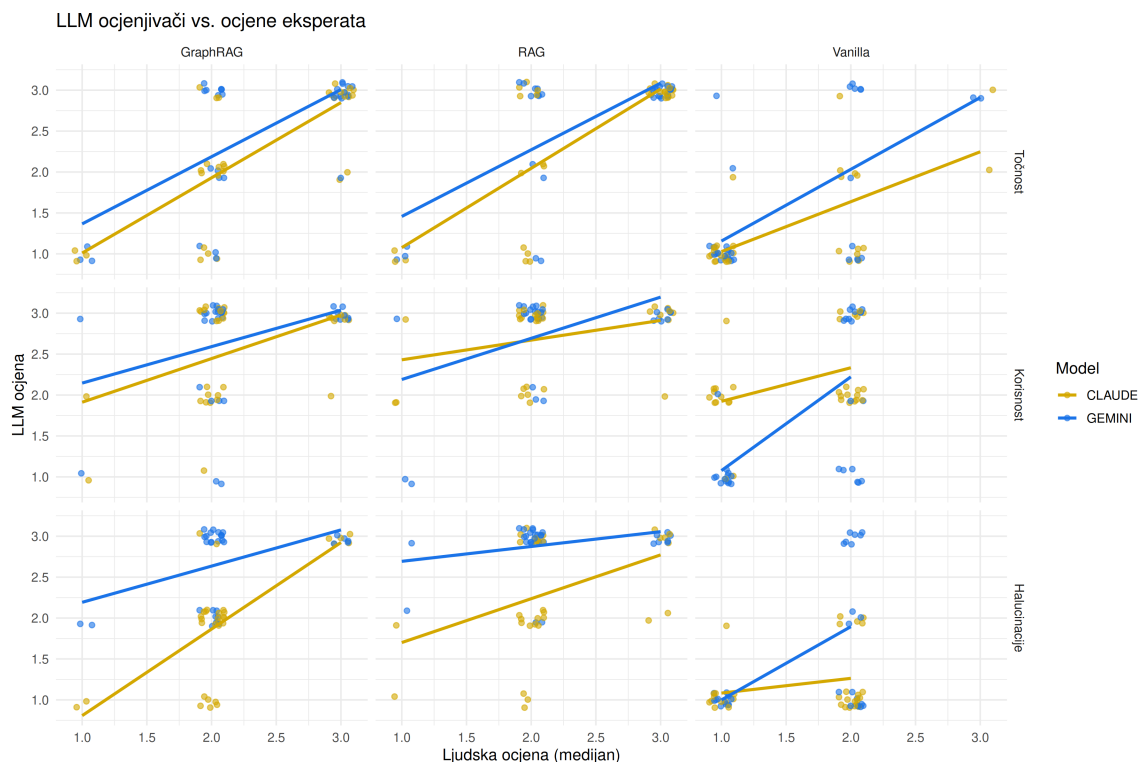
Tablica 9: Spearmanova korelacija LLM ocjenjivača s ljudskom evaluacijom (ρ)

Dimenzija	Claude		Gemini	
	ρ	p	ρ	p
Točnost	0,81	< 0,0001	0,74	< 0,0001
Korisnost	0,49	< 0,0001	0,61	< 0,0001
Halucinacije	0,59	< 0,0001	0,58	< 0,0001

Oba LLM ocjenjivača koreliraju s ljudskom evaluacijom znatno jače nego bilo koja automatska metrika iz Tablice 7 (usporedivo $\rho = 0,49\text{--}0,81$ ovdje naspram $\rho = 0,25\text{--}0,51$ za ROUGE-L, BERTScore i kosinusnu sličnost), što potvrđuje očekivanje da jezični model, sposoban razumjeti sadržaj i kontekst odgovora, bolje aproksimira ljudsku prosudbu od metrika utemeljenih na leksičkom ili vektorskom podudaranju. Claude bolje korelira s ljudskom ocjenom točnosti (0,81 naspram 0,74), dok Gemini nešto bolje korelira s korisnošću (0,61 naspram 0,49); na dimenziji halucinacija oba su modela slično usklađena s ljudskom prosudbom (0,58–0,59). Nijedan koeficijent, iako najviši u ovom radu, ne prelazi $\rho \approx 0,8$, što znači da čak ni najbolji LLM ocjenjivač nije savršena zamjena za ljudsku evaluaciju, samo njezina znatno jefti-

nija i skalabilnija aproksimacija.

Slika 14 prikazuje istu povezanost raspršeno, po dimenziji i konfiguraciji: svaka točka predstavlja jedno pitanje, a linija regresijski trend LLM ocjene u odnosu na medijan ljudske ocjene za dani model.



Slika 14: Odnos LLM ocjena (Claude, Gemini) i ljudskih ocjena, po dimenziji i konfiguraciji.

6.6.2. Međusobna usporedba: Claude vs. Gemini

Tablica 10: Usporedba ocjena Claude i Gemini ocjenjivača (upareni Wilcoxonov test)

Dimenzija	ρ	W	p	$p_{\text{Bonf.}}$
Točnost	0,86	9,5	0,0001	0,0003
Korisnost	0,72	320,0	0,2409	1,0000
Halucinacije	0,73	21,0	< 0,0001	< 0,0001

Claude i Gemini visoko su međusobno korelirani na sve tri dimenzije ($\rho = 0,72\text{--}0,86$), odnosno slažu se oko toga koji su odgovori relativno bolji, a koji lošiji. Wilcoxonov test uparenih razlika pritom otkriva da se njihove apsolutne razine ocjenjivanja ipak sustavno razlikuju na točnosti i, osobito, na halucinacijama ($p_{\text{Bonf.}} < 0,001$ na objema): Gemini u prosjeku dodjeljuje više ocjene i za točnost (2,22 naspram 2,00 kod Claudea) i, znatno izraženije, za halucinacije (2,38 naspram 1,83 kod Claudea, na skali 1–3), dok su njihove ocjene korisnosti međusobno usklađene i u rangui i u razini (2,40 naspram 2,47; Wilcoxon nije značajan, $p_{\text{Bonf.}} = 1,0000$). Praktična posljedica jest da je Gemini osjetno blaži ocjenjivač po pitanju halucinacija, tj. rjeđe

kažnjava sitne izmišljene detalje, dok Claude na toj dimenziji znatno strože razlikuje odgovore. Budući da se ova razlika odnosi na apsolutnu razinu, a ne na relativno rangiranje triju konfiguracija, ne dovodi u pitanje zaključke iz poglavlja 6.3, ali upozorava da apsolutne vrijednosti LLM-as-a-Judge ocjena nisu izravno usporedive između različitih modela ocjenjivača.

6.7. Latencija

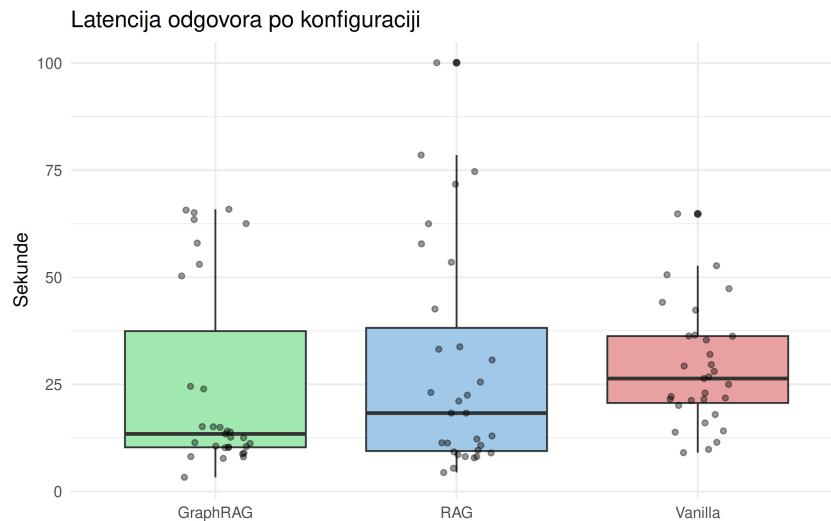
Tablica 11: Statistika latencije odgovora (u sekundama)

Konfiguracija	$\bar{x}$	Med.	P_{95}	s
Vanilla	28,61	26,36	51,64	13,53
RAG	28,93	18,29	76,61	25,91
GraphRAG	24,63	13,43	65,39	22,07

Friedmanov test nad uparenim mjerenjima latencije po pitanju ne pokazuje statistički značajnu razliku između triju konfiguracija ($\chi^2(2) = 4,71$, $p = 0,0949$). Iako RAG i Vanilla imaju sličnu prosječnu latenciju, a GraphRAG u prosjeku nešto nižu, razlike nisu dosljedne na razini pojedinih pitanja te ne doseže razinu statističke značajnosti, pa se latencija triju pristupa u ovom uzorku ne može smatrati pouzdano različitom.

Sve tri konfiguracije pokazuju znatno veći medijan od prosjeka (najizraženije kod Graph RAG: medijan 13,43 s naspram prosjeka 24,63 s) te visoke vrijednosti 95. percentila (do 76,61 s kod RAG-a), što upućuje na desno asimetričnu raspodjelu s povremenim znatno sporijim odgovorima, umjesto na ujednačeno povišenu latenciju. Za praktičnu primjenu to znači da je važnije planirati za rijetke, ali znatne odgode (npr. prilikom osobito opsežnih ili međudokument-nih pitanja) nego se osloniti isključivo na prosječno vrijeme odgovora.

Slika 15 prikazuje raspodjelu latencije po konfiguraciji, uključujući pojedinačna mjerenja (točke) preko kojih se vidi široki raspon i preklapanje distribucija triju pristupa, u skladu s nalazom da razlika nije statistički značajna.



Slika 15: Distribucija latencije odgovora po konfiguraciji.

6.8. Sinteza rezultata

Statistička analiza provedena u ovom poglavlju (poglavlja 6.1–6.7) daje pet glavnih, međusobno konzistentnih nalaza.

Dohvat konteksta jasno i dosljedno nadmašuje model bez dohvata. Na sve tri dimenzije ljudske evaluacije (točnost, korisnost, halucinacije) i na sve tri automatske metrike (ROUGE-L, BERTScore, kosinusna sličnost), RAG i GraphRAG statistički su značajno bolji od Vanilla konfiguracije, uz vrlo velike veličine efekta ($r \geq 0,86$ posvuda). Nijedna konfiguracija s dohvatom nije, ni po jednoj promatranoj mjeri, lošija ili statistički neuspoređena s Vanilla polaznom točkom. Ovo je naj snažniji i najjednoznačniji nalaz cijele evaluacije.

GraphRAG ne donosi mjerljivu prednost nad RAG-om. Ni na jednoj od triju dimenzija ljudske evaluacije, kao ni na jednoj od triju automatskih metrika, razlika između RAG-a i GraphRAG-a nije statistički značajna. (post-hoc $p_{\text{Bonf.}} = 1,0000$ u svih šest usporedbi), uz umjerene, ali nedovoljne veličine efekta ($r = 0,33\text{--}0,53$). Deskriptivna statistika (Tablica 3) čak sugerira da RAG ima nešto viši prosjek točnosti od GraphRAG-a.

Kosinusna sličnost najbolja je automatska aproksimacija ljudske prosudbe, ali nijedna automatska metrika nije dovoljna sama za sebe. Sve tri automatske metrike statistički značajno koreliraju s ljudskom evaluacijom, no korelacije su tek slabe do umjerene ($\rho = 0,25\text{--}0,51$), pri čemu kosinusna sličnost dosljedno prednjači na sve tri dimenzije. LLM-as-a-Judge pristup (Claude, Gemini) korelira s ljudskom evaluacijom znatno jače ($\rho = 0,49\text{--}0,81$) od bilo koje automatske metrike, čime se potvrđuje odabir kombinacije metoda opisan u poglavlju 5.5, automatske metrike korisne su za brzo, skalabilno filtriranje, ali konačna prosudba treba se oslanjati na LLM-as-a-Judge i ljudsku evaluaciju.

Claude i Gemini se slažu u rangiranju, ali ne i u apsolutnoj strogosti ocjenjivanja. Visoka međusobna korelacija ($\rho = 0,72\text{--}0,86$) pokazuje da oba modela slično rangiraju odgovore, no Gemini je sustavno blaži ocjenjivač na dimenziji halucinacija (i, u manjoj mjeri,

točnosti), dok su na korisnosti oba modela usklađena i u rangui i u razini. Nijedan od dva modela nije univerzalno “pouzdaniji”: Claude bolje prati ljudsku ocjenu točnosti, Gemini bolje prati korisnost, a njihova razlika u strogosti znači da se apsolutne LLM ocjene ne smiju izravno uspoređivati između modela bez korekcije.

Latencija ne pokazuje pouzdanu razliku između konfiguracija, unatoč intuiciji da dohvat treba usporiti odgovor. Friedmanov test ne otkriva značajnu razliku ($p = 0,0949$), a raspodjele latencije preklapaju se uz izraženu desnu asimetriju kod sve tri konfiguracije. Prema tome, u ovom sustavu dohvat konteksta ne dolazi s mjerljivom vremenskom cijenom u odnosu na izravno generiranje, što dodatno ide u prilog korištenju RAG-a nad Vanilla pristupom bez potrebe za kompromisom brzina–kvaliteta.

Konačno, sve navedene zaključke treba čitati uz ogradu utvrđenu u poglavlju 6.5, međuocjenjivačka pouzdanost ljudskih ocjena je niska na sve tri dimenzije, pa iako agregacija (medijan jedanaest ocjenjivača) čini glavni nalaz, razliku Vanilla naspram RAG/GraphRAG, dovoljno otpornim da premaši tu razinu šuma, manju i suptilniju razliku između RAG-a i GraphRAG-a valja tumačiti sa znatno više opreza. Šira interpretacija ovih nalaza i njihove implikacije za daljnji razvoj sustava dane su u poglavlju Zaključak.

7. Iterativni razvoj i dijagnostika sustava

Razvoj RAG chatbota za hrvatsko obrambeno pravo prošao je kroz **deset** iteracija, pri čemu je svaka verzija uvodila specifične konfiguracije s ciljem poboljšanja točnosti i stabilnosti odgovora.

7.1. Evolucija RAG sustava kroz iterativno poboljšavanje

Početna verzija je kreirana uz uporabu jezičnog modela otvorenog koda EuroLLM 9B (devet milijardi parametara) i isti je pokazao osnovnu funkcionalnost, ali s niskom točnošću. Vanilla model postigao je samo 19% točnosti, dok je RAG s 42% demonstrirao potencijal pristupa temeljenog na dohvaćanju relevantnog konteksta. Treća verzija postigla je iznenađujućih 90% točnosti, ali naknadnom analizom se otkrilo kako je evaluacijski skup pitanja (zlatni standard) bio uključen u korpus kao Q&A parovi, što je rezultiralo curenjem podataka (*data leakage*) i umjetno visokim rezultatima. Ovaj nalaz naglasio je važnost jasne separacije trenažnih podataka od evaluacijskog skupa te potrebu za pažljivim dizajnom eksperimenata u domeni pravnih chatbotova.

7.2. Specifični tehnički izazovi i njihova rješenja

Najznačajniji tehnički problem kroz sve iteracije bila je višejezična greška curenja (*bleeding*) greška, neželjeno generiranje teksta na bugarskom, srpskom ili ruskom jeziku, često u ćirilicom pismu. Problem je prvi put identificiran u verziji v06 gdje je u četiri slučaja odgovor završavao na bugarskom jeziku ili u ćirilici, primjerice: года. Этот срок указан в Статье 159. Uzrok je bio u činjenici da je EuroLLM višejezični model treniran na blisko srodnim južnoslavenskim jezicima, što u vektorskom prostoru stvara preklapanje između hrvatskog, srpskog i bugarskog. Pokušaji rješavanja ovog problema kroz prompt engineering, dodavanjem eksplicitnih uputa poput “NIKAD ne koristi ćirilicu, bugarski ili srpski jezik”, pokazali su se neučinkovitima, pa čak i kontraproduktivnima. Verzija v09 s agresivnim prompt upozorenjima rezultirala je paradoksalnim efektom: model je razvio svojevrsnu “samosvijest o uputama” (*meta-awareness*) i počeo citirati vlastite upute u izlazu, npr. УВАГА! Не користите ЋИРИЛИКУ или СРПСКИЕ ЕЗЫКИ, doslovno upozoravajući sam sebe na ukrajinskom jeziku.

7.3. Eksperimentiranje s izborom osnovnog modela: EuroLLM naspram Qwen2.5

Upravo opisani problem višejezičnog curenja, s ograničenom učinkovitošću prompt engineeringa u njegovu suzbijanju, motivirao je preispitivanje samog izbora osnovnog jezičnog modela. Iako je EuroLLM eksplicitno dizajniran za europske jezike, upravo ga je njegova izloženost blisko srodnim južnoslavenskim jezicima i ćirilicom pismu u ranijim inačicama činila sklonim generiranju teksta na bugarskom, srpskom ili ruskom jeziku.

U prvom koraku pokušano je problem ublažiti nadogradnjom na noviju i veću inačicu istog modela, `EuroLLM-22B-Instruct`, s 22 milijarde parametara i kontekstnim prozorom od 32 000 tokena. Veći model donio je mjerljivo poboljšanje kvalitete i jezične čistoće odgovora u odnosu na inačicu s 9 milijardi parametara; u provedenoj evaluaciji klizanje u ćirilčno pismo više nije zabilježeno. Međutim, model od 22 milijarde parametara, čak i uz četverobitnu kvantizaciju, nalazio se na samoj gornjoj granici memorijskih mogućnosti grafičkog procesora L4 (24 GB) dostupnog u plaćenju inačici platforme Google Colab, što je ostavljalo malo prostora za kontekst dohvaćenih dokumenata i povećavalo rizik od prekoračenja dostupne memorije tijekom evaluacije cijeloga skupa pitanja.

Kao alternativa razmotren je model `Qwen2.5-14B-Instruct`, arhitekturno usporediv dekoderski transformer s jednakim kontekstnim prozorom od 32 000 tokena, no treniran na pretežno globalnom, a ne specifično europskom korpusu. Usporedna evaluacija dvaju modela na istom skupu od 31 pitanja pokazala je da nijedan od njih ne pruža bezuvjetno bolju kvalitetu, nego da svaki ima specifične slabosti. Ni kod jednoga modela nije zabilježeno klizanje u ćirilicu, čime je najteži oblik problema iz ranijih verzija uklonjen promjenom osnovnoga modela. Time, međutim, jezična čistoća nije postala potpuna: kod modela `Qwen2.5` javljalo se zamjetno leksičko miješanje sa srodnim južnoslavenskim jezicima u latiničnom obliku (primjerice srbizmi i iskrivljeni oblici poput “bezbednost”, “okamjenu” ili “potrebuju”), dok je `EuroLLM-22B` u tom pogledu bio leksički čišći uz tek povremena odstupanja (npr. “korišćenje”).

S druge strane, `EuroLLM-22B` pokazao je ozbiljniju slabost u stabilnosti generiranja: u jednom je odgovoru model upao u repeticijsku petlju i jedanaest puta ponovio istu rečenicu, što je oblik nestabilnosti kakav `Qwen2.5` na istom skupu nije pokazao. Halucinacije u standardnoj (*vanilla*) konfiguraciji bile su prisutne kod oba modela, ponajprije u obliku izmišljenih brojeva “Narodnih novina”, no RAG konfiguracija ih je kod oba znatno smanjila.

Usporedba na razini tokenizacije otkrila je prednost na strani `EuroLLM`-a koja je relevantna za hrvatski jezik. Za identičnu testnu rečenicu (“S koliko godina časnički namjesnik ide u mirovinu?”) tokenizator modela `Qwen2.5` proizveo je 20 tokena, dok je `EuroLLM` istu rečenicu kodirao u svega 14 tokena, što predstavlja oko 30 % manje. Razlika proizlazi iz činjenice da `Qwen` koristi tokenizaciju na razini bajtova (*byte-level* BPE), zbog čega se hrvatska slova s dijakritičkim znakovima (č, ć, ž, š, đ) rastavljaju na više pod-tokena, dok ih tokenizator modela `EuroLLM`, treniran na korpusu bogatom europskim jezicima, češće zadržava kao cjelovite jedinice. Učinkovitija tokenizacija znači da u isti kontekstni prozor stane više dohvaćenog pravnog teksta te da generiranje odgovora iziskuje manje koraka.

Budući da dva modela nisu pokazala jednoznačnu razliku u kvaliteti odgovora, u tom se trenutku odabir sveo na pitanje resursne izvedivosti: na dostupnom grafičkom procesoru L4 (24 GB) “lakši” `Qwen2.5-14B-Instruct` ostavljao je više memorijskog prostora za stabilan rad i puni kontekst dohvaćenih dokumenata, dok se `EuroLLM-22B` nalazio na granici dostupne memorije i predstavljao rizik za pouzdano izvođenje cijele evaluacije. To je ograničenje, međutim, naknadno prestalo vrijediti prelaskom razvojnog okruženja na grafički procesor NVIDIA A100 s 40 GB memorije (potpoglavlje 4.3), pri čemu je memorijski prostor koji je ranije favorizirao `Qwen2.5-14B-Instruct` postao dovoljno velik i za `EuroLLM-22B`. Budući da je `EuroLLM-22B`

pritom pokazao bolju tokenizacijsku učinkovitost za hrvatski jezik i leksički čišće odgovore, a njegov jedini uočeni nedostatak, povremena repeticijska petlja, mogao se ublažiti kalibracijom parametra `repetition_penalty` (potpoglavlje 7.4), EuroLLM-22B-Instruct zadržan je kao konačni osnovni model sustava, dok je Qwen2.5-14B-Instruct ostao dokumentirana, ali naposljetku napuštena alternativa. Sama zamjena identifikatora modela između kandidata tehnički je bila nezahtjevna zahvaljujući standardiziranom sučelju biblioteke *Transformers*: izmjena se svela na zamjenu identifikatora modela, jer su tokenizacija i predložak razgovora (*chat template*) automatski preuzeti iz konfiguracije svakog modela, dok je cjelokupna RAG infrastruktura (model za vektorsku reprezentaciju, vektorska baza, strategija segmentacije i dohvrat) ostala nepromijenjena, čime je omogućena metodološki čista usporedba kandidata pod istovjetnim uvjetima.

7.4. Optimizacija generacijskih parametara i njihov utjecaj

Osim lingvističkih problema, iterativno testiranje otkrilo je kritičnu važnost pažljivog balansiranja generacijskih hiperparametara. Verzija v09 pokušala je povećati determinizam snižavanjem temperature s 0.7 na 0.3 kako bi se smanjila varijabilnost odgovora, što je rezultiralo katastrofalnim pogreškom nepostojanja razmaka, model je generirao 500+ znakova bez razmaka (npr. “SustavimenovanjaprinadelnacenicavGlavnomstožeru”). Niska temperatura uzrokovala je da model ulazi u repetitivne uzorke iz kojih se, bez dovoljno stohastičnosti, nije mogao izvući. Istovremeno, parametar `repetition_penalty` morao je biti pažljivo kalibriran: vrijednost 0.6 u v06 bila je preniska što je dopuštalo ponavljanje rečenica, dok je vrijednost 1.3 u srednjim verzijama, temeljenim na modelu EuroLLM-9B, predstavljala potreban korektiv. Prelaskom na novije i veće modele (EuroLLM-22B i Qwen2.5-14B), za koje se očekivalo da budu manje skloni repetitivnim obrascima, ta je vrijednost snižena na 1.1, čime je smanjen rizik da prejaka kazna potisne nužno ponavljanje pravne terminologije (primjerice “članak”, “Oružane snage”). Valja napomenuti kako niža vrijednost kazne nije posve uklonila rizik od repeticije: model EuroLLM-22B u jednom je odgovoru ipak upao u repeticijsku petlju, što pokazuje da podešavanje ovog parametra ublažava, ali ne jamči stabilnost generiranja te da ga treba nadopuniti provjerom naknadne obrade.

Kroz ove iteracije postalo je jasno da uspješan RAG sustav za pravnu domenu zahtjeva multimodalnu obranu od grešaka: kombinaciju prompt engineeringa za visokorazinsko usmjerenje, naknadne obrade za sigurnosnu provjeru te pažljivo izbalansiranih generacijskih parametara koji omogućuju dovoljnu kreativnost za izbjegavanje petlji, ali dovoljnu konzistentnost za precizne pravne odgovore. Također se pokazalo kako pojedinačne metrike poput BERTScore ili ROUGE-a nisu dovoljan pokazatelj kvalitete, potrebna je kombinacija automatskih metrika (posebno BERTScore koji korelira s ručnim ocjenama), LLM-as-a-Judge evaluacije za skalabilnost, te ekspertne validacije za referentne podatke u pravnoj domeni.

7.5. Zašto GraphRAG nije briljirao?

Opisati razloge zašto GraphRAG nije donio očekivano poboljšanje u odnosu na RAG, uključujući moguće tehničke i metodološke razloge. Kao polazište za analizu razmotriti barem sljedeće kandidate, uz provjeru koji se od njih doista mogu potkrijepiti podacima iz provedene evaluacije (potpoglavlje 6.1 i dalje):

- **Izostanak optimizacije upita prije dohvata** (potpoglavlje o optimizaciji upita u teorijskom pregledu RAG-a) - GraphRAG vektorski dohvaća početne čvorove istim, neizmijenjenim korisničkim upitom kao i RAG verzija, bez reformulacije ili usmjerenja upita prema vektorskom indeksu naspram obilaska grafa; kratka ili nejasno formulirana pitanja stoga mogu jednako loše pogoditi početne čvorove grafa kao i odlomke u RAG-u, čime prednost strukturiranog obilaska grafa ostaje neiskorištena već u prvom koraku dohvata.
- **Odsutnost formalnog ponovnog rangiranja** - obje verzije koriste tek heurističko rangiranje (prag sličnosti, hijerarhijski bonus, deduplikacija), bez cross-encoder ili LLM-as-judge koraka koji bi mogao ispraviti pogreške početne vektorske pretrage prije obilaska grafa.
- **Plitkoća obilaska grafa** - obilazak je ograničen na jedan korak po tipiziranim relacijama, čime se ne iskorištavaju potencijalno relevantne veze udaljenije od neposrednih susjeda.
- **Kvaliteta automatski izvedenih `ISTA_TEMA` veza i prepoznavanja entiteta** - pravovi i normalizacija opisani u potpoglavlju 4.6.3 mogu propustiti stvarne pravne poveznice ili, obrnuto, uvesti šum kroz generičke entitete.
- **Manja pokrivenost skupa pitanja za evaluaciju međudokumentnim vezama** - ako većina testnih pitanja ne zahtijeva objedinjavanje odredbi iz više propisa, prednost grafovske strukture nad plošnom vektorskom pretragom ionako nema priliku doći do izražaja.

8. Odraditi?

Etička i praktična napomena o podacima: korpus je javno dostupno zakonodavstvo (Narodne novine), pa nema problema s povjerljivošću nema. Jednom rečenicom opisati

Ograničenja: N=31, slaba inter-rater pouzdanost, jednostruki obilazak GraphRAG-a. Raspršeno u poglavlju 7, okupiti i okrupniti na jedno mjesto.

Popis kratica

9. Zaključak

Analiza iterativnog razvoja pokazala je kako razvoj specijaliziranog chatbota za pravnu domenu zahtjeva sistemski pristup dijagnostici i rješavanju problema. Ključna spoznaja je da **tehnička implementacija mora pratiti prompt engineering**, potrebno je duboko razumijevanje interakcije između modela, korpusa, prompt dizajna i generacijskih parametara. Iterativni pristup omogućio je identifikaciju problema koji nisu vidljivi u pojedinačnim testovima: od metodoloških grešaka poput curenja podataka, preko lingvističkih izazova višjezičnog modela i odabira primjerenijeg osnovnog modela, do suptilnih efekata hiperparametara koji mogu uzrokovati potpuno nečitljive odgovore.

Najvažnija lekcija je da **prompt engineering sam za sebe nije dovoljno otporan**, pokazalo se da eksplicitne tekstualne zabrane mogu čak pogoršati problem stvarajući “samo-svijest o uputama” u modelu. Promjena osnovnog modela uklonila je najteži oblik problema (ćirilično curenje), ali ne i njegove suptilnije latinične tragove, što potvrđuje da izbor modela ublažava, ali ne zamjenjuje potrebu za tehničkim zaštitama. Finalna implementacija nije u potpunosti primijenila sve identificirane korekcije, što znači da su pojedine greške poput greške nepostojanja razmaka i preostalog leksičkog miješanja **i dalje mogući u produkcijskom okruženju**, unatoč identifikaciji uzroka.

Konačna verzija s pojednostavljenim sistemskim promptom postiže zadovoljavajuće rezultate u kontroliranim uvjetima, ali **nedostatak otpornih tehničkih zaštita** (validacija naknadne obrade, optimalna temperatura) predstavlja rizik za produkcijsku upotrebu. Ovo potvrđuje glavnu tezu istraživanja: **za produkcijski RAG sustav u pravnoj domeni potreban je hibridni pristup** koji integrira prompt engineering za visokorazinsko usmjeravanje, tehničko filtriranje za kritične zabrane, naknadne obrade za validaciju izlaznih podataka te pažljivo eksperimentalno podešavanje generacijskih parametara. Bez ovog integralnog pristupa, sustav ostaje podložan greškama koje mogu kompromitirati pouzdanost u pravnom kontekstu gdje je faktička točnost kritična.

Popis literature

- [1] J. Naisbitt, *Megatrends: 10 new directions transforming our lives*. Warner Books, 1982., str. 17.
- [2] M. Al-Amin, M. S. Ali, A. Salam i dr., „History of generative Artificial Intelligence (AI) chatbots: past, present, and future development,” *arXiv preprint arXiv:2402.05122*, 2024.
- [3] A. M. Turing, „Computing Machinery and Intelligence,” *Mind*, sv. 59, br. 236, str. 433–460, 1950.
- [4] R. Lane, A. Hay, A. Schwarz, D. M. Berry i J. Shrager, „ELIZA Reanimated: The world’s first chatbot restored on the world’s first time sharing system,” *arXiv preprint arXiv:2501.06707*, 2025.
- [5] D. F. Murad, A. G. Iskandar, E. Fernando, T. S. Octavia i D. E. Maured, „Towards smart LMS to improve learning outcomes students using LenoBot with natural language processing,” *2019 6th International Conference on Information Technology, Computer and Electrical Engineering (ICITACEE)*, IEEE, 2019., str. 1–6.
- [6] E. Adamopoulou i L. Moussiades, „An Overview of Chatbot Technology,” *Artificial Intelligence Applications and Innovations (AIAI 2020)*, serija IFIP Advances in Information and Communication Technology, sv. 584, Springer Nature Switzerland AG, 2020., str. 373–383. DOI: 10.1007/978-3-030-49186-4_31.
- [7] R. S. Wallace, „Artificial Intelligence Markup Language (AIML) Version 1.0.1,” A.L.I.C.E. AI Foundation, teh. izv., 2001.
- [8] S. Team, *Hey Siri: An On-device DNN-powered Voice Trigger for Apple’s Personal Assistant*, <https://machinelearning.apple.com/research/hey-siri>, [Pristupljeno 09.01.2026], 2017.
- [9] P. Muangkammuen i N. Intiruk, „Automated Thai-FAQ Chatbot using RNN-LSTM,” *2019 5th International Conference on Engineering, Applied Sciences and Technology (ICEAST)*, IEEE, 2019., str. 1–4. DOI: 10.1109/ICEAST.2019.8712781.
- [10] A. Vaswani, N. Shazeer, N. Parmar i dr., „Attention is all you need,” *Advances in neural information processing systems*, sv. 30, 2017.
- [11] H. Naveed, A. U. Khan, S. Qiu i dr., „A comprehensive overview of large language models,” *ACM Transactions on Intelligent Systems and Technology*, sv. 16, br. 5, str. 1–72, 2025.

- [12] C. Staff, *What is an AI Engineer? (And how to become one)*, <https://www.coursera.org/articles/ai-engineer>, [Pristupljeno 13.01.2026], 2025.
- [13] C. Zhou, Q. Li, C. Li i dr., „A comprehensive survey on pretrained foundation models: A history from bert to chatgpt,” *International Journal of Machine Learning and Cybernetics*, sv. 16, br. 12, str. 9851–9915, 2025.
- [14] C. Huyen, *AI Engineering: Building Applications with Foundation Models*. Sebastopol, CA: O'Reilly Media, 2025., ISBN: 978-1-098-16630-4.
- [15] L. Ouyang, J. Wu, X. Jiang i dr., „Training Language Models to Follow Instructions with Human Feedback,” *arXiv preprint arXiv:2203.02155*, 2022.
- [16] E. Saravia, *Prompt Engineering Guide*, 12. 2022.
- [17] T. B. Brown, B. Mann, N. Ryder i dr., „Language Models are Few-Shot Learners,” *arXiv preprint arXiv:2005.14165*, 2020.
- [18] B. Han, T. Susnjak i A. Mathrani, „Automating Systematic Literature Reviews with Retrieval-Augmented Generation: A Comprehensive Overview,” *Applied Sciences*, sv. 14, br. 19, 2024., ISSN: 2076-3417. DOI: 10.3390/app14199103.
- [19] Y. A. Malkov i D. A. Yashunin, „Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, sv. 42, br. 4, str. 824–836, 2020. DOI: 10.1109/TPAMI.2018.2889473.
- [20] Y. Gao, Y. Xiong, X. Gao i dr., „Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2023.
- [21] T. Dettmers, A. Pagnoni, A. Holtzman i L. Zettlemoyer, „Qlora: Efficient finetuning of quantized llms,” *Advances in Neural Information Processing Systems*, sv. 36, str. 10 088–10 115, 2023.
- [22] J. Nielsen, *Usability Engineering*. Academic Press, 1993.
- [23] C.-Y. Lin, „ROUGE: A Package for Automatic Evaluation of Summaries,” *Proceedings of the ACL Workshop on Text Summarization Branches Out*, 2004., str. 74–81.
- [24] K. Papineni, S. Roukos, T. Ward i W.-J. Zhu, „BLEU: a Method for Automatic Evaluation of Machine Translation,” *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 2002., str. 311–318.
- [25] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger i Y. Artzi, „BERTScore: Evaluating Text Generation with BERT,” *International Conference on Learning Representations*, 2020.
- [26] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder i F. Wei, „Multilingual E5 Text Embeddings: A Technical Report,” *arXiv preprint arXiv:2402.05672*, 2024.
- [27] J. L. Fleiss, „Measuring Nominal Scale Agreement Among Many Raters,” *Psychological Bulletin*, sv. 76, br. 5, str. 378–382, 1971.
- [28] J. Cohen, „A Coefficient of Agreement for Nominal Scales,” *Educational and Psychological Measurement*, sv. 20, br. 1, str. 37–46, 1960.

- [29] J. R. Landis i G. G. Koch, „The Measurement of Observer Agreement for Categorical Data,” *Biometrics*, sv. 33, br. 1, str. 159–174, 1977.
- [30] K. Krippendorff, „Computing Krippendorff’s Alpha-Reliability,” Annenberg School for Communication, University of Pennsylvania, teh. izv., 2011.
- [31] K. H. Cheng, C. H. Hui i W. F. Cascio, „Leniency bias in performance ratings: The big-five correlates,” *Frontiers in psychology*, sv. 8, str. 521, 2017.
- [32] L. Zheng, W.-L. Chiang, Y. Sheng i dr., „Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” *Advances in Neural Information Processing Systems*, 2023.
- [33] P. Wang, L. Li, L. Chen i dr., „Large Language Models are not Fair Evaluators,” *arXiv preprint arXiv:2305.17926*, 2023.

Popis slika

1.	Originalni ispis koda ELIZA pronađen u Weizenbaumovim arhivama [5]	4
2.	Koder - dekode [10]	12
3.	Tokenizacija hrvatskog teksta prema trima generacijama OpenAI modela. Izvor: https://platform.openai.com/tokenizer .	14
4.	Dijagram toka različitih faza razvoja LLM-a od pred-treniranja do korištenja [11]	17
5.	Iskorištenost systemske i GPU memorije te diska tijekom triju uzastopnih sesija izvođenja u okruženju Google Colab (9. 7. 2026.).	32
6.	Prikaz vektoriziranih riječi u dvije dimenzije t-SNE metodom, gdje se semantički srodni pojmovi grupiraju u blizini jedan drugoga.	37
7.	Procesni lanac pripreme podataka za RAG (izgradnja vektorskog indeksa) i GraphRAG (izgradnja grafa) verzije chatbota.	44
8.	Obradni tijek korisničkog upita za Vanilla, RAG i GraphRAG verziju chatbota, do zajedničkog koraka generiranja odgovora.	46
9.	Distribucija ljudskih ocjena po konfiguraciji i dimenziji (medijan jedanaest ocjenjivača po pitanju).	57
10.	Usporedba triju verzija chatbota po prosječnim vrijednostima triju metrika.	58
11.	Usporedba triju verzija chatbota po prosječnim vrijednostima triju metrika, razvrstano po kategorijama pitanja.	58
12.	Toplinska karta metrika po pitanjima, s obojenim oznakama kategorija pitanja na y-osi.	59
13.	Toplinska karta Spearmanove korelacije automatskih metrika s dimenzijama ljudske evaluacije.	62
14.	Odnos LLM ocjena (Claude, Gemini) i ljudskih ocjena, po dimenziji i konfiguraciji.	64
15.	Distribucija latencije odgovora po konfiguraciji.	66

Popis tablica

1.	Dimenzije ljudske evaluacije i primarni evaluatori	52
2.	Usporedba evaluacijskih metoda	55
3.	Deskriptivna statistika ljudskih ocjena po konfiguraciji (skala 1–3, medijan jedanaest ocjenjivača po pitanju)	56
4.	Friedmanov test po dimenzijama ljudske evaluacije	60
5.	Post-hoc upareni Wilcoxonov test predznačenih rangova s Bonferroni korekcijom (ljudska evaluacija)	60
6.	Friedmanov test i post-hoc upareni Wilcoxonov test s Bonferroni korekcijom po automatskim metrikama	61
7.	Spearmanova korelacija (ρ) automatskih metrika s dimenzijama ljudske evaluacije (* $p < 0,05$, ** $p < 0,01$)	61
8.	Međuocjenjivačka pouzdanost (11 ocjenjivača, nakon isključivanja triju blago ocjenjujućih)	62
9.	Spearmanova korelacija LLM ocjenjivača s ljudskom evaluacijom (ρ)	63
10.	Usporedba ocjena Claude i Gemini ocjenjivača (upareni Wilcoxonov test)	64
11.	Statistika latencije odgovora (u sekundama)	65

Prilozi

1. Prilog 1 - Popis pitanja

Po potrebi će se dodati, za sada je zakomentirano jer je predugačko za prikaz u ovom dokumentu.

2. Prilog 2 - Colab - Vanilla verzija ChatBota

Po potrebi će se dodati, za sada je zakomentirano jer je predugačko za prikaz u ovom dokumentu.

3. Prilog 3 - Colab - RAG verzija ChatBota

Po potrebi će se dodati, za sada je zakomentirano jer je predugačko za prikaz u ovom dokumentu.

4. Prilog 4 - Colab - GraphRAG verzija ChatBota

Po potrebi će se dodati, za sada je zakomentirano jer je predugačko za prikaz u ovom dokumentu.